第5章 树与二叉树

【考纲内容】

扫一扫

视频讲解

（一）树的基本概念

（二）二叉树

二叉树的定义及其主要特征；二叉树的顺序存储结构和链式存储结构；二叉树的遍历；线索二叉树的基本概念和构造

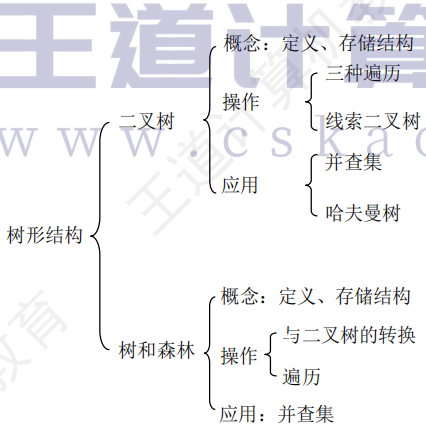
（三）树、森林

树的存储结构；森林与二叉树的转换；树和森林的遍历

（四）树与二叉树的应用

哈夫曼（Huffman）树和哈夫曼编码；并查集及其应用；堆及其应用

【知识框架】



【复习提示】

本章内容多以选择题或综合题的形式考查，但统考也会出涉及树遍历相关的算法题。树和二叉树的性质、遍历操作、转换、存储结构和操作特性等，满二叉树、完全二叉树、线索二叉树、哈夫曼树的定义和性质，都是选择题必然会涉及的内容。

5.1 树的基本概念

5.1.1 树的定义

树是 n（$n \geqslant 0$）个结点的有限集。当 $n=0$ 时，称为空树。在任意一棵非空树中应满足：

1）有且仅有一个特定的称为根的结点。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

2）当 $n>1$ 时，其余结点可分为 m（$m>0$）个互不相交的有限集 $T_1, T_2, \cdots, T_m$，其中每个集合本身又是一棵树，并且称为根的子树。

显然，树的定义是递归的，即在树的定义中又用到了其自身，树是一种递归的数据结构。树作为一种逻辑结构，同时也是一种分层结构，具有以下两个特点：

1）树的根结点没有前驱，除根结点外的所有结点有且只有一个前驱。

2）树中所有结点都可以有零个或多个后继。

树适用于表示具有层次结构的数据。树中的某个结点（除根结点外）最多只和上一层的一个结点（其父结点）有直接关系，根结点没有直接上层结点，因此在 n 个结点的树中有 $n-1$ 条边。而树中每个结点与其下一层的零个或多个结点（其孩子结点）都有直接关系。

5.1.2　基本术语

下面结合图 5.1 中的树来说明一些基本术语和概念。

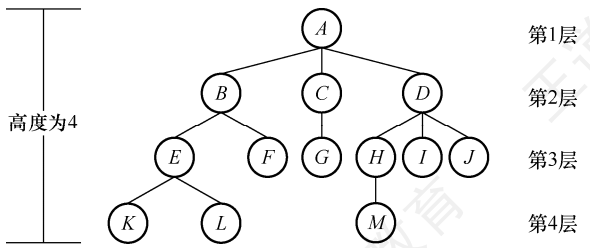


图 5.1　树的树形表示

1）祖先、子孙、双亲、孩子、兄弟和堂兄弟。

考虑结点 K，从根 A 到结点 K 的唯一路径上的所有其他结点，称为结点 K 的祖先。如结点 B 是结点 K 的祖先，而 K 是 B 的子孙，结点 B 的子孙包括 E, F, K, L。路径上最接近结点 K 的结点 E 称为 K 的双亲，而 K 为 E 的孩子。根 A 是树中唯一没有双亲的结点。有相同双亲的结点称为兄弟，如结点 K 和结点 L 有相同的双亲 E，即 K 和 L 为兄弟。双亲在同一层的结点互为堂兄弟，结点 G 与 E, F, H, I, J 互为堂兄弟。

2）结点的层次、深度和高度。

结点的层次从树根开始定义，根结点为第 1 层，它的孩子为第 2 层，以此类推。结点的深度就是结点所在的层次。树的高度（或深度）是树中结点的最大层数。结点的高度是以该结点为根的子树的高度。图 5.1 中树的高度为 4。

3）结点的度和树的度。

树中一个结点的孩子个数称为该结点的度，树中结点的最大度数称为树的度。如结点 B 的度为 2，结点 D 的度为 3，树的度为 3。

4）分支结点和叶结点。

度大于 0 的结点称为分支结点（也称非终端结点）；度为 0（没有孩子结点）的结点称为叶结点（也称终端结点）。在分支结点中，每个结点的分支数就是该结点的度。

5）有序树和无序树。

树中结点的各子树从左到右是有次序的，不能互换，称该树为有序树，否则称为无序树。假设图 5.1 为有序树，若将子结点位置互换，则变成一棵不同的树。

6）路径和路径长度。

树中两个结点之间的路径是由这两个结点之间所经过的结点序列构成的，而路径长度是路径上所经过的边的个数。

配套课程公众号[考机研]
永久联系微信:4550060

注 意

因为树中的分支是有向的，即从双亲指向孩子，所以树中的路径是从上向下的，同一双亲的两个孩子之间不存在路径。

7）森林。

命题追踪 ▶▶ 森林中树的数量、边数和结点数的关系（2016）

森林是 m（$m \geqslant 0$）棵互不相交的树的集合。森林的概念与树的概念十分相近，因为只要把树的根结点删去就成了森林。反之，只要给 m 棵独立的树加上一个结点，并把这 m 棵树作为该结点的子树，则森林就变成了树。

注 意

上述概念无须刻意记忆，根据实例理解即可。考研时不大可能直接考查概念，而都是结合具体的题目考查。做题时，遇到不熟悉的概念可以翻书，练习得多自然就记住了。

5.1.3 树的性质

树具有如下最基本的性质：

命题追踪 ▶▶ 树中结点数和度数的关系的应用（2010、2016）

1）树的结点数 n 等于所有结点的度数之和加 1。

结点的度是指该结点的孩子数量，每个结点与其每个孩子都由唯一的边相连，因此树中所有结点的度数之和等于树中的边数之和。树中的结点（除根外）都有唯一的双亲，因此结点数 n 等于边数之和加 1，即所有结点的度数之和加 1。

2）度为 m 的树中第 i 层上至多有 m^{i-1} 个结点（$i \geqslant 1$）。

第 1 层至多有 1 个结点（根结点），第 2 层至多有 m 个结点，第 3 层至多有 m^2 个结点，以此类推。使用数学归纳法可推出第 i 层至多有 m^{i-1} 个结点。

3）高度为 h 的 m 叉树至多有 $(m^h-1)/(m-1)$ 个结点。

当各层结点数达到最大时，树中至多有 $1+m+m^2+\cdots+m^{h-1}=(m^h-1)/(m-1)$ 个结点。

命题追踪 ▶▶ 指定结点数的三叉树的最小高度分析（2022）

4）度为 m、具有 n 个结点的树的最小高度 h 为 $\lceil \log_m(n(m-1)+1) \rceil$。

为使树的高度最小，在前 $h-1$ 层中，每层的结点数都要达到最大，前 $h-1$ 层最多有 $(m^{h-1}-1)/(m-1)$ 个结点，前 h 层最多有 $(m^h-1)/(m-1)$ 个结点。因此 $(m^{h-1}-1)/(m-1)<n \leqslant (m^h-1)/(m-1)$，即 $h-1<\log_m(n(m-1)+1) \leqslant h$，解得 $h_{\min}=\lceil \log_m(n(m-1)+1) \rceil$。

5）度为 m、具有 n 个结点的树的最大高度 h 为 $n-m+1$。

树的度为 m，因此至少有一个结点有 m 个孩子，它们处于同一层。为使树的高度最大，其他层可仅有一个结点，因此最大高度（层数）为 $n-m+1$。由此，也可逆推出高度为 h、度为 m 的树至少有 $h+m-1$ 个结点。

5.1.4 本节试题精选

一、单项选择题

01. 树最适合用来表示（ ）的数据。

配套课程公众号[考机研]
永久联系微信:4550060

A. 有序　　B. 无序

C. 任意元素之间具有多种联系　　D. 元素之间具有分支层次关系

02. 一棵有 n 个结点的树的所有结点的度数之和为（　）。

A. $n-1$　　B. n　　C. $n+1$　　D. $2n$

03. 树的路径长度是从树根到每个结点的路径长度的（　）。

A. 总和　　B. 最小值　　C. 最大值　　D. 平均值

04. 对于一棵具有 n 个结点、度为 4 的树来说，（　）。

A. 树的高度至多是 $n-3$　　B. 树的高度至多是 $n-4$

C. 第 i 层上至多有 $4(i-1)$个结点　　D. 至少在某一层上正好有 4 个结点

05. 度为 4、高度为 h 的树，（　）。

A. 至少有 $h+3$ 个结点　　B. 至多有 $4h-1$ 个结点

C. 至多有 $4h$ 个结点　　D. 至少有 $h+4$ 个结点

06. 假定一棵度为 3 的树中，结点数为 50，则其最小高度为（　）。

A. 3　　B. 4　　C. 5　　D. 6

07. 设有一棵度为 3 的树，其中度为 3 的结点数 $n_3=2$，度为 2 的结点数 $n_2=1$，叶结点数 $n_0=6$，则该树的结点总数为（　）。

A. 12　　B. 9　　C. 10　　D. ≥9 的任意整数

08. 设一棵 m 叉树中有 N_1 个度数为 1 的结点，N_2 个度数为 2 的结点……N_m 个度数为 m 的结点，则该树中共有（　）个叶结点。

A. $\sum_{i=1}^{m}(i-1)N_i$　　B. $\sum_{i=1}^{m}N_i$　　C. $\sum_{i=2}^{m}(i-1)N_i$　　D. $\sum_{i=2}^{m}(i-1)N_i+1$

09. 【2010 统考真题】在一棵度为 4 的树 T 中，若有 20 个度为 4 的结点，10 个度为 3 的结点，1 个度为 2 的结点，10 个度为 1 的结点，则树 T 的叶结点个数是（　）。

A. 41　　B. 82　　C. 113　　D. 122

10. 【2016 统考真题】若森林 F 有 15 条边、25 个结点，则 F 包含树的个数是（　）。

A. 8　　B. 9　　C. 10　　D. 11

二、综合应用题

01. 含有 n 个结点的三叉树的最小高度是多少？

02. 已知一棵度为 4 的树中，度为 0, 1, 2, 3 的结点数分别为 14, 4, 3, 2，求该树的结点总数 n 和度为 4 的结点个数，并给出推导过程。

03. 已知一棵度为 m 的树中，有 n_1 个度为 1 的结点，有 n_2 个度为 2 的结点……有 n_m 个度为 m 的结点，问该树有多少个叶结点？

5.1.5　答案与解析

一、单项选择题

01. D

树是一种分层结构，它特别适合组织那些具有分支层次关系的数据。

02. A

除根结点外，其他每个结点都是某个结点的孩子，因此树中所有结点的度数加 1 等于结点数，

配套课程公众号[考机研]
永久联系微信:4550060

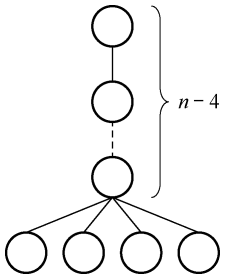
即所有结点的度数之和等于总结点数减1。这是一个重要的结论，做题时经常用到。

03. A

树的路径长度是指树根到每个结点的路径长的总和，根到每个结点的路径长度的最大值应是树的高度减1。注意与哈夫曼树的带权路径长度相区别。

04. A

要使得具有 n 个结点、度为4的树的高度最大，就要使得每层的结点数尽可能少，类似下图所示的树，除最后一层外，每层的结点数是1，最终该树的高度为 $n-3$。树的度为4只能说明存在某结点正好（也最多）有4个孩子结点，选项D错误。



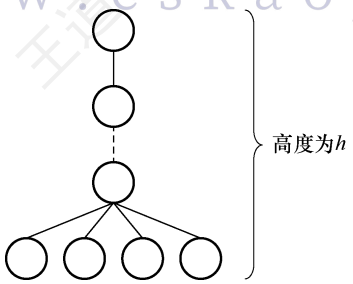
05. A

要使得度为4、高度为 h 的树的总结点数最少，需要满足以下两个条件：

① 至少有一个结点有4个分支。

② 每层的结点数目尽可能少。

情况类似下图所示的树，结点个数为 $h+3$。



要使得度为4、高度为 h 的树的总结点数最多，应使每个非叶结点的度均为4，即满树，总结点个数最多为 $1+4+4^2+\cdots+4^{h-1}$。

对于上面的两题，应画出草图来求解，就能一目了然。

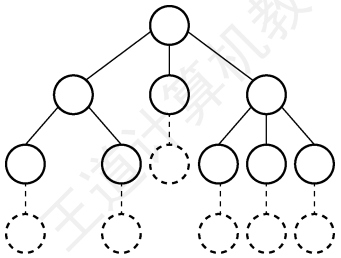
06. C

要求满足条件的树，那么该树是一棵完全三叉树。在度为3的完全三叉树中，第1层有1个结点，第2层有 $3^1=3$ 个结点，第3层有 $3^2=9$ 个结点，第4层有 $3^3=27$ 个结点，因此结点数之和为 $1+3+9+27=40$，第5层的结点数 $=50-40=10$ 个，因此最小高度为5。

07. D

总结点数 $n=n_0+n_1+n_2+n_3=6+n_1+1+2=n_1+9$，总度数 $=n-1=n_1+2n_2+3n_3=n_1+2+6=n_1+8$，根据题目条件无法得出 n 的具体值，只能证明 n 是一个大于或等于9的任意整数。画出满足题目条件的树，可以是如下图所示的一棵树，该树中无法确定 n 的具体数量。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060



08．D

设叶结点数为 N_0，总结点数为 N，则 $N=N_1+2N_2+3N_3+\cdots+mN_m+1$，又因为 $N=N_0+N_1+N_2+N_3+\cdots+N_m$，所以 $N_0=N_2+2N_3+\cdots+(m-1)N_m+1=\sum_{i=2}^{m}(i-1)N_i+1$。

09．B

设树中度为 i（$i=0, 1, 2, 3, 4$）的结点数分别为 n_i，树中结点总数为 n，则 $n=$ 分支数 $+1$，而分支数又等于树中各结点的度之和，即 $n=1+n_1+2n_2+3n_3+4n_4=n_0+n_1+n_2+n_3+n_4$。依题意，$n_1+2n_2+3n_3+4n_4=10+2+30+80=122$，$n_1+n_2+n_3+n_4=10+1+10+20=41$，可得出 $n_0=82$，即树 T 的叶结点的个数是 82。

10．C

解法 1：树有一个重要性质，即在 n 个结点的树中有 $n-1$ 条边，“那么对于每棵树，其结点数比边数多 1”。本题森林中的结点数比边数多 10（$25-15=10$），显然共有 10 棵树。

解法 2：仔细分析后发现此题也是考查图的性质：生成树和生成森林。对于图的生成树有一个重要的性质，即图中顶点数若为 n，则其生成树含有 $n-1$ 条边。对比解法 1 中树的性质，不难发现两种解法都用到了性质“树中结点数比边数多 1”，后面的分析如解法 1。

王道计算机教育 www.cskaoyan.com

二、综合应用题

01．【解答】

要求含有 n 个结点的三叉树的最小高度，那么满足条件的一定是一棵完全三叉树，设含有 n 个结点的完全三叉树的高度为 h，第 h 层至少有 1 个结点，至多有 3^{h-1} 个结点。则有

$$1+3^1+3^2+\cdots+3^{h-2}<n\leqslant 1+3^1+3^2+\cdots+3^{h-2}+3^{h-1}$$

即 $(3^{h-1}-1)/2<n\leqslant(3^h-1)/2$，得 $3^{h-1}<2n+1\leqslant 3^h$，即 $h<\log_3(2n+1)+1$，$h\geqslant\log_3(2n+1)$。

因为 h 只能为正整数，$h=\lceil\log_3(2n+1)\rceil$，所这种三叉树的最小高度是 $\lceil\log_3(2n+1)\rceil$。

02．【解答】

设树中度为 i（$i=0, 1, 2, 3, 4$）的结点数为 n_i，则结点总数 $n=n_0+n_1+n_2+n_3+n_4$，即 $n=23+n_4$，根据“树中所有结点的度数加 1 等于结点数”的结论，有 $n=0+n_1+2n_2+3n_3+4n_4+1$，即有 $n=17+4n_4$。

综合两式得 $n_4=2$，$n=25$。所以该树的结点总数为 25，度为 4 的结点个数为 2。

03．【解答】

树中的结点数等于所有结点的度数加 1，因此有 $n=\sum_{i=0}^{m}in_i+1=n_1+2n_2+3n_3+\cdots+mn_m+1$。

又有 $n=n_0+n_1+n_2+\cdots+n_m$，所以

$$\begin{aligned}n_0&=(n_1+2n_2+3n_3+\cdots+mn_m+1)-(n_1+n_2+\cdots+n_m)\\&=n_2+2n_3+\cdots+(m-1)n_m+1=1+\sum_{i=2}^{m}(i-1)n_i\end{aligned}$$

配套课程公众号[考机研]
永久联系微信:4550060

注 意

综合以上几题，常用于求解树结点与度之间关系的有：

① 总结点数 $= n_0 + n_1 + n_2 + \cdots + n_m$。

② 总分支数 $= 1n_1 + 2n_2 + \cdots + mn_m$（度为 m 的结点引出 m 条分支）。

③ 总结点数 = 总分支数 + 1。

这类题目常在选择题中出现，读者对以上关系应当熟练掌握并灵活应用。

5.2 二叉树的概念

5.2.1 二叉树的定义及其主要特性

1．二叉树的定义

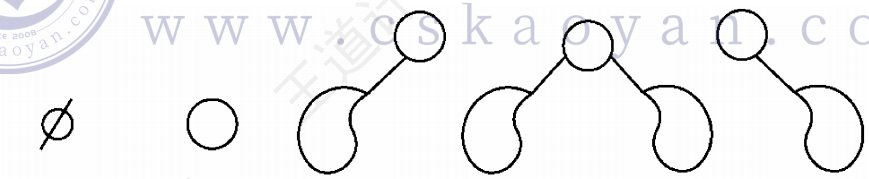
二叉树是一种特殊的树形结构，其特点是每个结点至多只有两棵子树（二叉树中不存在度大于 2 的结点），并且二叉树的子树有左右之分，其次序不能任意颠倒。

与树相似，二叉树也以递归的形式定义。二叉树是 n（$n \geqslant 0$）个结点的有限集合：

① 或者为空二叉树，即 $n = 0$。

② 或者由一个根结点和两个互不相交的被称为根的左子树和右子树组成。左子树和右子树又分别是一棵二叉树。

二叉树是有序树，若将其左、右子树颠倒，则成为另一棵不同的二叉树。即使树中结点只有一棵子树，也要区分它是左子树还是右子树。二叉树的 5 种基本形态如图 5.2 所示。



(a) 空二叉树　(b) 只有根结点　(c) 只有左子树　(d) 左右子树都有　(e) 只有右子树

图 5.2　二叉树的 5 种基本形态

二叉树与度为 2 的有序树的区别：

① 度为 2 的树至少有 3 个结点，而二叉树可以为空。

② 度为 2 的有序树的孩子的左右次序是相对于另一个孩子而言的，若某个结点只有一个孩子，则这个孩子就无须区分其左右次序，而二叉树无论其孩子数是否为 2，均需确定其左右次序，即二叉树的结点次序不是相对于另一结点而言的，而是确定的。

2．几种特殊的二叉树

1）满二叉树。一棵高度为 h，且有 $2^h - 1$ 个结点的二叉树称为满二叉树，即二叉树中的每层都含有最多的结点，如图 5.3(a)所示。满二叉树的叶结点都集中在二叉树的最下一层，并且除叶结点之外的每个结点度数均为 2。

可以对满二叉树按层序编号：约定编号从根结点（根结点编号为 1）起，自上而下，自左向右。这样，每个结点对应一个编号，对于编号为 i 的结点，若有双亲，则其双亲为$\lfloor i/2 \rfloor$，若有左孩子，则左孩子为 $2i$；若有右孩子，则右孩子为 $2i + 1$。

配套课程公众号[考机研]
永久联系微信:4550060

命题追踪 ▶▶ 完全二叉树中结点数和叶结点数的关系（2009、2011、2018）

2）完全二叉树。高度为 h、有 n 个结点的二叉树，当且仅当其每个结点都与高度为 h 的满二叉树中编号为 1～n 的结点一一对应时，称为完全二叉树，如图 5.3(b)所示。

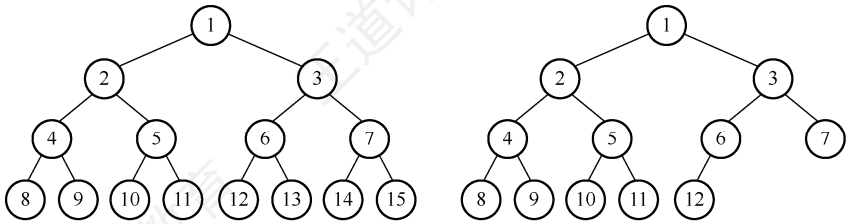


图 5.3　两种特殊形态的二叉树[①]

3）二叉排序树。左子树上所有结点的关键字均小于根结点的关键字；右子树上所有结点的关键字均大于根结点的关键字；左子树和右子树又各是一棵二叉排序树。

4）平衡二叉树。树中任意一个结点的左子树和右子树的高度之差的绝对值不超过 1。关于二叉排序树和平衡二叉树的详细介绍，见本书中的 7.3 节。

命题追踪 ▶▶ 正则 k 叉树树高和结点数的关系的应用（2016）

5）正则二叉树。树中每个分支结点都有 2 个孩子，即树中只有度为 0 或 2 的结点。

3．二叉树的性质

1）非空二叉树上的叶结点数等于度为 2 的结点数加 1，即 $n_0=n_2+1$。

证明：设度为 0, 1 和 2 的结点个数分别为 n_0, n_1 和 n_2，结点总数 $n=n_0+n_1+n_2$。

再看二叉树中的分支数，除根结点外，其余结点都有一个分支进入，设 B 为分支总数，则 $n=B+1$。这些分支是由度为 1 或 2 的结点射出的，因此又有 $B=n_1+2n_2$。

于是得 $n_0+n_1+n_2=n_1+2n_2+1$，则 $n_0=n_2+1$。

注　意

该性质经常在选择题中涉及，希望读者牢记并灵活应用。

2）非空二叉树的第 k 层最多有 2^{k-1} 个结点（$k\geqslant 1$）。

第 1 层最多有 $2^{1-1}=1$ 个结点（根），第 2 层最多有 $2^{2-1}=2$ 个结点，以此类推，可以证明其为一个公比为 2 的等比数列 2^{k-1}。

3）高度为 h 的二叉树至多有 2^h-1 个结点（$h\geqslant 1$）。

该性质利用性质 2 求前 h 项的和，即等比数列求和的结果。

注　意

性质 2 和性质 3 还可以拓展到 m 叉树的情况，即 m 叉树的第 k 层最多有 m^{k-1} 个结点，高度为 h 的 m 叉树至多有 $(m^h-1)/(m-1)$ 个结点。

4）对完全二叉树按从上到下、从左到右的顺序依次编号 1, 2, …, n，则有以下关系：

① 最后一个分支结点的编号为 $\lfloor n/2\rfloor$，若 $i\leqslant\lfloor n/2\rfloor$，则结点 i 为分支结点，否则为叶结点。

① 完全二叉树可视为从满二叉树中删去若干最底层、最右边的一些连续叶结点后所得到的二叉树。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

② 叶结点只可能在最后两层上出现（相当于在相同高度的满二叉树的最底层、最右边减少一些连续叶结点，当减少 2 个或以上叶结点时，次底层将出现叶结点）。

③ 若有度为 1 的结点，则最多只可能有一个，且该结点只有左孩子而无右孩子（度为 1 的分支结点只可能是最后一个分支结点，其结点编号为$\lfloor n/2 \rfloor$）。

④ 按层序编号后，一旦出现某结点（如编号 i）为叶结点或只有左孩子的情况，则编号大于 i 的结点均为叶结点（与结论①和结论③是相通的）。

⑤ 若 n 为奇数，则每个分支结点都有左、右孩子；若 n 为偶数，则编号最大的分支结点（编号为 $n/2$）只有左孩子，没有右孩子，其余分支结点都有左、右孩子。

⑥ 当 $i>1$ 时，结点 i 的双亲结点的编号为$\lfloor i/2 \rfloor$。

⑦ 若结点 i 有左、右孩子，则左孩子编号为 $2i$，右孩子编号为 $2i+1$。

⑧ 结点 i 所在层次（深度）为$\lfloor \log_2 i \rfloor + 1$。

5）具有 n 个（$n>0$）结点的完全二叉树的高度为$\lceil \log_2(n+1) \rceil$或$\lfloor \log_2 n \rfloor + 1$。

设高度为 h，根据性质 3 和完全二叉树的定义有

$$2^{h-1}-1<n\leqslant 2^h-1 \quad \text{或者} \quad 2^{h-1}\leqslant n<2^h$$

得 $2^{h-1}<n+1\leqslant 2^h$，即 $h-1<\log_2(n+1)\leqslant h$，因为 h 为正整数，所以 $h=\lceil \log_2(n+1) \rceil$，或者得 $h-1\leqslant \log_2 n<h$，所以 $h=\lfloor \log_2 n \rfloor+1$。

5.2.2　二叉树的存储结构

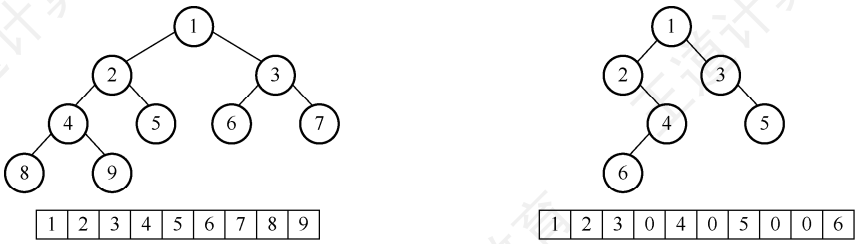
1. 顺序存储结构

二叉树的顺序存储是指用一组连续的存储单元依次自上而下、自左至右存储完全二叉树上的结点元素，即将完全二叉树上编号为 i 的结点元素存储在一维数组下标为 $i-1$ 的分量中。

依据二叉树的性质，完全二叉树和满二叉树采用顺序存储比较合适，树中结点的序号可以唯一地反映结点之间的逻辑关系，这样既能最大可能地节省存储空间，又能利用数组元素的下标值确定结点在二叉树中的位置，以及结点之间的关系。

命题追踪 ▶▶ 特定条件下二叉树树形及占用存储空间的分析（2020）

但对于一般的二叉树，为了让数组下标能反映二叉树中结点之间的逻辑关系，只能添加一些并不存在的空结点，让其每个结点与完全二叉树上的结点相对照，再存储到一维数组的相应分量中。然而，在最坏情况下，一个高度为 h 且只有 h 个结点的单支树却需要占据近 2^h-1 个存储单元。二叉树的顺序存储结构如图 5.4 所示，其中 0 表示并不存在的空结点。



(a) 完全二叉树的顺序存储结构　　(b) 一般二叉树的顺序存储结构

图 5.4　二叉树的顺序存储结构

注　意

建议从数组下标 1 开始存储树中的结点，保证数组下标和结点编号一致。

配套课程公众号[考机研]
永久联系微信:4550060

2．链式存储结构

顺序存储的空间利用率较低，因此二叉树一般都采用链式存储结构，用链表结点来存储二叉树中的每个结点。在二叉树中，结点结构通常包括若干数据域和若干指针域，二叉链表至少包含 3 个域：数据域 data、左指针域 lchild 和右指针域 rchild，如图 5.5 所示。

图 5.5　二叉树链式存储的结点结构

图 5.6 所示为一棵二叉树及其对应的二叉链表。而实际上在不同的应用中，还可以增加某些指针域，如增加指向父结点的指针后，变为三叉链表的存储结构。

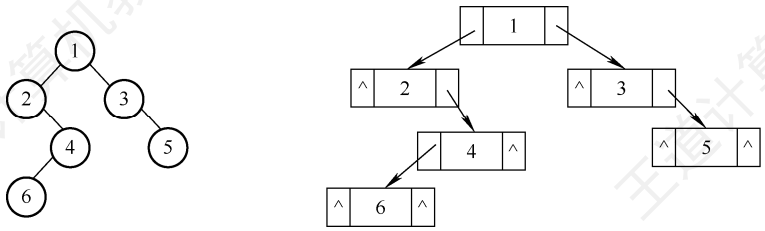


图 5.6　二叉链表的存储结构

二叉树的链式存储结构描述如下：

```
typedef struct BiTNode{
    ElemType data;                  //数据域
    struct BiTNode *lchild,*rchild;    //左、右孩子指针
}BiTNode,*BiTree;
```

使用不同的存储结构时，实现二叉树操作的算法也会不同，因此要根据实际应用场合（二叉树的形态和需要进行的运算）来选择合适的存储结构。

容易验证，在含有 n 个结点的二叉链表中，含有 $n+1$ 个空链域（重要结论，经常出现在选择题中）。在下一节中，我们将利用这些空链域来组成另一种链表结构——线索链表。

5.2.3　本节试题精选

一、单项选择题

01. 下列关于二叉树的说法中，正确的是（　）。

A. 度为 2 的有序树就是二叉树

B. 含有 n 个结点的二叉树的高度为 $\lfloor \log_2 n \rfloor + 1$

C. 在完全二叉树中，若一个结点没有左孩子，则它必是叶结点

D. 含有 n 个结点的完全二叉树的高度为 $\lceil \log_2 n \rceil$

02. “二叉树为空”意味着二叉树（　）。

A. 根结点没有子树　　B. 不存在

C. 没有结点　　D. 由一些没有赋值的空结点构成

03. 下列关于完全二叉树的说法中，正确的是（　）。

A. 在完全二叉树中，叶结点的双亲的左兄弟（若存在）一定不是叶结点

B. 任何一棵二叉树中，叶结点数为度为 2 的结点数减 1，即 $n_0 = n_2 - 1$

C. 完全二叉树不适合顺序存储结构，只有满二叉树适合顺序存储结构

D. 结点按完全二叉树层序编号的二叉树中，第 i 个结点的左孩子的编号为 $2i$

配套课程公众号[考机研]
永久联系微信:4550060

04. 具有10个叶结点的二叉树中有（ ）个度为2的结点。

A. 8　　B. 9　　C. 10　　D. 11

05. 设高度为 h 的二叉树上只有度为0和度为2的结点，则此类二叉树中所包含的结点数至少为（ ）。

A. h　　B. $2h-1$　　C. $2h+1$　　D. $h+1$

06. 具有 n 个结点且高度为 n 的二叉树的数目为（ ）。

A. $\log_2 n$　　B. $n/2$　　C. n　　D. 2^{n-1}

07. 假设一棵二叉树的结点个数为50，则它的最小高度是（ ）。

A. 4　　B. 5　　C. 6　　D. 7

08. 设二叉树有 $2n$ 个结点，且 $m<n$，则不可能存在（ ）的结点。

A. n 个度为0　　B. $2m$ 个度为0　　C. $2m$ 个度为1　　D. $2m$ 个度为2

09. 一个具有1025个结点的二叉树的高 h 为（ ）。

A. 11　　B. 10　　C. 11～1025　　D. 10～1024

10. 设二叉树只有度为0和2的结点，其结点个数为15，则该二叉树的最大深度为（ ）。

A. 4　　B. 5　　C. 8　　D. 9

11. 高度为 h 的完全二叉树最少有（ ）个结点。

A. 2^h　　B. 2^h+1　　C. 2^{h-1}　　D. 2^h-1

12. 已知一棵完全二叉树的第6层（设根为第1层）有8个叶结点，则完全二叉树的结点个数最少是（ ）。

A. 39　　B. 52　　C. 111　　D. 119

13. 若一棵深度为6的完全二叉树的第6层有3个叶结点，则该二叉树共有（ ）个叶结点。

A. 17　　B. 18　　C. 19　　D. 20

14. 一棵完全二叉树上有1001个结点，其中叶结点的个数是（ ）。

A. 250　　B. 500　　C. 254　　D. 501

15. 若一棵二叉树有126个结点，在第7层（根结点在第1层）至多有（ ）个结点。

A. 32　　B. 64　　C. 63　　D. 不存在第7层

16. 一棵有124个叶结点的完全二叉树，最多有（ ）个结点。

A. 247　　B. 248　　C. 249　　D. 250

17. 某完全二叉树 T 中，结点个数最大的层有8个结点，则 T 中至多有（ ）个结点。

A. 8　　B. 15　　C. 23　　D. 31

18. 一棵有 n 个结点的二叉树采用二叉链存储结点，其中空指针数为（ ）。

A. n　　B. $n+1$　　C. $n-1$　　D. $2n$

19. 设有 n（$n\geq 1$）个结点的二叉树采用三叉链表表示，其中每个结点包含三个指针，分别指向其左孩子、右孩子及双亲（若不存在，则置为空），则下列说法中正确的是（ ）。

I. 树中空指针的数量为 $n+2$　　II. 所有度为2的结点均被三个指针指向

III. 每个叶结点均被一个指针所指向

A. I　　B. I、II　　C. I、III　　D. II、III

20. 在一棵完全二叉树中，其根的序号为1，（ ）可判定序号为 p 和 q 的两个结点是否在同一层。

A. $\lfloor\log_2 p\rfloor=\lfloor\log_2 q\rfloor$　　B. $\log_2 p=\log_2 q$

C. $\lfloor\log_2 p\rfloor+1=\lfloor\log_2 q\rfloor$　　D. $\lfloor\log_2 p\rfloor=\lfloor\log_2 q\rfloor+1$

21. 在一个用数组表示的完全二叉树中，根结点的下标为1，那么下标为17和19的结点的

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

最近公共祖先的下标是（　）。

A. 1　　B. 2　　C. 4　　D. 8

22. 假定一棵三叉树的结点数为 50，则它的最小高度为（　）。

A. 3　　B. 4　　C. 5　　D. 6

23. 具有 n 个结点的三叉树用三叉链表表示，则树中空指针域的个数为（　）。

A. $3n+1$　　B. $2n+1$　　C. $3n-1$　　D. $3n$

24. 对于一棵满二叉树，共有 n 个结点和 m 个叶结点，高度为 h，则（　）。

A. $n=h+m$　　B. $n+m=2h$　　C. $m=h-1$　　D. $n=2^h-1$

25. 【2009 统考真题】已知一棵完全二叉树的第 6 层（设根为第 1 层）有 8 个叶结点，则该完全二叉树的结点个数最多是（　）。

A. 39　　B. 52　　C. 111　　D. 119

26. 【2011 统考真题】若一棵完全二叉树有 768 个结点，则该二叉树中叶结点的个数是（　）。

A. 257　　B. 258　　C. 384　　D. 385

27. 【2018 统考真题】设一棵非空完全二叉树 T 的所有叶结点均位于同一层，且每个非叶结点都有 2 个子结点。若 T 有 k 个叶结点，则 T 的结点总数是（　）。

A. $2k-1$　　B. $2k$　　C. k^2　　D. 2^k-1

28. 【2020 统考真题】对于任意一棵高度为 5 且有 10 个结点的二叉树，若采用顺序存储结构保存，每个结点占 1 个存储单元（仅存放结点的数据信息），则存放该二叉树需要的存储单元数量至少是（　）。

A. 31　　B. 16　　C. 15　　D. 10

29. 【2022 统考真题】若三叉树 T 中有 244 个结点（叶结点的高度为 1），则 T 的高度至少是（　）。

A. 8　　B. 7　　C. 6　　D. 5

王道计算机教育 www.cskaoyan.com

二、综合应用题

01. 在一棵完全二叉树中，含有 n_0 个叶结点，当度为 1 的结点数为 1 时，该树的高度是多少？当度为 1 的结点数为 0 时，该树的高度是多少？

02. 一棵有 n 个结点的满二叉树有多少个分支结点和多少个叶结点？该满二叉树的高度是多少？

03. 已知完全二叉树的第 9 层有 240 个结点，则整个完全二叉树有多少个结点？有多少个叶结点？

04. 一棵高度为 h 的满 m 叉树有如下性质：根结点所在层次为第 1 层，第 h 层上的结点都是叶结点，其余各层上每个结点都有 m 棵非空子树，若按层次自顶向下，同一层自左向右，顺序从 1 开始对全部结点进行编号，试问：

1）各层的结点个数是多少？

2）编号为 i 的结点的双亲结点（若存在）的编号是多少？

3）编号为 i 的结点的第 k 个孩子结点（若存在）的编号是多少？

4）编号为 i 的结点有右兄弟的条件是什么？其右兄弟结点的编号是多少？

05. 已知一棵二叉树按顺序存储结构进行存储，设计一个算法，求编号分别为 i 和 j 的两个结点的最近的公共祖先结点的值。

06. 【2016 统考真题】若一棵非空 k（$k\geqslant 2$）叉树 T 中的每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题并给出推导过程。

1）若 T 有 m 个非叶结点，则 T 中的叶结点有多少个？

2）若 T 的高度为 h（单结点的树 $h=1$），则 T 的结点数最多为多少个？最少为多少个？

配套课程公众号[考机研]
永久联系微信:4550060

5.2.4 答案与解析

一、单项选择题

01. C

在二叉树中，若某个结点只有一个孩子，则这个孩子的左右次序是确定的；而在度为2的有序树中，若某个结点只有一个孩子，则这个孩子就无须区分其左右次序，选项A错误。选项B仅当是完全二叉树时才有意义，对于任意一棵二叉树，高度可能为$\lfloor\log_2 n\rfloor+1$～n。在完全二叉树中，若有度为1的结点，则只可能有一个，且该结点只有左孩子而无右孩子，选项C正确。完全二叉树的高度为$\lceil\log_2(n+1)\rceil$或$\lfloor\log_2 n\rfloor+1$，也可通过举例$n=4$来排除，选项D错误。

02. C

“二叉树为空”意味着二叉树中没有结点，但并不意味着二叉树不存在。注意，线性表可以是空表，树可以是空树，但图不能是空图（图中不能没有结点）。

03. A

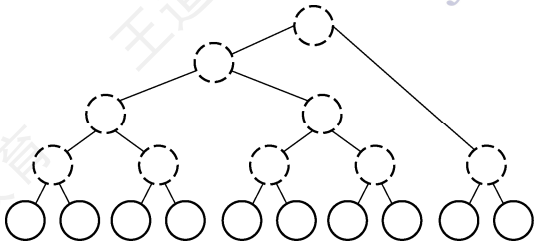
在完全二叉树中，叶结点的双亲的左兄弟的孩子一定在其前面（且一定存在），所以双亲的左兄弟（若存在）一定不是叶结点，选项A正确。n_0应等于n_2+1，选项B错误。完全二叉树和满二叉树均可以采用顺序存储结构，选项C错误。第i个结点的左孩子不一定存在，选项D错误。

选项B的这种通用公式适用于所有二叉树，我们应能立即联想到采用特殊值代入法验证，如画一个只含3个结点的满二叉树的草图来验证是否满足条件。

04. B

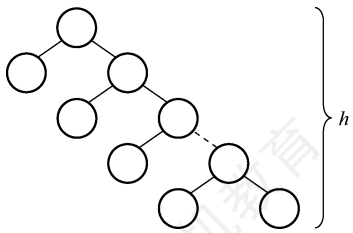
由二叉树的性质$n_0=n_2+1$，得$n_2=n_0-1=10-1=9$。

【另解】画出草图，如下图所示。首先画出10个叶结点，然后每2个结点向上合并，构造一个新的度为2的分支结点，直到构成如下图所示的二叉树，其中度为2的分支结点数为9。



05. B

结点最少的情况如下图所示。除根结点层只有1个结点外，其他$h-1$层均有两个结点，结点总数$=2(h-1)+1=2h-1$。



06. D

除根结点外，在其余$n-1$个结点中，每个结点要么是其父结点的左孩子，要么是其父结点的右孩子，每个结点都有两种可能，$n-1$个结点共有2^{n-1}种不同的组合形态。

07. C

配套课程公众号[考机研]
永久联系微信:4550060

要求满足条件的树，分析可知当这 50 个结点构成一棵完全二叉树时高度最小，$h=\lfloor\log_2 n\rfloor+1=\lfloor\log_2 50\rfloor+1=6$。

【另解】第 1 层最多有 1 个结点，第 2 层最多有 2^1 个结点，第 3 层最多有 2^2 个结点，第 4 层最多有 2^3 个结点，以此类推，可以得到 h 最少为 6。

08．C

由二叉树的性质 1 可知 $n_0=n_2+1$，结点总数 $=2n=n_0+n_1+n_2=n_1+2n_2+1$，则 $n_1=2(n-n_2)-1$，所以 n_1 为奇数，说明该二叉树中不可能有 $2m$ 个度为 1 的结点。

09．C

当二叉树为单支树时具有最大高度，即每层上只有一个结点，最大高度为 1025。而当树为完全二叉树时，其高度最小，最小高度为 $\lfloor\log_2 n\rfloor+1=11$。

10．C

建议画图，第一层有 1 个结点，其余 $h-1$ 层各有 2 个结点，总结点数 $=1+2(h-1)=15$，$h=8$。

11．C

高度为 h 的完全二叉树中，第 1 层～第 $h-1$ 层构成一个高度为 $h-1$ 的满二叉树，结点个数为 $2^{h-1}-1$。第 h 层至少有一个结点，所以最少的结点个数 $=(2^{h-1}-1)+1=2^{h-1}$。

12．A

第 6 层有叶结点说明完全二叉树的高度可能为 6 或 7，显然树高为 6 时结点最少。若第 6 层上有 8 个叶结点，则前 5 层为满二叉树，所以完全二叉树的结点个数最少为 $2^5-1+8=39$ 个结点。

13．A

深度为 6 的完全二叉树，第 5 层共有 $2^4=16$ 个结点。第 6 层最左边有 3 个叶结点，其对应的双亲结点为第 5 层最左边的两个结点，所以第 5 层剩余的结点均为叶结点，共有 $16-2=14$ 个，加上第 6 层的 3 个叶结点，共有 17 个叶结点。

14．D

由完全二叉树的性质，最后一个分支结点的序号为 $\lfloor 1001/2\rfloor=500$，所以叶结点个数为 501。

【另解】$n=n_0+n_1+n_2=n_0+n_1+(n_0-1)=2n_0+n_1-1$，因为 $n=1001$，而在完全二叉树中，n_1 只能取 0 或 1。当 $n_1=1$ 时，n_0 为小数，不符合题意。所以 $n_1=0$，于是有 $n_0=501$。

15．C

要使二叉树第 7 层的结点数最多，只考虑树高为 7 层的情况，7 层满二叉树有 127 个结点，126 仅比 127 少 1 个结点，只能少在第 7 层，所以第 7 层最多有 $2^6-1=63$ 个结点。

16．B

在非空的二叉树当中，由度为 0 和 2 的结点数的关系 $n_0=n_2+1$ 可知 $n_2=123$；总结点数 $n=n_0+n_1+n_2=247+n_1$，其最大值为 248（n_1 的取值为 1 或 0，当 $n_1=1$ 时结点最多）。注意，由完全二叉树总结点数的奇偶性可以确定 n_1 的值，但不能根据 n_0 来确定 n_1 的值。

【另解】$124<2^7=128$，所以第 8 层没满，前 7 层为完全二叉树，由此可推算第 8 层可能有 120 个叶结点，第 7 层的最右 4 个为叶结点，考虑最多的情况，这 4 个叶结点中的最左边可以有 1 个左孩子（不改变叶结点数），因此结点总数 $=2^7-1+120+1=248$。

17．C

在完全二叉树中，第 4 层刚好最多有 8 个结点（前 4 层对应高度为 4 的满二叉树），若第 5 层也有 8 个结点，则对应于结点个数最多的情况，此时树高为 5，总结点数为 $15+8=23$。

18．B

非空指针数 = 总分支数 $=n-1$，空指针数 = 2×结点总数 − 非空指针数 $=2n-(n-1)=n+1$。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

【另解】在树中，1个指针对应1个分支，n个结点的树共有$n-1$个分支，即$n-1$个非空指针，每个结点都有2个指针域，所以空指针数$=2n-(n-1)=n+1$。

19．A

二叉链表表示的二叉树中空指针的数量为$n+1$，三叉链表表示的二叉树多了一个根结点指向双亲的空指针，所以树中空指针的数量为$n+2$，选项I正确。若根结点的度为2，则只有左、右两个孩子指向它，选项II错误。若整棵树只有一个根结点，则没有指针指向它，选项III错误。

20．A

由完全二叉树的性质，编号为i（$i\geqslant 1$）的结点所在的层次为$\lfloor \log_2 i \rfloor+1$，若两个结点位于同一层，则一定有$\lfloor \log_2 p \rfloor+1=\lfloor \log_2 q \rfloor+1$，因此有$\lfloor \log_2 p \rfloor=\lfloor \log_2 q \rfloor$成立。

21．C

当根结点下标为1时，下标为i的结点的父结点下标为$\lfloor i/2 \rfloor$，那么下标为17的祖先的下标有8, 4, 2, 1，下标为19的祖先的下标有9, 4, 2, 1，因此两者最近的公共祖先的下标是4。

22．C

分析可知，满足条件的三叉树可以是完全三叉树，这棵树的第i（$i\geqslant 1$）层最多有3^{i-1}个结点。设高度为h，则$3^0+3^1+\cdots+3^{h-1}=(3^h-1)/2$是结点数的上限，问题是求解$50\leqslant(3^h-1)/2$的最小$h$值，即$h\geqslant \log_3 101$，有$h=\lceil \log_3 101 \rceil=5$。

23．B

三叉树采用三叉链表表示，每个结点均有3个指针域指向3个孩子，共有$3n$个指针域，但n个结点构成的一棵树中只需要$n-1$个指针（对于$n-1$条边），因此空指针域有$2n+1$个。

24．D

对于高度为h的满二叉树，结点总数$n=2^0+2^1+\cdots+2^{h-1}=2^h-1$，叶结点数$m=2^{h-1}$。

25．C

第6层有叶结点，完全二叉树的高度可能为6或7，显然树高为7时结点最多。完全二叉树与满二叉树相比，只是在最下一层的右边缺少部分叶结点，而最后一层之上是个满二叉树，且只有最后两层上有叶结点。若第6层上有8个叶结点，则前6层为满二叉树，而第7层缺失$8\times 2=16$个叶结点，所以完全二叉树的结点个数最多为$2^7-1-16=111$。

26．C

最后一个分支结点的编号为$\lfloor 768/2 \rfloor=384$，所以叶结点的个数为$768-384=384$。

【另解】$n=n_0+n_1+n_2=n_0+n_1+(n_0-1)=2n_0+n_1-1$，其中$n=768$，而在完全二叉树中，$n_1$只能取0或1，当$n_1=0$时，$n_0$为小数，不符合题意。因此$n_1=1$，所以$n_0=384$。

27．A

非叶结点的度均为2，且所有叶结点都位于同一层的完全二叉树就是满二叉树。对于一棵高度为h的满二叉树（空树$h=0$），其最后一层全部是叶结点，数目为2^{h-1}；总结点数为2^h-1。因此当$2^{h-1}=k$时，可以得到$2^h-1=2k-1$。

28．A

二叉树采用顺序存储时，用数组下标来表示结点之间的父子关系。对于一棵高度为5的二叉树，为了满足任意性，其1～5层的所有结点都要被存储起来，即考虑为一棵高度为5的满二叉树，共需要存储单元的数量为$1+2+4+8+16=31$。

29．C

高度一定的三叉树中结点数最多的情况是满三叉树。高度为5的满三叉树的结点数$=3^0+3^1+3^2+3^3+3^4=121$，高度为6的满三叉树的结点数$=3^0+3^1+3^2+3^3+3^4+3^5=364$。三叉树$T$的结点数为244，$121<244<364$，因此$T$的高度至少为6。

王道计算机教育 www.wsbkaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

二、综合应用题

01.【解答】

在非空的二叉树中，由度为 0 和度为 2 的结点之间的关系 $n_0=n_2+1$，可知 $n_2=n_0-1$。因此总结点数 $n=n_0+n_1+n_2=2n_0+n_1-1$。

①当 $n_1=1$ 时，$n=2n_0$，$h=\lceil\log_2(n+1)\rceil=\lceil\log_2(2n_0+1)\rceil$。

②当 $n_1=0$ 时，$n=2n_0-1$，$h=\lceil\log_2(n+1)\rceil=\lceil\log_2(2n_0)\rceil=\lceil\log_2(n_0)\rceil+1$。

02.【解答】

满二叉树中 $n_1=0$，由二叉树的性质 1 可知 $n_0=n_2+1$，即 $n_2=n_0-1$，$n=n_0+n_1+n_2=2n_0-1$，则 $n_0=(n+1)/2$。分支结点个数 $n_2=n-(n+1)/2=(n-1)/2$。高度为 h 的满二叉树的结点数 $n=1+2^1+2^2+\cdots+2^{h-1}=2^h-1$，即高度 $h=\log_2(n+1)$。

03.【解答】

在完全二叉树中，若第 9 层是满的，则结点数 $=2^{9-1}=256$，而现在第 9 层只有 240 个结点，说明第 9 层未满，是最后一层。1～8 层是满的，所以总结点数 $=2^8-1+240=495$。

因为第 9 层是最后一层，所以第 9 层的结点都是叶结点。且第 9 层的 240 个结点的双亲在第 8 层中，其双亲个数为 120，即第 8 层有 120 个分支结点，其余为叶结点，所以第 8 层的叶结点个数为 $2^{8-1}-120=8$。因此，总的叶结点个数 $=8+240=248$。

【另解】总结点数 $n=n_0+n_1+n_2$，$n_2=n_0-1$，$n=n_0+n_1+n_2=2n_0+n_1-1$。若 $n_1=1$，则 $2n_0+n_1-1=2n_0=495$，不符合；若 $n_1=0$，则 $2n_0+n_1-1=2n_0-1=495$，则 $n_0=248$。

注　意

对于本题，应理解完全二叉树中只有最底层的结点是不满的，其他各层的结点都是满的。

04.【解答】

1）第 1 层有 $m^0=1$ 个结点，第 2 层有 m^1 个结点，第 3 层有 m^2 个结点……一般地，第 i 层有 m^{i-1} 个结点（$1\leqslant i\leqslant h$）。

2）在 m 叉树的情形下，结点 i 的第 1 个子女编号为 $j=(i-1)m+2$，反过来，结点 i 的双亲的编号是 $\lfloor(i-2)/m\rfloor+1$，根结点没有双亲，所以要求 $i>1$。

3）因为结点 i 的第 1 个子女编号为 $(i-1)m+2$，若设该结点子女的序号为 $k=1,2,\cdots,m$，则第 k 个子女结点的编号为 $(i-1)m+k+1$（$1\leqslant k\leqslant m$）。

4）结点 i 不是其双亲的第 m 个子女时才有右兄弟。设其双亲编号为 j，可得 $j=\lfloor(i+m-2)/m\rfloor$，结点 j 的第 m 个子女的编号为 $(j-1)m+m+1=jm+1=\lfloor(i+m-2)/m\rfloor m+1$，所以当结点的编号 $i\leqslant\lfloor(i+m-2)/m\rfloor m$ 时才有右兄弟，右兄弟的编号为 $i+1$。或者，对于任意一个双亲结点 j，其第 m 个子女结点的编号是 $jm+1$，故若不为第 m 个子女结点，则 $(i-1)\%m!=0$。

05.【解答】

首先，必须明确二叉树中任意两个结点必然存在最近的公共祖先结点，最坏的情况下是根结点（两个结点分别在根结点的左右分支中），而且从最近的公共祖先结点到根结点的全部祖先结点都是公共的。由二叉树顺序存储的性质可知，任意一个结点 i 的双亲结点的编号为 $i/2$。求解 i 和 j 最近公共祖先结点的算法步骤如下（设从数组下标 1 开始存储）：

1）若 $i>j$，则结点 i 所在层次大于或等于结点 j 所在层次。结点 i 的双亲结点为结点 $i/2$，若 $i/2=j$，则结点 $i/2$ 是原结点 i 和结点 j 的最近公共祖先结点，若 $i/2\neq j$，则令 $i=i/2$，即以该结点 i 的双亲结点为起点，采用递归的方法继续查找。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

2）若 $j>i$，则结点 j 所在层次大于或等于结点 i 所在层次。结点 j 的双亲结点为结点 $j/2$，若 $j/2=i$，则结点 $j/2$ 是原结点 i 和结点 j 的最近公共祖先结点，若 $j/2\neq i$，则令 $j=j/2$。重复上述过程，直到找到它们最近的公共祖先结点为止。

本题代码如下：

```
ElemType Comm_Ancestor(SqTree T,int i,int j){
//本算法在二叉树中查找结点 i 和结点 j 的最近公共祖先结点
    if(T[i]!='#'&&T[j]!='#'){          //结点存在
        while(i!=j){                   //两个编号不同时循环
            if(i>j)
                i=i/2;                 //向上找 i 的祖先
            else
                j=j/2;                 //向上找 j 的祖先
        }
        return  T[i];
    }
}
```

由解题中算法的步骤描述可知，本题也很容易地联想到采用递归的方法求解。

06.【解答】

1）正则 k 叉树中仅含有两类结点：叶结点（个数记为 n_0）和度为 k 的分支结点（个数记为 n_k）。树 T 中的结点总数 $n=n_0+n_k=n_0+m$。树中所含的边数 $e=n-1$，这些边均是从 m 个度为 k 的结点发出的，即 $e=mk$。整理得 $n_0+m=mk+1$，所以 $n_0=(k-1)m+1$。

2）高度为 h 的正则 k 叉树 T 中，含最多结点的树形为：除第 h 层外，第 1 到第 $h-1$ 层的结点都是度为 k 的分支结点；而第 h 层均为叶结点，即树是“满”树。此时第 j（$1\leqslant j\leqslant h$）层的结点数为 k^{j-1}，结点总数 M_1 为

$$M_1=\sum_{j=1}^{h}k^{j-1}=\frac{k^h-1}{k-1}$$

含最少结点的正则 k 叉树的树形为：第 1 层只有根结点，第 2 到第 $h-1$ 层仅含 1 个分支结点和 $k-1$ 个叶结点，第 h 层有 k 个叶结点。也就是说，除根外，第 2 到第 h 层中每层的结点数均为 k，所以 T 中所含结点总数 M_2 为

$$M_2=1+(h-1)k$$

5.3 二叉树的遍历和线索二叉树

5.3.1 二叉树的遍历

二叉树的遍历是指按某条搜索路径访问树中每个结点，使得每个结点均被访问一次，而且仅被访问一次。二叉树是一种非线性结构，每个结点都可能有两棵子树，因此需要寻找一种规律，以便使二叉树上的结点能排列在一个线性队列上，进而便于遍历。

命题追踪 ▶▶ 二叉树遍历方式的分析（2009、2011、2012）

命题追踪 ▶▶ （算法题）二叉树遍历的相关应用（2014、2017、2022）

由二叉树的递归定义可知，遍历一棵二叉树便要决定对根结点 N、左子树 L 和右子树 R 的访问顺序。按照先遍历左子树再遍历右子树的原则，常见的遍历次序有先序（NLR）、中序（LNR）和后序（LRN）三种遍历算法，其中序指的是根结点在何时被访问。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

1．先序遍历（PreOrder）

若二叉树为空，则什么也不做；否则，

1）访问根结点；

2）先序遍历左子树；

3）先序遍历右子树。

图 5.7 中的虚线表示对该二叉树进行先序遍历的路径，得到先序遍历序列为 124635。

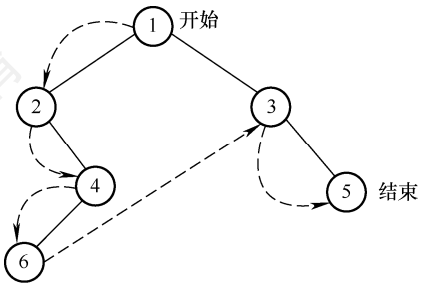


图 5.7　二叉树的先序遍历

对应的递归算法如下：

```
void PreOrder(BiTree T){
    if(T!=NULL){
        visit(T);                    //访问根结点
        PreOrder(T->lchild);         //递归遍历左子树
        PreOrder(T->rchild);         //递归遍历右子树
    }
}
```

2．中序遍历（InOrder）

若二叉树为空，则什么也不做；否则，

1）中序遍历左子树；

2）访问根结点；

3）中序遍历右子树。

命题追踪 ▶▶ 中序序列中结点关系的分析（2017、2024）

图 5.8 中的虚线表示对该二叉树进行中序遍历的路径，得到中序遍历序列为 264135。

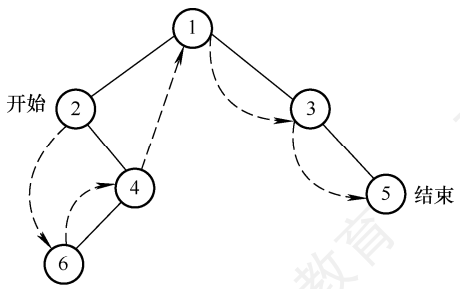


图 5.8　二叉树的中序遍历

对应的递归算法如下：

```
void InOrder(BiTree T){
    if(T!=NULL){
        InOrder(T->lchild);          //递归遍历左子树
```

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

```
            visit(T);                    //访问根结点
            InOrder(T->rchild);          //递归遍历右子树
        }
    }
```

3．后序遍历（PostOrder）

若二叉树为空，则什么也不做；否则，

1）后序遍历左子树；

2）后序遍历右子树；

3）访问根结点。

图 5.9 中的虚线表示对该二叉树进行后序遍历的路径，得到后序遍历序列为 642531。

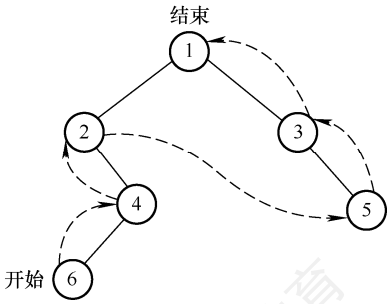


图 5.9　二叉树的后序遍历

对应的递归算法如下：

```
void PostOrder(BiTree T){
    if(T!=NULL){
        PostOrder(T->lchild);    //递归遍历左子树
        PostOrder(T->rchild);    //递归遍历右子树
        visit(T);                //访问根结点
    }
}
```

上述三种遍历算法中，递归遍历左、右子树的顺序都是固定的，只是访问根结点的顺序不同。不管采用哪种遍历算法，每个结点都访问一次且仅访问一次，所以时间复杂度都是 $O(n)$。在递归遍历中，递归工作栈的栈深恰好为树的深度，所以在最坏情况下，二叉树是有 n 个结点且深度为 n 的单支树，遍历算法的空间复杂度为 $O(n)$。

4．层次遍历

图 5.10 所示为二叉树的层次遍历，即按照箭头所指方向，按照 1, 2, 3, 4 的层次顺序，自上而下、从左至右对二叉树中的各个结点进行逐层访问。

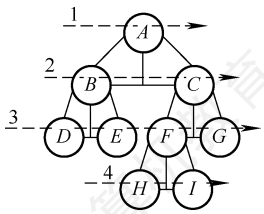


图 5.10　二叉树的层次遍历

进行层次遍历时，需要借助一个队列。层次遍历的思想如下：①首先将根结点入队。②若队

配套课程公众号[考机研]
永久联系微信:4550060

列非空，则队头结点出队，访问该结点，若它有左孩子，则将其左孩子入队；若它有右孩子，则将其右孩子入队。③重复步骤②，直至队列为空。

二叉树的层次遍历算法如下：

```
void LevelOrder(BiTree T){
    InitQueue(Q);                    //初始化辅助队列
    BiTree p;
    EnQueue(Q,T);                    //将根结点入队
    while(!IsEmpty(Q)){              //队列不空则循环
        DeQueue(Q, p);               //队头结点出队
        visit(p);                    //访问出队结点
        if(p->lchild!=NULL)
            EnQueue(Q,p->lchild);    //若左孩子不空，则左孩子入队
        if(p->rchild!=NULL)
            EnQueue(Q,p->rchild);    //若右孩子不空，则右孩子入队
    }
}
```

在复习过程中，读者应将上述二叉树层次遍历的算法作为一个模板，熟练掌握其执行过程，并达到熟练手写的程度。

注　意

遍历是二叉树各种操作的基础，例如对于一棵给定二叉树求结点的双亲、求结点的孩子、求二叉树的深度、求叶结点个数、判断两棵二叉树是否相同等。所有这些操作都是在遍历的过程中进行的，因此必须掌握二叉树的各种遍历过程，并能灵活运用以解决各种问题。

5. 由遍历序列构造二叉树

命题追踪 ▶▶ 先序序列对应的不同二叉树的分析（2015）

对于一棵给定的二叉树，其先序序列、中序序列、后序序列和层序序列都是确定的。然而，只给出四种遍历序列中的任意一种，却不能唯一地确定一棵二叉树。若已知中序序列，再给出其他三种遍历序列中的任意一种，就可以唯一地确定一棵二叉树。

（1）由先序序列和中序序列构造二叉树

命题追踪 ▶▶ 先序序列和中序序列相同时确定的二叉树（2017）

命题追踪 ▶▶ 由先序序列和中序序列构造一棵二叉树（2020、2021）

在先序序列中，第一个结点一定是二叉树的根结点；而在中序遍历中，根结点必然将中序序列分割成两个子序列，前一个子序列是根的左子树的中序序列，后一个子序列是根的右子树的中序序列。左子树的中序序列和先序序列的长度是相等的，右子树的中序序列和先序序列的长度是相等的。根据这两个子序列，可以在先序序列中找到左子树的先序序列和右子树的先序序列，如图 5.11 所示。如此递归地分解下去，便能唯一地确定这棵二叉树。

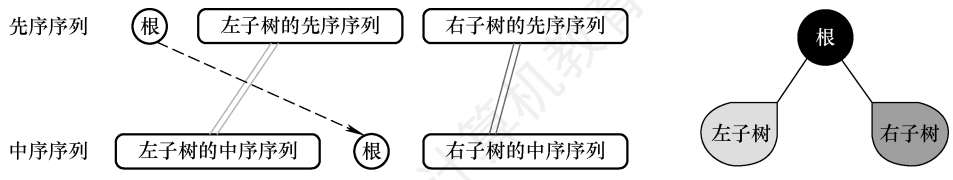


图 5.11　由先序序列和中序序列构造二叉树

配套课程公众号[考机研]
永久联系微信:4550060

例如，求先序序列（*ABCDEFGHI*）和中序序列（*BCAEDGHFI*）所确定的二叉树。首先，由先序序列可知 *A* 为二叉树的根结点。中序序列中 *A* 之前的 *BC* 为左子树的中序序列，*EDGHFI* 为右子树的中序序列。然后，由先序序列可知 *B* 是左子树的根结点，*D* 是右子树的根结点。以此类推，就能将剩下的结点继续分解下去，最后得到的二叉树如图 5.12(c)所示。

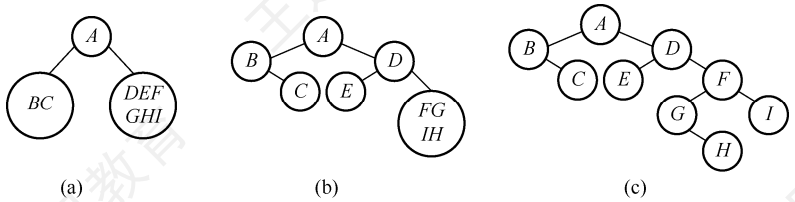


图 5.12　一棵二叉树的构造过程

（2）由后序序列和中序序列构造二叉树

命题追踪 ▶▶ 由后序序列和树形构造一棵二叉树（2017、2023）

同理，由二叉树的后序序列和中序序列也可以唯一地确定一棵二叉树。因为后序序列的最后一个结点就如同先序序列的第一个结点，可以将中序序列分割成两个子序列，如图 5.13 所示，然后采用类似的方法递归地进行分解，进而唯一地确定这棵二叉树。

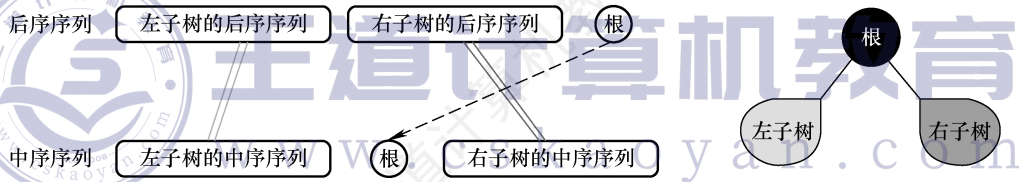


图 5.13　由后序序列和中序序列构造二叉树

请读者分析后序序列（*CBEHGIFDA*）和中序序列（*BCAEDGHFI*）所确定的二叉树。

（3）由层序序列和中序序列构造二叉树

在层序遍历中，第一个结点一定是二叉树的根结点，这样就将中序序列分割成了左子树的中序序列和右子树的中序序列。若存在左子树，则层序序列的第二个结点一定是左子树的根，可进一步划分左子树；若存在右子树，则层序序列中紧接着的下一个结点一定是右子树的根，可进一步划分右子树，如图 5.14 所示。采用这种方法继续分解，就能唯一确定这棵二叉树。

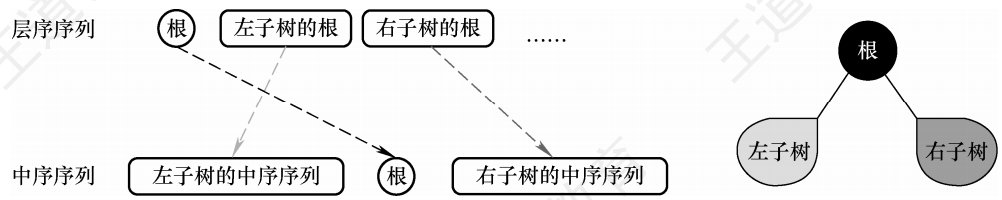


图 5.14　由层序序列和中序序列构造二叉树

请读者分析层序序列（*ABDCEFGIH*）和中序序列（*BCAEDGHFI*）所确定的二叉树。

注意，先序序列、后序序列和层序序列的两两组合，无法唯一确定一棵二叉树。例如，图 5.15 所示的两棵二叉树的先序序列都为 *AB*，后序序列都为 *BA*，层序序列都为 *AB*。

配套课程公众号[考机研]
永久联系微信:4550060

图 5.15　两棵不同的二叉树

5.3.2　线索二叉树

1．线索二叉树的基本概念

遍历二叉树是以一定的规则将二叉树中的结点排列成一个线性序列，从而得到几种遍历序列，使得该序列中的每个结点（第一个和最后一个除外）都有一个直接前驱和直接后继。

命题追踪 ▶▶ 线索二叉树的定义（2010）

传统的二叉链表存储仅能体现一种父子关系，不能直接得到结点在遍历中的前驱或后继。前面提到，在含 n 个结点的二叉树中，有 $n+1$ 个空指针。这是因为每个叶结点都有 2 个空指针，每个度为 1 的结点都有 1 个空指针，空指针总数为 $2n_0+n_1$，又 $n_0=n_2+1$，所以空指针总数为 $n_0+n_1+n_2+1=n+1$。由此设想能否利用这些空指针来存放指向其前驱或后继的指针？这样就可以像遍历单链表那样方便地遍历二叉树。引入线索二叉树正是为了加快查找结点前驱和后继的速度。

规定：若无左子树，令 lchild 指向其前驱结点；若无右子树，令 rchild 指向其后继结点。如图 5.16 所示，还需增加两个标志域，以标识指针域指向左（右）孩子或前驱（后继）。

lchild	ltag	data	rtag	rchild

图 5.16　线索二叉树的结点结构

其中，标志域的含义如下：

$$\texttt{ltag}=\begin{cases}0, & \texttt{lchild域指示结点的左孩子}\\ 1, & \texttt{lchild域指示结点的前驱}\end{cases}$$

$$\texttt{rtag}=\begin{cases}0, & \texttt{rchild域指示结点的右孩子}\\ 1, & \texttt{rchild域指示结点的后继}\end{cases}$$

线索二叉树的存储结构描述如下：

```
typedef struct ThreadNode{
    ElemType data;                              //数据元素
    struct ThreadNode *lchild,*rchild;          //左、右孩子指针
    int ltag,rtag;                              //左、右线索标志
}ThreadNode,*ThreadTree;
```

以这种结点结构构成的二叉链表作为二叉树的存储结构，称为*线索链表*，其中指向结点前驱和后继的指针称为*线索*。加上线索的二叉树称为*线索二叉树*。

2．中序线索二叉树的构造

二叉树的线索化是将二叉链表中的空指针改为指向前驱或后继的线索。而前驱或后继的信息只有在遍历时才能得到，因此线索化的实质就是遍历一次二叉树。

命题追踪 ▶▶ 中序线索二叉树中线索的指向（2014）

以中序线索二叉树的建立为例。附设指针 pre 指向刚刚访问过的结点，指针 p 指向正在访问的结点，即 pre 指向 p 的前驱。在中序遍历的过程中，检查 p 的左指针是否为空，若为空就将它指向 pre；检查 pre 的右指针是否为空，若为空就将它指向 p，如图 5.17 所示。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

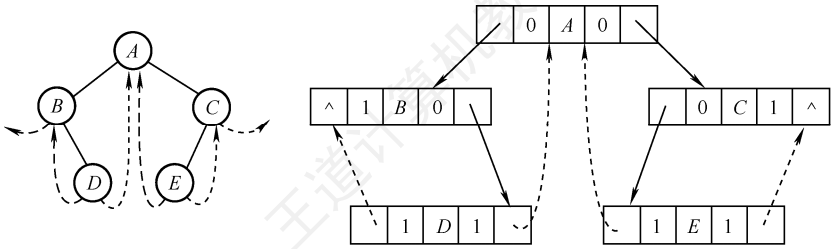


图 5.17　中序线索二叉树及其二叉链表示

通过中序遍历对二叉树线索化的递归算法如下：

```
void InThread(ThreadTree &p,ThreadTree &pre){
    if(p!=NULL){
        InThread(p->lchild,pre);        //递归，线索化左子树
        if(p->lchild==NULL){            //当前结点的左子树为空
            p->lchild=pre;              //建立当前结点的前驱线索
            p->ltag=1;
        }
        if(pre!=NULL&&pre->rchild==NULL){//前驱结点非空且其右子树为空
            pre->rchild=p;              //建立前驱结点的后继线索
            pre->rtag=1;
        }
        pre=p;                          //标记当前结点成为刚刚访问过的结点
        InThread(p->rchild,pre);        //递归，线索化右子树
    }
}
```

通过中序遍历建立中序线索二叉树的主过程算法如下：

```
void CreateInThread(ThreadTree T){
    ThreadTree pre=NULL;
    if(T!=NULL){                        //非空二叉树，线索化
        InThread(T,pre);                //线索化二叉树
        pre->rchild=NULL;               //处理遍历的最后一个结点
        pre->rtag=1;
    }
}
```

为方便起见，可在二叉树的线索链表上也添加一个头结点，令其 lchild 域的指针指向二叉树的根结点，其 rchild 域的指针指向中序遍历时访问的最后一个结点；令二叉树中序序列中的第一个结点的 lchild 域指针和最后一个结点的 rchild 域指针均指向头结点。这好比为二叉树建立了一个双向线索链表，方便从前往后或从后往前对线索二叉树进行遍历，如图 5.18 所示。

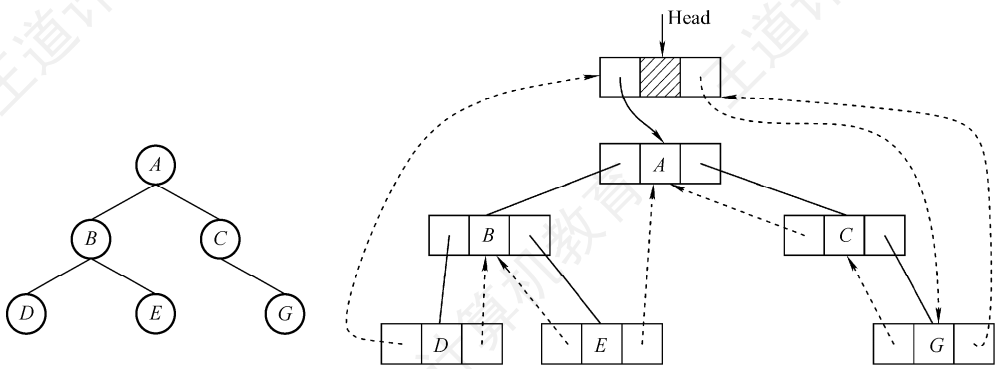


图 5.18　带头结点的中序线索二叉树

配套课程公众号[考机研]
永久联系微信:4550060

3．中序线索二叉树的遍历

中序线索二叉树的结点中隐含了线索二叉树的前驱和后继信息。在对其进行遍历时，只要先找到序列中的第一个结点，然后依次找结点的后继，直至其后继为空。在中序线索二叉树中找结点后继的规律是：若其右标志为“1”，则右链为线索，指示其后继，否则遍历右子树中第一个访问的结点（右子树中最左下的结点）为其后继。不含头结点的线索二叉树的遍历算法如下。

1）求中序线索二叉树的中序序列下的第一个结点：

```
ThreadNode *Firstnode(ThreadNode *p){
    while(p->ltag==0) p=p->lchild;  //最左下结点（不一定是叶结点）
    return p;
}
```

2）求中序线索二叉树中结点 p 在中序序列下的后继：

```
ThreadNode *Nextnode(ThreadNode *p){
    if(p->rtag==0) return Firstnode(p->rchild); //右子树中最左下结点
    else return p->rchild;          //若 rtag==1 则直接返回后继线索
}
```

请读者自行分析并完成求中序线索二叉树的最后一个结点和结点 p 前驱的运算[①]。

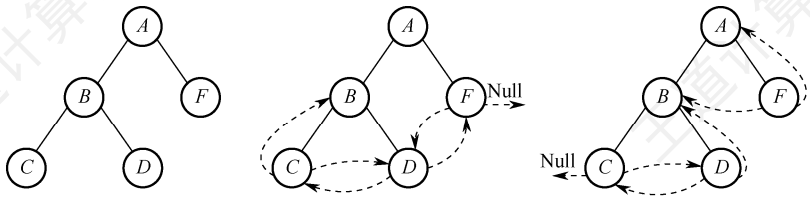
3）利用上面两个算法，可写出不含头结点的中序线索二叉树的中序遍历的算法：

```
void Inorder(ThreadNode *T){
    for(ThreadNode *p=Firstnode(T);p!=NULL; p=Nextnode(p))
        visit(p);
}
```

4．先序线索二叉树和后序线索二叉树

上面给出了建立中序线索二叉树的代码，建立先序线索二叉树和建立后序线索二叉树的代码类似，只需变动线索化改造的代码段与调用线索化左右子树递归函数的位置。

以图 5.19(a)的二叉树为例给出手工求先序线索二叉树的过程：先序序列为 *ABCDF*，然后依次判断每个结点的左右链域，若为空，则将其改造为线索。结点 *A*, *B* 均有左右孩子；结点 *C* 无左孩子，将左链域指向前驱 *B*，无右孩子，将右链域指向后继 *D*；结点 *D* 无左孩子，将左链域指向前驱 *C*，无右孩子，将右链域指向后继 *F*；结点 *F* 无左孩子，将左链域指向前驱 *D*，无右孩子，也无后继，所以置空，得到的先序线索二叉树如图 5.19(b)所示。求后序线索二叉树的过程：后序序列为 *CDBFA*，结点 *C* 无左孩子，也无前驱，所以置空，无右孩子，将右链域指向后继 *D*；结点 *D* 无左孩子，将左链域指向前驱 *C*，无右孩子，将右链域指向后继 *B*；结点 *F* 无左孩子，将左链域指向前驱 *B*，无右孩子，将右链域指向后继 *A*，得到的后序线索二叉树如图 5.19(c)所示。



(a) 一颗二叉树　(b) 先序线索二叉树　(c) 后序线索二叉树

图 5.19　先序线索二叉树和后序线索二叉树

如何在先序线索二叉树中找结点的后继？若有左孩子，则左孩子就是其后继；若无左孩子但有右孩子，则右孩子就是其后继；若为叶结点，则右链域直接指示了结点的后继。

① 将函数 1 中的 ltag 和 lchild 换成 rtag 和 rchild，即求中序线索二叉树的最后一个结点。将函数 2 中的 rtag 和 rchild 换成 ltag 和 lchild，此外调用函数改为求左子树的中序线索二叉树的最后一个结点，即求中序线索二叉树中结点 p 的前驱。

配套课程公众号[考机研]
永久联系微信:4550060

命题追踪 ▶▶ 后序线索二叉树中线索的指向（2013）

在后序线索二叉树中找结点的后继较为复杂，可分三种情况：①若结点 x 是二叉树的根，则其后继为空；②若结点 x 是其双亲的右孩子，或是其双亲的左孩子且其双亲没有右子树，则其后继即双亲；③若结点 x 是其双亲的左孩子，且其双亲有右子树，则其后继为双亲的右子树上按后序遍历列出的第一个结点。图 5.19(c)中找结点 B 的后继无法通过链域找到，可见在后序线索二叉树上找后继时需知道结点双亲，即需采用带标志域的三叉链表作为存储结构。

5.3.3 本节试题精选

一、单项选择题

01. 在下列关于二叉树遍历的说法中，正确的是（ ）。

A. 若有一个结点是二叉树中某个子树的中序遍历结果序列的最后一个结点，则它一定是该子树的前序遍历结果序列的最后一个结点

B. 若有一个结点是二叉树中某个子树的前序遍历结果序列的最后一个结点，则它一定是该子树的中序遍历结果序列的最后一个结点

C. 若有一个叶结点是二叉树中某个子树的中序遍历结果序列的最后一个结点，则它一定是该子树的前序遍历结果序列的最后一个结点

D. 若有一个叶结点是二叉树中某个子树的前序遍历结果序列的最后一个结点，则它一定是该子树的中序遍历结果序列的最后一个结点

02. 在任何一棵二叉树中，若结点 a 有左孩子 b、右孩子 c，则在结点的先序序列、中序序列、后序序列中，（ ）。

A. 结点 b 一定在结点 a 的前面　　B. 结点 a 一定在结点 c 的前面

C. 结点 b 一定在结点 c 的前面　　D. 结点 a 一定在结点 b 的前面

03. 设 n, m 为一棵二叉树上的两个结点，在中序遍历时，n 在 m 前的条件是（ ）。

A. n 在 m 右方　　B. n 是 m 祖先　　C. n 在 m 左方　　D. n 是 m 子孙

04. 设 n, m 为一棵二叉树上的两个结点，在后序遍历时，n 在 m 前的充分条件是（ ）。

A. n 在 m 右方　　B. n 是 m 祖先　　C. n 在 m 左方　　D. n 是 m 子孙

05. 某非空二叉树采用顺序存储结构，树中的结点信息按完全二叉树的层次序列依次存放在如下所示的一维数组中，则该二叉树的后序遍历序列为（ ）。

0	1	2	3	4	5	6	7	8	9	10	11	12
a	*b*	*c*		*d*	*e*	*f*			*g*			*h*

A. *ghbefhca*　　B. *gbdehcfa*　　C. *gdbhefca*　　D. *bgdehcfa*

06. 在二叉树的前序序列、中序序列和后序序列中，所有叶结点的先后顺序（ ）。

A. 都不相同　　B. 完全相同

C. 前序和中序相同，而与后序不同　　D. 中序和后序相同，而与前序不同

07. 对二叉树的结点从 1 开始进行连续编号，要求每个结点的编号大于其左、右孩子的编号，同一结点的左、右孩子中，其左孩子的编号小于其右孩子的编号，可采用（ ）次序的遍历实现编号。

A. 先序遍历　　B. 中序遍历　　C. 后序遍历　　D. 层次遍历

08. 按某种顺序对二叉树的结点进行编号，编号为 $1, 2, \cdots, n$，规定：树中任一结点 v，其编号等于 v 的左子树上的最小编号减 1，而 v 的右子树中的最小编号等于 v 的左子树上的

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

最大编号加 1，则说明该二叉树是按（　）次序编号的。

A. 中序遍历　　B. 先序遍历　　C. 后序遍历　　D. 层次遍历

09. 前序序列为 *A*, *B*, *C*，后序序列为 *C*, *B*, *A* 的二叉树共有（　）。

A. 1 棵　　B. 2 棵　　C. 3 棵　　D. 4 棵

10. 一棵完全二叉树的后序遍历序列为 *CDBFGEA*，则其先序遍历序列是（　）。

A. *CBDAFEG*　　B. *ABECDFG*　　C. *ABCDEFG*　　D. 无法确定

11. 设结点 *X* 和 *Y* 是二叉树中任意的两个结点。在该二叉树的先序遍历序列中 *X* 在 *Y* 之前，而在其后序遍历序列中 *X* 在 *Y* 之后，则 *X* 和 *Y* 的关系是（　）。

A. *X* 是 *Y* 的左兄弟　　B. *X* 是 *Y* 的右兄弟

C. *X* 是 *Y* 的祖先　　D. *X* 是 *Y* 的后裔

12. 若二叉树中结点的先序序列是…*a*…*b*…，中序序列是…*b*…*a*…，则（　）。

A. 结点 *a* 和结点 *b* 分别在某结点的左子树和右子树中

B. 结点 *b* 在结点 *a* 的右子树中

C. 结点 *b* 在结点 *a* 的左子树中

D. 结点 *a* 和结点 *b* 分别在某结点的两棵非空子树中

13. 一棵二叉树的先序遍历序列为 1234567，它的中序遍历序列可能是（　）。

A. 3124567　　B. 1234567　　C. 4135627　　D. 1463572

14. 下列序列中，不能唯一地确定一棵二叉树的是（　）。

A. 层次序列和中序序列　　B. 先序序列和中序序列

C. 后序序列和中序序列　　D. 先序序列和后序序列

15. 若一棵二叉树的中序序列和后序序列相同，则（　）。

A. 二叉树为空树或二叉树任一结点没有左子树

B. 二叉树为空树或二叉树任一结点没有右子树

C. 二叉树为空树或二叉树中每个结点的度为 1

D. 二叉树为空树或二叉树为满二叉树

16. 已知一棵二叉树的后序序列为 *DABEC*，中序序列为 *DEBAC*，则先序序列为（　）。

A. *ACBED*　　B. *DECAB*　　C. *DEABC*　　D. *CEDBA*

17. 已知一棵二叉树的先序遍历结果为 *ABCDEF*，中序遍历结果为 *CBAEDF*，则后序遍历的结果为（　）。

A. *CBEFDA*　　B. *FEDCBA*　　C. *CBEDFA*　　D. 不确定

18. 已知一棵二叉树的层次序列为 *ABCDEF*，中序序列为 *BADCFE*，则先序序列为（　）。

A. *ACBEDF*　　B. *ABCDEF*　　C. *BDFECA*　　D. *FCEDBA*

19. 某二叉树中结点 *x* 在先序、中序、后序遍历序列中的编号分别为 pre(*x*)、in(*x*)、post(*x*)（假设都从 1 开始依次顺序编号），*a* 和 *b* 是该二叉树中的两个结点，其中 *a* 是 *b* 的祖先，则下列选项中不可能出现的是（　）。

A. pre(*a*) < pre(*b*)　　B. post(*a*) < post(*b*)　　C. in(*a*) < in(*b*)　　D. in(*a*) > in(*b*)

20. 某二叉树采用二叉链表存储结构，若要删除该二叉链表中的所有结点，并释放它们占用的存储空间，则采用（　）遍历方法最合适。

A. 中序　　B. 层次　　C. 后序　　D. 先序

21. 某二叉树 *T* 采用二叉链表存储结构，*T* 的中序遍历序列为一个升序序列，要求采用某种方法对 *T* 进行某种操作之后得到一棵新的二叉树 *T'*，要求 *T'* 的中序遍历序列为一个降序

王道计算机教育 www.kaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

序列，则下列关于该算法的叙述中，正确的是（ ）。

A. 采用中序遍历的方法最合适　　B. 采用后序遍历的方法最合适

C. T'中的根结点还是原 T 中的根结点　　D. T'中的叶结点不一定是原 T 中的叶结点

22. 引入线索二叉树的目的是（ ）。

A. 加快查找结点的前驱或后继的速度　　B. 为了能在二叉树中方便插入和删除

C. 为了能方便找到双亲　　D. 使二叉树的遍历结果唯一

23. 线索二叉树是一种（ ）结构。

A. 逻辑　　B. 逻辑和存储　　C. 物理　　D. 线性

24. n 个结点的线索二叉树上含有的线索数为（ ）。

A. $2n$　　B. $n-1$　　C. $n+1$　　D. n

25. 判断线索二叉树中*p结点有右孩子结点的条件是（ ）。

A. `p!=NULL`　　B. `p->rchild!=NULL`

C. `p->rtag==0`　　D. `p->rtag==1`

26. 一棵左子树为空的二叉树在先序线索化后，其中空的链域的个数是（ ）。

A. 不确定　　B. 0个　　C. 1个　　D. 2个

27. 在线索二叉树中，下列说法不正确的是（ ）。

A. 在中序线索树中，若某结点有右孩子，则其后继结点是它的右子树的最左下结点

B. 在中序线索树中，若某结点有左孩子，则其前驱结点是它的左子树的最右下结点

C. 线索二叉树是利用二叉树的 $n+1$ 个空指针来存放结点的前驱和后继信息的

D. 每个结点通过线索都可以直接找到它的前驱和后继

28. 二叉树在线索化后，仍不能有效求解的问题是（ ）。

A. 先序线索二叉树中求先序后继　　B. 中序线索二叉树中求中序后继

C. 中序线索二叉树中求中序前驱　　D. 后序线索二叉树中求后序后继

29. 若 X 是二叉中序线索树中一个有左孩子的结点，且 X 不为根，则 X 的前驱为（ ）。

A. X 的双亲　　B. X 的右子树中最左的结点

C. X 的左子树中最右的结点　　D. X 的左子树中最右的叶结点

30. 若 X 是后序线索二叉树中的叶结点，且 X 存在左兄弟 Y，则 X 的右线索指向的是（ ）。

A. X 的双亲　　B. 以 Y 为根的子树的最左下结点

C. X 的左兄弟 Y　　D. 以 Y 为根的子树的最右下结点

31. （ ）的遍历仍需要栈的支持。

A. 前序线索树　　B. 中序线索树　　C. 后序线索树　　D. 所有线索树

32. 某二叉树的先序序列和后序序列正好相反，则该二叉树一定是（ ）。

A. 空或只有一个结点　　B. 高度等于其结点数

C. 任意一个结点无左孩子　　D. 任意一个结点无右孩子

33. 某非空二叉树的先序序列和中序序列正好相反，则下列叙述中正确的是（ ）。

A. 该二叉树一定只有一个结点

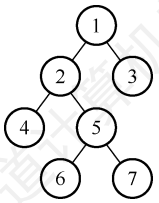
B. 只有一个叶结点的二叉树一定满足条件

C. 任意一个结点无左孩子的二叉树一定满足条件

D. 任意一个结点无右孩子的二叉树一定满足条件

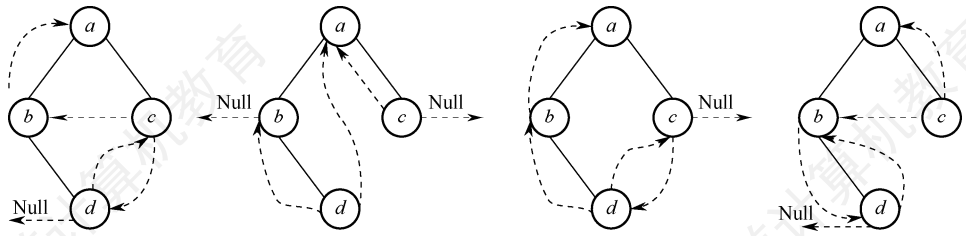
34. 【2009统考真题】给定二叉树如下图所示。设N代表二叉树的根，L代表根结点的左子树，R代表根结点的右子树。若遍历后的结点序列是3175624，则其遍历方式是（ ）。

配套课程公众号[考机研]
永久联系微信:4550060



A. LRN　　B. NRL　　C. RLN　　D. RNL

35.【2010 统考真题】下列线索二叉树中（用虚线表示线索），符合后序线索树定义的是（　）。



36.【2011 统考真题】一棵二叉树的前序遍历序列和后序遍历序列分别为 1, 2, 3, 4 和 4, 3, 2, 1，该二叉树的中序遍历序列不会是（　）。

A. 1, 2, 3, 4　　B. 2, 3, 4, 1　　C. 3, 2, 4, 1　　D. 4, 3, 2, 1

37.【2012 统考真题】若一棵二叉树的前序遍历序列为 *a*, *e*, *b*, *d*, *c*，后序遍历序列为 *b*, *c*, *d*, *e*, *a*，则根结点的孩子结点（　）。

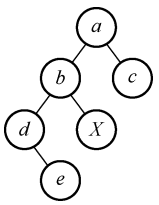
A. 只有 *e*　　B. 有 *e*、*b*　　C. 有 *e*、*c*　　D. 无法确定

38.【2013 统考真题】若 *X* 是后序线索二叉树中的叶结点，且 *X* 存在左兄弟结点 *Y*，则 *X* 的右线索指向的是（　）。

A. *X* 的父结点　　B. 以 *Y* 为根的子树的最左下结点

C. *X* 的左兄弟结点 *Y*　　D. 以 *Y* 为根的子树的最右下结点

39.【2014 统考真题】若对下图所示的二叉树进行中序线索化，则结点 *X* 的左、右线索指向的结点分别是（　）。

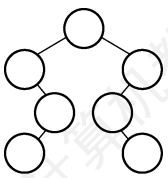


A. *e*, *c*　　B. *e*, *a*　　C. *d*, *c*　　D. *b*, *a*

40.【2015 统考真题】先序序列为 *a*, *b*, *c*, *d* 的不同二叉树的个数是（　）。

A. 13　　B. 14　　C. 15　　D. 16

41.【2017 统考真题】某二叉树的树形如下图所示，其后序序列为 *e*, *a*, *c*, *b*, *d*, *g*, *f*，树中与结点 *a* 同层的结点是（　）。



A. *c*　　B. *d*　　C. *f*　　D. *g*

42.【2017 统考真题】要使一棵非空二叉树的先序序列与中序序列相同，其所有非叶结点须

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

满足的条件是（ ）。

A. 只有左子树　　B. 只有右子树

C. 结点的度均为1　　D. 结点的度均为2

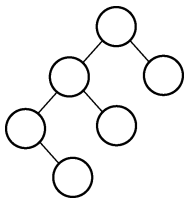
43. 【2022统考真题】若结点 p 与 q 在二叉树 T 的中序遍历序列中相邻，且 p 在 q 之前，则下列 p 与 q 的关系中，不可能的是（ ）。

I. q 是 p 的双亲　　II. q 是 p 的右孩子

III. q 是 p 的右兄弟　　IV. q 是 p 的双亲的双亲

A. 仅I　　B. 仅III　　C. 仅II、III　　D. 仅II、IV

44. 【2023统考真题】已知一棵二叉树的树形如下图所示，若其后序遍历序列为 *fdbeca*，则其先（前）序遍历序列是（ ）。



A. *aedfbc*　　B. *acebdf*　　C. *cabefd*　　D. *dfebac*

45. 【2024统考真题】若 p、q 和 v 均为二叉树 T 中的结点，v 有两个孩子结点，T 的中序遍历序列形如“…, p, v, q, …”，则在下列叙述中，正确的是（ ）。

A. p 没有右孩子，q 没有左孩子　　B. p 没有右孩子，q 有左孩子

C. p 有右孩子，q 没有左孩子　　D. p 有右孩子，q 有左孩子

二、综合应用题

01. 若某非空二叉树的先序序列和后序序列正好相反，则该二叉树的形态是什么？

02. 若某非空二叉树的先序序列和后序序列正好相同，则该二叉树的形态是什么？

03. 假设二叉树采用二叉链表存储结构，设计一个非递归算法求二叉树的高度。

04. 二叉树按二叉链表形式存储，试编写一个判别给定二叉树是否是完全二叉树的算法。

05. 假设二叉树采用二叉链表存储结构存储，试设计一个算法，计算一棵给定二叉树的所有双分支结点个数。

06. 设树 B 是一棵采用链式结构存储的二叉树，编写一个把树 B 中所有结点的左、右子树进行交换的函数。

07. 假设二叉树采用二叉链存储结构存储，设计一个算法，求先序遍历序列中第 k（$1 \leqslant k \leqslant$ 二叉树中结点个数）个结点的值。

08. 已知二叉树以二叉链表存储，编写算法完成：对于树中每个元素值为 x 的结点，删除以它为根的子树，并释放相应的空间。

09. 在二叉树中查找值为 x 的结点，试编写算法（用C语言）打印值为 x 的结点的所有祖先，假设值为 x 的结点不多于一个。

10. 设一棵二叉树的结点结构为(LLINK,INFO,RLINK)，ROOT为指向该二叉树根结点的指针，p和q分别为指向该二叉树中任意两个结点的指针，试编写算法ANCESTOR(ROOT,p,q,r)，找到p和q的最近公共祖先结点r。

11. 假设二叉树采用二叉链表存储结构，设计一个算法，求非空二叉树 b 的宽度（具有结点数最多的那一层的结点个数）。

12. 设有一棵满二叉树（所有结点值均不同），已知其先序序列为 *pre*，设计一个算法求其后序序列 *post*。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

13. 设计一个算法将二叉树的叶结点按从左到右的顺序连成一个单链表，表头指针为 head。二叉树按二叉链表方式存储，链接时用叶结点的右指针域来存放单链表指针。

14. 试设计判断两棵二叉树是否相似的算法。所谓二叉树 T_1 和 T_2 相似，指的是 T_1 和 T_2 都是空的二叉树或都只有一个根结点；或者 T_1 的左子树和 T_2 的左子树是相似的，且 T_1 的右子树和 T_2 的右子树是相似的。

15. 【2014 统考真题】二叉树的带权路径长度（WPL）是二叉树中所有叶结点的带权路径长度之和。给定一棵二叉树 T，采用二叉链表存储，结点结构为

left	weight	right

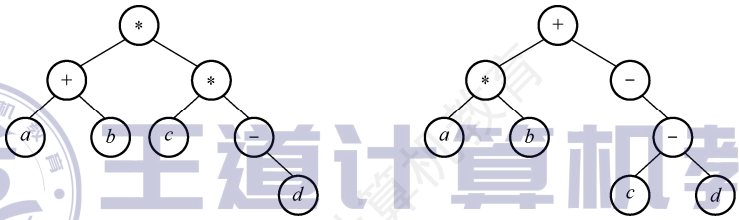
其中叶结点的 weight 域保存该结点的非负权值。设 root 为指向 T 的根结点的指针，请设计求 T 的 WPL 的算法，要求：

1）给出算法的基本设计思想。

2）使用 C 或 C++语言，给出二叉树结点的数据类型定义。

3）根据设计思想，采用 C 或 C++语言描述算法，关键之处给出注释。

16. 【2017 统考真题】请设计一个算法，将给定的表达式树（二叉树）转换为等价的中缀表达式（通过括号反映操作符的计算次序）并输出。例如，当下列两棵表达式树作为算法的输入时：



输出的等价中缀表达式分别为 (a+b)*(c*(-d)) 和 (a*b)+(-(c-d))。

二叉树结点定义如下：

```
typedef struct node{
    char data[10];                  //存储操作数或操作符
    struct node *left, *right;
}BTree;
```

要求：

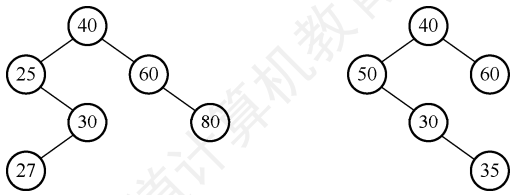
1）给出算法的基本设计思想。

2）根据设计思想，采用 C 或 C++语言描述算法，关键之处给出注释。

17. 【2022 统考真题】已知非空二叉树 T 的结点值均为正整数，采用顺序存储方式保存，数据结构定义如下：

```
typedef struct {                    //MAX_SIZE 为已定义常量
     int SqBiTNode[MAX_SIZE];       //保存二叉树结点值的数组
     int ElemNum;                   //实际占用的数组元素个数
} SqBiTree;
```

T 中不存在的结点在数组 SqBiTNode 中用 -1 表示。例如，对于下图所示的两棵非空二叉树 T_1 和 T_2，



二叉树 T_1　　二叉树 T_2

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

T_1的存储结果如下：

T1.SqBiTNode	40	25	60	-1	30	-1	80	-1	-1	27		

T1.ElemNum=10

T_2的存储结果如下：

T2.SqBiTNode	40	50	60	-1	30	-1	-1	-1	-1	-1	35	

T2.ElemNum=11

请设计一个尽可能高效的算法，判定一棵采用这种方式存储的二叉树是否为二叉搜索树，若是，则返回true，否则，返回false。要求：

1）给出算法的基本设计思想。

2）根据设计思想，采用C或C++语言描述算法，关键之处给出注释。

5.3.4 答案与解析

一、单项选择题

01. C

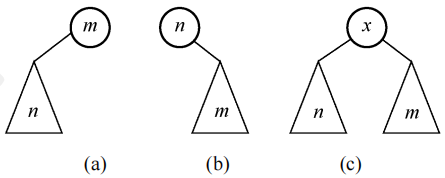
二叉树中序遍历的最后一个结点一定是从根开始沿右子女指针链走到底的结点，设用p指示。若结点p不是叶结点（其左子树非空），则前序遍历的最后一个结点在它的左子树中，选项A、B错误；若结点p是叶结点，则前序与中序遍历的最后一个结点就是它，选项C正确。若中序遍历的最后一个结点p不是叶结点，它还有一个左子女q，结点q是叶结点，那么结点q是前序遍历的最后一个结点，但不是中序遍历的最后一个结点，选项D错误。

02. C

三种遍历方式中，都先遍历左子树，再遍历右子树，因此b一定在c的前面访问。

03. C

中序遍历时，先访问左子树，再访问根结点，后访问右子树。n在m前的3种可能性如下图所示，从中看出n总是在m的左方。



【另解】设n和m的最近公共祖先p，则有以下可能：

情形1，m和n分别在p的左、右（右、左）分支上；情形2，m或n为p结点，另一结点在p的分支上。只有n和m分别处于p的左、右分支上，m为祖先结点且n位于m的左分支，n为祖先结点且m位于n的右分支，符合题意。

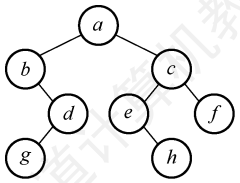
04. D

后序遍历的顺序是LRN，若n在N的左子树，m在N的右子树，则在后序遍历的过程中n在m之前访问；若n是m的子孙，设m在N的位置，则n无论是在m的左子树还是在右子树，在后序遍历的过程中n都在m之前访问。其他都不可以。选项C要成立，就需加上两个结点位于同一层这个条件。

05. C

在二叉树的数组存储结构中，下标为i的结点的左、右孩子的下标分别为$2i+1$和$2i+2$（若存在），画出二叉树的形态如下图所示，则后序遍历序列为$gdbhefca$。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060



06．B

三种遍历方式中，访问左、右子树的先后顺序是不变的，只是访问根结点的顺序不同，因此叶结点的先后顺序完全相同。此外，读者可以采用特殊值法，画一个结点数为 3 的满二叉树，采用三种遍历方式来验证答案的正确性。

07．C

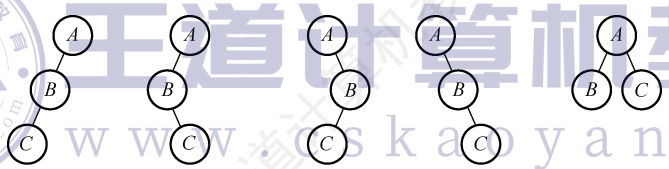
对每个顶点从 1 开始按序编号，要求结点编号大于其左、右孩子编号，并且左孩子编号小于右孩子编号。编号越大说明遍历顺序越靠后，因此，三者遍历顺序为先左子树、再右子树、后根结点。4 个选项中仅后序遍历满足要求。

08．B

结点 v 的编号比其左子树上的最小编号还小，而 v 的右子树中的最小编号大于 v 的左子树中的最大编号，因此 v 的编号比其左、右子树上的所有编号都小，显然是按先序遍历次序。

09．D

前序为 A、B、C 的不同二叉树共有 5 种，其中后序为 C、B、A 的有 4 种（前 4 种），都是单支树，如下图所示。



王道计算机教育 www.cskaoyan.com

10．C

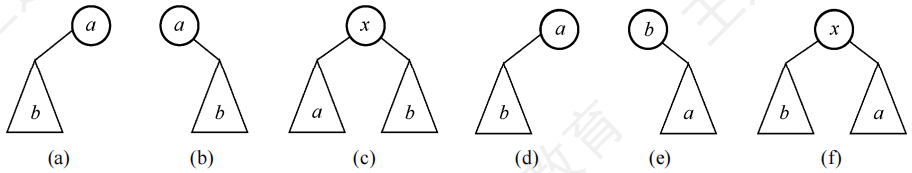
7 个结点的完全二叉树是一棵 3 层的满二叉树，画出相应二叉树的树形，根据后序遍历序列填入相应的结点，得到相应的完全二叉树，求得其先序遍历序列为 *ABCDEFG*。

11．C

二叉树的前序遍历为 NLR，后序遍历为 LRN。根据题意，在前序序列中 X 在 Y 之前，在后序序列中 X 在 Y 之后，若设 X 在根的位置，Y 在其左子树或右子树中，即满足要求。

12．C

先序序列是…a…b…，因此 a 和 b 结点的 3 种情况如下图(a)～(c)所示。中序序列是…b…a…，因此 a 和 b 结点的 3 种情况如下图(d)～(f)所示，相同部分是 b 在 a 的左子树中。



13．B

解法 1：由题可知，1 为根结点，2 为 1 的孩子。对于选项 A，3 应为 1 的左孩子，前序序列应为 13…，不符题意。类似地，选项 C 也是错误的。对于选项 B，2 为 1 的右孩子，3 为 2 的右孩子……满足题意。对于选项 D，463572 应为 1 的右子树，2 为 1 的右孩子，46357 为 2 的左子树，3 为 2 的左孩子，46 为 3 的左子树，57 为 3 的右子树，前序序列 4、6 应相连，5、7 应相连，不符题意。

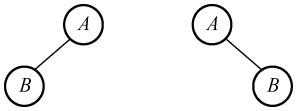
配套课程公众号[考机研]
永久联系微信:4550060

解法 2：前序遍历时需要借助栈。二叉树的前序序列和中序序列的关系相当于以前序序列作为入栈次序，以中序序列作为出栈次序。题中以 1234567 入栈；对于选项 A，第一个出栈的是 3，所以 1 不可能在 2 之前出栈，错误。对于选项 C，1 不可能在 3 之前出栈，错误。对于选项 D，6 第三个出栈，此时栈顶元素是 5，不是 3，错误。选项 B 正确。

解法 3：因前序序列和中序序列可以确定一棵二叉树，所以可试着用题目中的序列构造出相应的二叉树，即可得知，只有选项 B 的序列可以构造出二叉树。

14．D

先序序列为 NLR，后序序列为 LRN，虽然可以唯一确定树的根结点，但无法划分左、右子树。例如，先序序列为 *AB*，后序序列为 *BA*，则其对应的二叉树如下图所示。

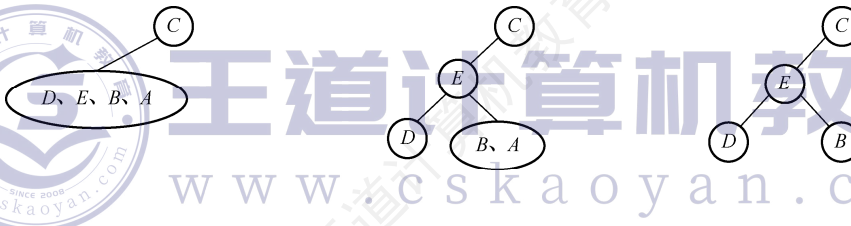


15．B

中序遍历是“左根右”，后序遍历是“左右根”，当任一结点没有右子树时，两种遍历都是“左根”。显然，当二叉树为空树或只有根结点时，其中序序列和后序序列也相同。

16．D

根据后序序列与中序序列可构造出二叉树，如下图所示。由图可知先序序列为 *CEDBA*。

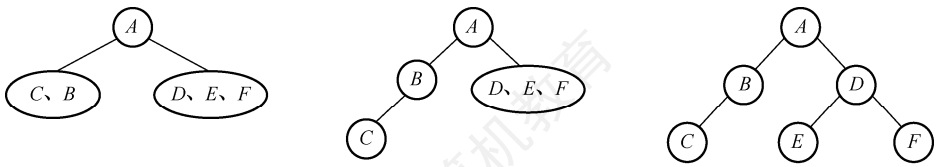


(a) 确定根结点　(b) 确定左子树根结点　(c) 确定剩下的子树

17．A

对于这种遍历序列问题，先根据遍历的性质排除若干项，若还无法确定答案，则再根据遍历结果得到二叉树，找到对应遍历序列。例如，在本题中，已知先序和中序遍历结果，可知本树的根结点为 *A*，左子树有 *C* 和 *B*，其余为右子树，则后序遍历结果中，*A* 一定在最后，并且 *C* 和 *B* 一定在前面，排除选项 B 和 D。又因先序中有 *DEF*，中序中有 *EDF*，则 *D* 为这个子树的根，所以 *D* 在后序中排在 *EF* 之后。

根据二叉树的递归定义，要确定二叉树，就要分别找到根结点和左、右子树。因此，根据遍历结果，必定要确定根结点位置和如何划分左、右子树，才可以确定最终的二叉树。因此，仅有先序和后序遍历不能唯一确定一棵二叉树，而二者之一加上中序遍历都可以唯一确定一棵二叉树。如在本题中，根据先序和中序遍历的结果确定二叉树的过程如下图所示。

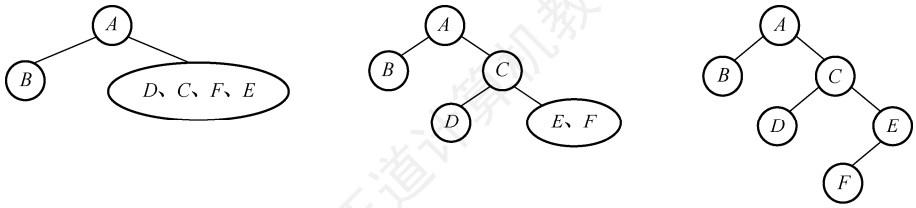


(a) 确定根结点　(b) 确定左子树　(c) 确定右子树

18．B

可构造出二叉树如下图所示。因此，先序序列为 *ABCDEF*。

配套课程公众号[考机研]
永久联系微信:4550060



(a) 确定根结点　(b) 确定右子树的根结点　(c) 确定剩下的子树

19．B

先序遍历是根左右，祖先 a 先于子孙 b 访问，即 pre(a) < pre(b)，因此选项 A 一定成立。后序遍历是左右根，子孙 b 先于祖先 a 访问，即 post(b) < post(a)，因此选项 B 一定不成立。中序遍历是左根右，左子树中的子孙先于祖先访问，右子树中的子孙后于祖先访问，因此子孙的编号可能小于祖先的编号，也可能大于祖先的编号，选项 C 和 D 都有可能。

20．C

删除一个结点时，需要先递归地删除它的左、右孩子，并释放它们所占的存储空间，然后删除该结点，并删除它所占的存储空间，这正好和后序遍历的访问顺序相吻合。

21．B

只要交换 T 中所有分支结点的左右子树，就能得到一棵中序遍历序列为降序序列的树，而这并不会改变根结点，叶结点也仅仅交换位置，仍是原 T 中的叶结点，选项 C、D 错误。交换 T 中所有分支结点的左右子树，要么先处理根结点，然后递归地处理左右子树，即先序遍历；要么先处理左右子树，然后处理根结点，即后序遍历；中序遍历是不适合的。选项 A 错误，选项 B 正确。

22．A

线索是前驱结点和后继结点的指针，引入线索的目的是加快对二叉树的遍历。

23．C

二叉树是一种逻辑结构，但线索二叉树是加上线索后的链表结构，即它是二叉树在计算机内部的一种存储结构，所以是一种物理结构。

24．C

n 个结点共有链域指针 $2n$ 个，其中，除根结点外，每个结点都被一个指针指向。剩余的链域建立线索，共 $2n-(n-1)=n+1$ 个线索。

25．C

线索二叉树中用 `ltag/rtag` 标识结点的左/右指针域是否为线索，其值为 1 时，对应指针域为线索，其值为 0 时，对应指针域为左/右孩子。

26．D

对左子树为空的二叉树进行先序线索化，根结点的左子树为空并且也没有前驱结点（先遍历根结点），先序遍历的最后一个元素为叶结点，左、右子树均为空且有前驱无后继结点，所以线索化后，树中空链域有 2 个。

27．D

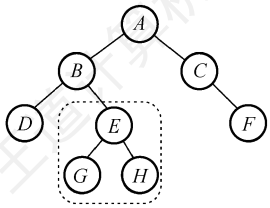
不是每个结点通过线索都可以直接找到它的前驱和后继。在先序线索二叉树中查找一个结点的先序后继很简单，而查找先序前驱必须知道该结点的双亲结点。同样，在后序线索二叉树中查找一个结点的后序前驱也很简单，而查找后序后继也必须知道该结点的双亲结点，二叉链表中没有存放双亲的指针。

28．D

后序线索二叉树不能有效解决求后序后继的问题。如下图所示，结点 E 的右指针指向右孩子，

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

而在后序序列中 E 的后继结点为 B，在查找 E 的后继时仍然只能按常规方法来查找。



29．C

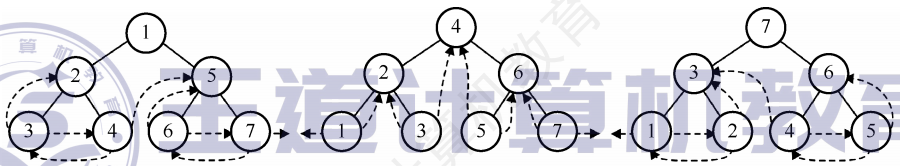
在二叉中序线索树中，某结点若有左孩子，则按照中序“左根右”的顺序，该结点的前驱结点为左子树中最右的一个结点（注意，并不一定是最右叶结点）。

30．A

在二叉树的后序遍历中，叶结点 X 的后继是其双亲，因此 X 的右线索应指向该结点。

31．C

后序线索树遍历时，最后访问根结点，若从右孩子 x 返回访问父结点，则结点 x 的右孩子不一定为空（右指针无法指向其后继），因此通过指针可能无法遍历整棵树。如下图所示，结点中的数字表示遍历的顺序，图(c)中结点 6 的右指针指向其右孩子 5，而不指向其后序后继结点 7，因此后序遍历还需要栈的支持，而图(a)和图(b)均可遍历。



32．B

非空二叉树的先序序列和后序序列相反，即“根左右”与“左右根”顺序相反，因此树只有根结点，或根结点只有左子树或右子树，其子树也有同样的性质，任意结点只有一个孩子，才能满足先序序列和后序序列正好相反。此时树形应为一个长链，树中仅有一个叶结点。

33．D

非空二叉树的先序序列和中序序列相反，即“根左右”与“左根右”顺序相反，因此树只有根结点，或任意一个结点只有左孩子，此时树形应该是一棵向左倾斜的单支树，这棵单支树只有一个叶结点。但是，只有一个叶结点的二叉树不能保证任意一个结点无右孩子。

34．D

分析遍历后的结点序列，可以看出根结点是在中间被访问的，而且右子树结点在左子树之前，则遍历的方法是 RNL。本题考查的遍历方法并不是二叉树遍历的 3 种基本遍历方法，对于考生而言，重要的是掌握遍历的思想。

35．D

题中所给二叉树的后序序列为 $dbca$。结点 d 无前驱和左子树，左链域空，无右子树，右链域指向其后继结点 b；结点 b 无左子树，左链域指向其前驱结点 d；结点 c 无左子树，左链域指向其前驱结点 b，无右子树，右链域指向其后继结点 a。

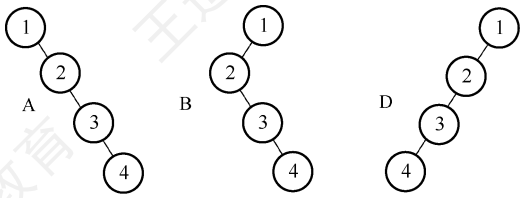
36．C

前序序列为 NLR，后序序列为 LRN，因为前序序列和后序序列刚好相反，所以不可能存在一个结点同时有左右孩子，即二叉树的高度为 4。1 为根结点，根结点只能有左孩子（或右孩子），

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

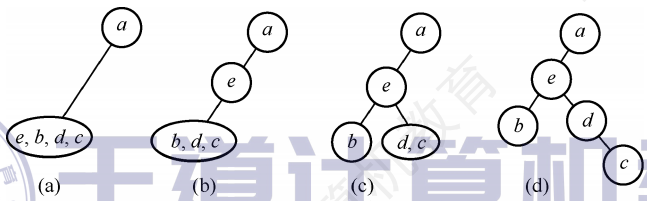
因此在中序序列中，1 或在序列首或在序列尾，选项 A、B、C、D 皆满足要求。仅考虑以 1 的孩子结点 2 为根结点的子树，它也只能有左孩子（或右孩子），因此在中序序列中，2 或在序列首或在序列尾，选项 A、B、D 皆满足要求。

【另解】画出各选项与题干信息所对应的二叉树如下，所以选项 A、B、D 均满足。



37．A

前序序列和后序序列不能唯一确定一棵二叉树，但可以确定二叉树中结点的祖先关系：当两个结点的前序序列为 XY 与后序序列为 YX 时，则 X 为 Y 的祖先。考虑前序序列 a, e, b, d, c、后序序列 b, c, d, e, a，可知 a 为根结点，e 为 a 的孩子结点；此外，由 a 的孩子结点的前序序列 e, b, d, c、后序序列 b, c, d, e，可知 e 是 bcd 的祖先，所以根结点的孩子结点只有 e。



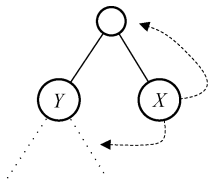
排除法：显然 a 为根结点，且确定 e 为 a 的孩子结点，排除选项 D。各种遍历算法中左右子树的遍历次序是固定的，若 b 也为 a 的孩子结点，则在前序序列和后序序列中 e、b 的相对次序应是不变的，所以排除选项 B，同理排除选项 C。

特殊法：前序序列和后序序列对应多棵不同的二叉树树形，我们只需画出满足该条件的任意一棵二叉树即可，任意一棵二叉树必定满足正确选项的要求。

显然选择选项 A，最终得到的二叉树满足题设中前序序列和后序序列的要求。

38．A。

根据后序线索二叉树的定义，X 结点为叶结点且有左兄弟，因此这个结点为右孩子结点，利用后序遍历的方式可知 X 结点的后序后继是其父结点，即其右线索指向的是父结点。为了更加形象，在解题的过程中可以画出如下所示的草图。



39．D

线索二叉树的线索实际上指向的是相应遍历序列特定结点的前驱结点和后继结点，所以先写出二叉树的中序遍历序列 $debxac$，中序遍历中在 x 左边和右边的字符，就是它在中序线索化的左、右线索，即 b, a。

40．B

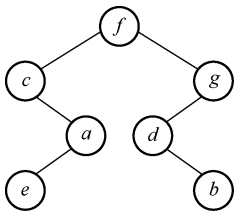
根据二叉树前序遍历和中序遍历的递归算法中递归工作栈的状态变化得出：前序序列和中序

配套课程公众号[考机研]
永久联系微信:4550060

序列的关系相当于以前序序列为入栈次序，以中序序列为出栈次序。因为前序序列和中序序列可以唯一地确定一棵二叉树，所以题意相当于“以序列 a, b, c, d 为入栈次序，则出栈序列的个数为？”，对于 n 个不同元素入栈，出栈序列的个数为 $\frac{1}{n+1}C_{2n}^{n}=14$。

41. B

后序序列先访问左子树，接着访问右子树，最后访问父结点，递归进行。根结点左子树的叶结点首先被访问，它是 e。接下来是它的父结点 a，然后是 a 的父结点 c。接着访问根结点的右子树。它的叶结点 b 首先被访问，然后是 b 的父结点 d，再后是 d 的父结点 g，最后是根结点 f，如下图所示。因此 d 与 a 同层，选项 B 正确。



42. B

先序序列先访问父结点，接着访问左子树，然后访问右子树。中序序列先访问左子树，接着访问父结点，然后访问右子树，递归进行。若所有非叶结点只有右子树，则先序序列和中序序列都先访问父结点，后访问右子树，递归进行。

43. B

对于此类题，每种情况只需举出一个反例即可。如图 1 所示，q 是 p 的双亲，中序遍历序列为{p, q}，选项 I 可能。如图 2 所示，q 是 p 的右孩子，中序遍历序列为{p, q}，选项 II 可能。如图 4 所示，q 是 p 的双亲的双亲，中序遍历序列为{x, p, q}，选项 IV 可能。如图 3 所示，q 是 p 的右兄弟，F 是 q 和 p 的父结点，中序遍历要求先遍历左子树，再访问根结点，最后遍历右子树，因此一定先访问 p，再访问 F，最后访问 q，p 和 q 不可能相邻出现，选项 III 不可能。

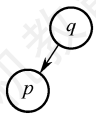


图 1

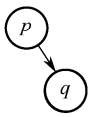


图 2

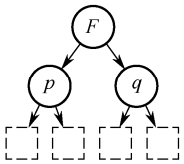


图 3

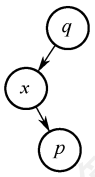
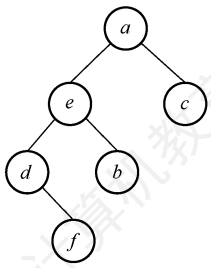


图 4

44. A

根据二叉树的树形和后序遍历序列，可以轻松地将各字母填入结点中，如下图所示。



然后对该二叉树进行先序遍历，得到序列 *aedfbc*。

配套课程公众号[考机研]
永久联系微信:4550060

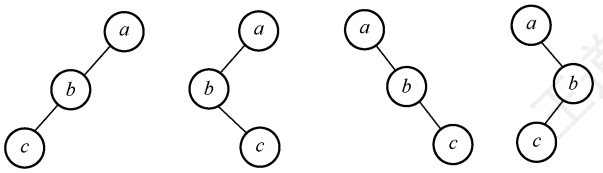
45. A

根据中序遍历的特点，v 有左右子树，以 v 为根的子树的中序序列为：v 的左子树的中序序列, v, v 的右子树的中序序列。又因为 T 的中序序列为“$\cdots, p, v, q, \cdots$”，可知 p 属于 v 的左子树，q 属于 v 的右子树。在 v 的左子树的中序序列中：假设 p 有右孩子，则 p 的右孩子在中序序列中应在 p 之后，与 p 是最后一个遍历结点矛盾，因此 p 不存在右孩子。在 v 的右子树的中序序列中：假设 q 存在左孩子，则 q 的左孩子在中序序列中应在 q 之前，与 q 是第一个遍历结点矛盾，因此 q 不存在左孩子。

二、综合应用题

01.【解答】

二叉树的先序序列是 NLR，后序序列是 LRN。要使 NLR = NRL（后序序列反序）成立，L 或 R 应为空，这样的二叉树每层只有一个结点，即二叉树的形态是其高度等于结点个数。以 3 个结点 a, b, c 为例，其形态如下图所示。



02.【解答】

二叉树的先序序列是 NLR，后序序列是 LRN。要使 NLR = LRN 成立，L 和 R 应均为空，所以满足条件的二叉树只有一个根结点。

03.【解答】

采用层次遍历的算法，设置变量 `level` 记录当前结点所在的层数，设置变量 `last` 指向当前层的最右结点，每次层次遍历出队时与 `last` 指针比较，若两者相等，则层数加 1，并让 `last` 指向下一层的最右结点，直到遍历完成。`level` 的值即二叉树的高度。

算法实现如下：

```
int Btdepth(BiTree T){
//采用层次遍历的非递归方法求解二叉树的高度
    if(!T)
        return  0;                  //树空，高度为 0
    int front=-1,rear=-1;
    int last=0,level=0;             //last 指向当前层的最右结点
    BiTree Q[MaxSize];              //设置队列 Q，元素是二叉树结点指针且容量足够
    Q[++rear]=T;                    //将根结点入队
    BiTree p;
    while(front<rear){              //队不空，则循环
            p=Q[++front];           //队列元素出队，即正在访问的结点
        if(p->lchild)
            Q[++rear]=p->lchild;//左孩子入队
        if(p->rchild)
            Q[++rear]=p->rchild;//右孩子入队
        if(front==last){            //处理该层的最右结点
                level++;            //层数增 1
            last=rear;              //last 指向下层
        }
    }
    return  level;
}
```

求某层的结点个数、每层的结点个数、树的最大宽度等，都可采用与此题类似的思想。当然，

配套课程公众号[考机研]
永久联系微信:4550060

此题可编写为递归算法，其实现如下：

```
int Btdepth2(BiTree T){
    if(T==NULL)
        return  0;                    //空树，高度为0
    ldep=Btdepth2(T->lchild);         //左子树高度
    rdep=Btdepth2(T->rchild);         //右子树高度
    if(ldep>rdep)
         return ldep+1;               //树的高度为子树最大高度加根结点
    else
         return rdep+1;
}
```

04.【解答】

根据完全二叉树的定义，具有 n 个结点的完全二叉树与满二叉树中编号从 1～n 的结点一一对应。算法思想：采用层次遍历算法，将所有结点加入队列（包括空结点）。遇到空结点时，查看其后是否有非空结点。若有，则二叉树不是完全二叉树。

算法实现如下：

```
bool IsComplete(BiTree T){
//本算法判断给定二叉树是否为完全二叉树
    InitQueue(Q);
    if(!T)
        return true;                  //空树为满二叉树
    EnQueue(Q,T);
    while(!IsEmpty(Q)){
        DeQueue(Q,p);
        if(p){                        //结点非空，将其左、右子树入队列
            EnQueue(Q,p->lchild);
            EnQueue(Q,p->rchild);
        }
        else                          //结点为空，检查其后是否有非空结点
            while(!IsEmpty(Q)){
                DeQueue(Q,p);
                if(p)                 //结点非空，则二叉树为非完全二叉树
                    return false;
            }
    }
    return true;
}
```

05.【解答】

计算一棵二叉树 b 中所有双分支结点个数的递归模型 f(b) 如下：

```
f(b)=0                              若 b=NULL
f(b)=f(b->lchild)+f(b->rchild)+1    若*b 为双分支结点
f(b)=f(b->lchild)+f(b->rchild)      其他情况（*b 为单分支结点或叶结点）
```

具体算法实现如下：

```
int DsonNodes(BiTree b){
    if(b==NULL)
        return 0;
    else if(b->lchild!=NULL&&b->rchild!=NULL)  //双分支结点
        return  DSonNodes(b->lchild)+DsonNodes(b->rchild)+1;
    else
        return  DSonNodes(b->lchild)+DsonNodes(b->rchild);
}
```

当然，本题也可以设置一个全局变量 Num，每遍历到一个结点时，判断每个结点是否为分支

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

结点（左、右结点都不为空，注意是双分支），若是，则 Num++。

06.【解答】

采用递归算法实现交换二叉树的左、右子树，首先交换 b 结点的左孩子的左、右子树，然后交换 b 结点的右孩子的左、右子树，最后交换 b 结点的左、右孩子，当结点为空时递归结束（后序遍历的思想）。算法实现如下：

```
void swap(BiTree b){
//本算法递归地交换二叉树的左、右子树
    if(b){
        swap(b->lchild);          //递归地交换左子树
        swap(b->rchild);          //递归地交换右子树
        temp=b->lchild;           //交换左、右孩子结点
        b->lchild=b->rchild;
        b->rchild=temp;
    }
}
```

07.【解答】

设置一个全局变量 i（初值为 1）来表示进行先序遍历时，当前访问的是第几个结点。然后可以借用先序遍历的代码模型，先序遍历二叉树。当二叉树 b 为空时，返回特殊字符'#'；当 k==i 时，该结点即要找的结点，返回 b->data；当 k≠i 时，递归地在左子树中查找，若找到则返回该值，否则继续递归地在右子树中查找，并返回其结果。对应的递归模型如下：

```
f(b,k)='#'                                          当 b=NULL 时
f(b,k)=b->data                                      当 i=k 时
f(b,k)=((ch=f(b->lchild,k))=='#'?f(b->rchild,k):ch)  其他情况
```

算法的实现如下：

```
int i=1;                          //遍历序号的全局变量
ElemType PreNode(BiTree b,int k){
//本算法查找二叉树先序遍历序列中第 k 个结点的值
    if(b==NULL)                   //空结点，则返回特殊字符
        return '#';
    if(i==k)                      //相等，则当前结点即第 k 个结点
        return b->data;
    i++;                          //下一个结点
    ch=PreNode(b->lchild,k);      //左子树中递归寻找
    if(ch!='#')                   //在左子树中，则返回该值
        return ch;
    ch=PreNode(b->rchild,k);      //在右子树中递归寻找
        return ch;
}
```

本题实质上就是一个遍历算法的实现，只不过用一个全局变量来记录访问的序号，求其他遍历序列的第 k 个结点也采用相似的方法。二叉树的遍历算法可以引申出大量的算法题，因此考生务必要熟练掌握二叉树的遍历算法。

08.【解答】

删除以元素值 x 为根的子树，只要能删除其左、右子树，就可以释放值为 x 的根结点，因此宜采用后序遍历。算法思想：删除值为 x 的结点，意味着应将其父结点的左（右）子女指针置空，用层次遍历易于找到某结点的父结点。本题要求删除树中每个元素值为 x 的结点的子树，因此要遍历完整棵二叉树。算法实现如下：

```
void DeleteXTree(BiTree &bt){       //删除以 bt 为根的子树
    if(bt){
```

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

```
            DeleteXTree(bt->lchild);
            DeleteXTree(bt->rchild);      //删除 bt 的左子树、右子树
            free(bt);                     //释放被删结点所占的存储空间
        }
    }
    //在二叉树上查找所有以 x 为元素值的结点，并删除以其为根的子树
    void Search(BiTree bt,ElemType x){
        BiTree Q[];                       //Q 是存放二叉树结点指针的队列，容量足够大
        if(bt){
            if(bt->data==x){              //若根结点值为 x，则删除整棵树
                DeleteXTree(bt);
                exit(0);
            }
            InitQueue(Q);
            EnQueue(Q,bt);
            while(!IsEmpty(Q)){
                DeQueue(Q,p);
                if(p->lchild)                         //若左子女非空
                    if(p->lchild->data==x){           //左子树符合则删除左子树
                        DeleteXTree(p->lchild);
                        p->lchild=NULL;
                    }                                 //父结点的左子女置空
                    else
                        EnQueue(Q,p->lchild);         //左子树入队列
                if(p->rchild)                         //若右子女非空
                    if(p->rchild->data==x){           //右子女符合则删除右子树
                        DeleteXTree(p->rchild);
                        p->rchild=NULL;
                    }                                 //父结点的右子女置空
                    else
                        EnQueue(Q,p->rchild);         //右子女入队列
            }
        }
    }
```

09.【解答】

算法思想：采用非递归后序遍历，最后访问根结点，访问到值为 x 的结点时，栈中所有元素均为该结点的祖先，依次出栈打印即可。算法实现如下：

```
    typedef struct{
        BiTree  t;
        int     tag;
    }stack;  //tag=0 表示左子女被访问，tag=1 表示右子女被访问
    void  Search(BiTree bt,ElemType x){
    //在二叉树 bt 中，查找值为 x 的结点，并打印其所有祖先
        stack s[];                                    //栈容量足够大
        top=0;
        while(bt!=NULL||top>0){
            while(bt!=NULL&&bt->data!=x){             //结点入栈
                s[++top].t=bt;
                s[top].tag=0;
                bt=bt->lchild;                        //沿左分支向下
            }
            if(bt!=NULL&&bt->data==x){
                printf("所查结点的所有祖先结点的值为:\n");  //找到 x
                for(i=1;i<=top;i++)
                printf("%d",s[i].t->data);            //输出祖先值后结束
```

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

```
                exit(1);
            }
            while(top!=0&&s[top].tag==1)
                top--;                                  //退栈（空遍历）
            if(top!=0){
                s[top].tag=1;
                bt=s[top].t->rchild;                    //沿右分支向下遍历
            }
        } //while(bt!=NULL||top>0)
    }
```

因为查找的过程就是后序遍历的过程，所以使用的栈的深度不超过树的深度。

10.【解答】

后序遍历最后访问根结点，即在递归算法中，根是压在栈底的。本题要找 p 和 q 的最近公共祖先结点 r，不失一般性，设 p 在 q 的左边。算法思想：采用后序非递归算法，栈中存放二叉树结点的指针，当访问到某结点时，栈中所有元素均为该结点的祖先。后序遍历必然先遍历到结点 p，栈中元素均为 p 的祖先。先将栈复制到另一辅助栈中。继续遍历到结点 q 时，将栈中元素从栈顶开始逐个到辅助栈中去匹配，第一个匹配（相等）的元素就是结点 p 和 q 的最近公共祖先。算法实现如下：

```
    typedef struct{
        BiTree t;
        int tag; //tag=0 表示左子女已被访问，tag=1 表示右子女已被访问
    }stack;
    stack s[],s1[];                                 //栈，容量足够大
    BiTree Ancestor(BiTree ROOT,BiTNode *p,BiTNode *q){
    //本算法求二叉树中 p 和 q 指向结点的最近公共结点
        top=0;bt=ROOT;
        while(bt!=NULL||top>0){
            while(bt!=NULL){
                s[++top].t=bt;
                s[top].tag=0;
                bt=bt->lchild;
                }                                   //沿左分支向下
            while(top!=0&&s[top].tag==1){
            //假定 p 在 q 的左侧，遇到 p 时，栈中元素均为 p 的祖先
                if(s[top].t==p){
                    for(i=1;i<=top;i++)
                        s1[i]=s[i];
                        top1=top;
                }                                   //将栈 s 的元素转入辅助栈 s1 保存
                if(s[top].t==q)                     //找到 q 结点
                    for(i=top;i>0;i--){//将栈中元素的树结点到 s1 中去匹配
                        for(j=top1;j>0;j--)
                            if(s1[j].t==s[i].t)
                                return  s[i].t; //p 和 q 的最近公共祖先已找到
                    }
                    top--;                          //退栈
            } //while
            if(top!=0){
                s[top].tag=1;
                bt=s[top].t->rchild;
            }                                       //沿右分支向下遍历
        } //while
        return  NULL;                               //p 和 q 无公共祖先
    }
```

配套课程公众号[考机研]
永久联系微信:4550060

11.【解答】

采用层次遍历的方法求出所有结点的层次，并将所有结点和对应的层次放在一个队列中。然后通过扫描队列求出各层的结点总数，最大的层结点总数即二叉树的宽度。算法实现如下：

```
typedef struct{
    BiTree data[MaxSize];          //保存队列中的结点指针
    int level[MaxSize];            //保存 data 中相同下标结点的层次
    int front,rear;
}Qu;
int BTWidth(BiTree b){
    BiTree p;
    int k,max,i,n;
    Qu.front=Qu.rear=-1;           //队列为空
    Qu.rear++;
    Qu.data[Qu.rear]=b;            //根结点指针入队
    Qu.level[Qu.rear]=1;           //根结点层次为 1
    while(Qu.front<Qu.rear){
        Qu.front++;                //出队
        p=Qu.data[Qu.front];       //出队结点
        k=Qu.level[Qu.front];      //出队结点的层次
        if(p->lchild!=NULL){       //左孩子入队
            Qu.rear++;
            Qu.data[Qu.rear]=p->lchild;
            Qu.level[Qu.rear]=k+1;
        }
        if(p->rchild!=NULL){       //右孩子入队
            Qu.rear++;
            Qu.data[Qu.rear]=p->rchild;
            Qu.level[Qu.rear]=k+1;
        }
    }//while
    max=0;i=0;                     //max 保存同一层最多的结点个数
    k=1;                           //k 表示从第一层开始查找
    while(i<=Qu.rear){             //i 扫描队中所有元素
        n=0;                       //n 统计第 k 层的结点个数
        while(i<=Qu.rear&&Qu.level[i]==k){
            n++;
            i++;
        }
        k=Qu.level[i];
        if(n>max)   max=n;         //保存最大的 n
    }
    return max;
}
```

注 意

本题队列中的结点，在出队后仍需要保留在队列中，以便求二叉树的宽度，所以设置的队列采用非环形队列，否则在出队后可能被其他结点覆盖，无法再求二叉树的宽度。

12.【解答】

对一般二叉树，仅根据先序或后序序列，不能确定另一个遍历序列。但对满二叉树，任意一个结点的左、右子树均含有相等的结点数，同时，先序序列的第一个结点作为后序序列的最后一个结点，由此得到将先序序列 pre[l1..h1]转换为后序序列 post[l2..h2]的递归模型如下：

f(pre,l1,h1,post,l2,h2)=不做任何事情　　h1<l1 时

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

f(pre,l1,h1,post,l2, h2)=post[h2]=pre[l1]　　其他情况

取中间位置 half=(h1-l1)/2;

将 pre[l1+1,l1+half]左子树转换为 post[l2,l2+half-1],

即 f(pre,l1+1,l1+half,post,l2,l2+half-1);

将 pre[l1+half+1,h1]右子树转换为 post[l2+half,h2-1],

即 f(pre,l1+half+1,h1,post,l2+half,h2-1)。

其中，post[h2]=pre[l1]表示后序序列的最后一个结点（根结点）等于先序序列的第一个结点（根结点）。相应的算法实现如下：

```
void PreToPost(ElemType pre[],int l1,int h1,ElemType post[],int l2,int h2){
    int half;
    if(h1>=l1){
        post[h2]=pre[l1];
        half=(h1-l1)/2;
        PreToPost(pre,l1+1,l1+half,post,l2,l2+half-1);      //转换左子树
        PreToPost(pre,l1+half+1,h1,post,l2+half,h2-1);      //转换右子树
    }
}
```

例如，有以下代码：

```
ElemType *pre="ABCDEFG";
ElemType post[MaxSize];
PreToPost(pre,0,6,post,0,6);
printf("后序序列: ");
for(int i=0;i<=6;i++)
    printf("%c",post[i]);
printf("\n");
```

执行结果如下：

```
后序序列: C D B F G E A
```

13.【解答】

通常使用的先序、中序和后序遍历对于叶结点的访问顺序都是从左到右，这里选择中序递归遍历。算法思想：设置前驱结点指针 pre，初始为空。第一个叶结点由指针 head 指向，遍历到叶结点时，就将它前驱的 rchild 指针指向它，最后一个叶结点的 rchild 为空。算法实现如下：

```
LinkedList head, pre=NULL;                          //全局变量
LinkedList InOrder(BiTree bt){
    if(bt){
        InOrder(bt->lchild);                        //中序遍历左子树
        if(bt->lchild==NULL&&bt->rchild==NULL)      //叶结点
            if(pre==NULL){
                head=bt;
                pre=bt;
            }                                       //处理第一个叶结点
            else{
                pre->rchild=bt;
                pre=bt;
            }                                       //将叶结点链入链表
        InOrder(bt->rchild);                        //中序遍历右子树
        pre->rchild=NULL;                           //设置链表尾
    }
    return head;
}
```

上述算法的时间复杂度为 $O(n)$，辅助变量使用 head 和 pre，栈空间复杂度为 $O(n)$。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

14.【解答】

本题采用递归的思想求解，若 T_1 和 T_2 都是空树，则相似；若有一个为空另一个不空，则必然不相似；否则递归地比较它们的左、右子树是否相似。递归函数的定义如下：

1）f(T1,T2)=1；若 T1==T2==NULL。

2）f(T1,T2)=0；若 T1 和 T2 之一为 NULL，另一个不为 NULL。

3）f(T1,T2)=f(T1->lchild,T2->lchild)&&f(T1->rchild,T2->rchild)；若 T1 和 T2 均不为 NULL。

因此，算法实现如下：

```
int similar(BiTree T1,BiTree T2){
//采用递归的算法判断两棵二叉树是否相似
    int leftS,rightS;
    if(T1==NULL&&T2==NULL)          //两棵树皆空
        return  1;
    else if(T1==NULL||T2==NULL)     //只有一棵树为空
        return 0;
    else{                           //递归判断
        leftS=similar(T1->lchild,T2->lchild);
        rightS=similar(T1->rchild,T2->rchild);
        return leftS&&rightS;
    }
}
```

15.【解答】

二叉树的带权路径长度有两种常见的计算方法：①根据二叉树的带权路径长度的定义，二叉树的 WPL 值 = 树中全部叶结点的带权路径长度之和。②根据带权二叉树的性质，二叉树的 WPL 值 = 树中所有非叶结点的权值之和（记住该结论即可，不要求证明）。根据两种常见的计算方法，本题不难写出下列两种解法。

王道计算机教育 www.cskaoyan.com

1）算法的基本设计思想。

① 本问题可采用递归算法实现。根据定义：

二叉树的 WPL 值 = 树中全部叶结点的带权路径长度之和

= 根结点左子树中全部叶结点的带权路径长度之和 + 根结点右子树中全部叶结点的带权路径长度之和

叶结点的带权路径长度 = 该结点的 weight 域的值×该结点的深度

设根结点的深度为 0，若某结点的深度为 d 时，则其孩子结点的深度为 $d+1$。

在递归遍历二叉树结点的过程中，若遍历到叶结点，则返回该结点的带权路径长度，否则返回其左右子树的带权路径长度之和。

② 若借用非叶结点的 weight 域保存其孩子结点中 weight 域值的和，则树的 WPL 等于树中所有非叶结点 weight 域值之和。

采用后序遍历策略，在遍历二叉树 T 时递归计算每个非叶结点的 weight 域的值，则树 T 的 WPL 等于根结点左子树的 WPL 加上右子树的 WPL，再加上根结点中 weight 域的值。在递归遍历二叉树结点的过程中，若遍历到叶结点，则 return 0 并且退出递归，否则递归计算其左右子树的 WPL 和自身结点的权值。

2）二叉树结点的数据类型定义如下。

```
typedef struct node{
    int weight;
    struct node *left,*right;
}BTree;
```

配套课程公众号[考机研]
永久联系微信:4550060

3）算法的代码如下。

①基于方法 1 的算法实现：

```
int WPL(BTree *root)                    //根据 WPL 的定义采用递归算法实现
{   return WPL1(root,0);
}
int WPL1(BTree *root,int d)             //d 为结点深度
{   if(root->left==NULL&&root->right==NULL)
    return(root->weight*d);
        else
            return (WPL1(root->left,d+1)+WPL1(root->right,d+1));
}
```

②基于方法 2 的算法实现：

```
int WPL(BTree *root)                    //基于递归的后序遍历算法实现
{   int w_l,w_r;
    if(root->left==NULL&&root->right==NULL)
        return 0;
    else
    {   w_l=WPL(root->left);            //计算左子树的 WPL
        w_r=WPL(root->right);           //计算右子树的 WPL
        root->weight=root->left->weight+root->right->weight;
                                        //填写非叶结点的 weight 域
        return (w_l+w_r+root->weight);  //返回 WPL 值
    }
}
```

注　意

上述两种算法为官方标准答案，当遍历到度为 1 的结点时，会传入空指针，导致空指针异常。但是，作为 408 考试的算法题，不要求考虑特殊的边界条件，只要算法思想正确，代码逻辑正确，即可得满分。因此，在复习过程中，无须花过多的时间抠代码的各种边界条件。

16.【解答】

1）算法的基本设计思想。

表达式树的中序序列加上必要的括号即等价的中缀表达式。可以基于二叉树的中序遍历策略得到所需的表达式。

表达式树中分支结点所对应的子表达式的计算次序，由该分支结点所处的位置决定。为得到正确的中缀表达式，需要在生成遍历序列的同时，在适当位置增加必要的括号。显然，表达式的最外层（对应根结点）和操作数（对应叶结点）不需要添加括号。

2）算法实现。

将二叉树的中序遍历递归算法稍加改造即可得本题的答案。除根结点和叶结点外，遍历到其他结点时在遍历其左子树之前加上左括号，遍历完右子树后加上右括号。

```
void BtreeToE(BTree *root){
    BtreeToExp(root,1);                 //根的高度为 1
}
void BtreeToExp( BTree *root,int deep){
    if(root==NULL) return;              //空结点返回
    else if(root->left==NULL&&root->right==NULL)   //若为叶结点
        printf("%s", root->data);       //输出操作数，不加括号
    else{
        if(deep>1) printf("(");         //若有子表达式则加一层括号
```

配套课程公众号[考机研]
永久联系微信:4550060

```
            BtreeToExp(root->left, deep+1);
            printf("%s", root->data);          //输出操作符
            BtreeToExp(root->right, deep+1);
            if(deep>1) printf(")");            //若有子表达式则加一层括号
        }
    }
```

17.【解答1】

1）算法的基本设计思想。

对于采用顺序存储方式保存的二叉树，根结点保存在SqBiTNode[0]中；当某结点保存在SqBiTNode[i]中时，若有左孩子，则其值保存在SqBiTNode[2i+1]中；若有右孩子，则其值保存在SqBiTNode[2i+2]中；若有双亲结点，则其值保存在SqBiTNode[(i-1)/2]中。

二叉搜索树需要满足的条件是：任意一个结点值大于其左子树中的全部结点值，小于其右子树中的全部结点值。中序遍历二叉搜索树得到一个升序序列。

使用整型变量val记录中序遍历过程中已遍历结点的最大值，初值为一个负整数。若当前遍历的结点值小于或等于val，则算法返回false，否则，将val的值更新为当前结点的值。

2）算法实现。

```
#define false 0
#define true 1
typedef int bool;
bool judgeInOrderBST(SqBiTree bt,int k,int *val){//初始调用时k的值是0
    if(k<bt.ElemNum&&bt.SqBiTNode[k]!=-1){
        if(!judgeInOrderBST(bt,2*k+1,val)) return false;
        if(bt.SqBiTNode[k]<=*val) return false;
        *val=bt.SqBiTNode[k];
        if(!judgeInOrderBST(bt,2*k+2,val)) return false;
    }
    return true;
}
```

王道计算机教育 www.cskaoyan.com

【解答2】

1）算法的基本设计思想。

对于采用顺序存储方式保存的二叉树，根结点保存在SqBiTNode[0]中；当某结点保存在SqBiTNode[i]中时，若有左孩子，则其值保存在SqBiTNode[2i+1]中；若有右孩子，则其值保存在SqBiTNode[2i+2]中；若有双亲结点，则其值保存在SqBiTNode[(i-1)/2]中。

二叉搜索树需要满足的条件是：任意一个结点值大于其左子树中的全部结点值，小于其右子树中的全部结点值。设置两个数组pmax和pmin。根据二叉搜索树的定义，SqBiTNode[i]中的值应该大于以SqBiTNode[2i+1]为根的子树中的最大值（保存在pmax[2i+1]中），小于以SqBiTNode[2i+2]为根的子树中的最小值（保存在pmin[2i+1]中）。初始时，用数组SqBiTNode中前ElemNum个元素的值对数组pmax和pmin初始化。

在数组SqBiTNode中从后向前扫描，扫描过程中逐一验证结点与子树之间是否满足上述的大小关系。

2）算法实现。

```
#define false 0
#define true 1
typedef int bool;
bool judgeBST(SqBiTree bt){
    int k,m,*pmin,*pmax;
    pmin=(int *)malloc(sizeof(int)*(bt.ElemNum));
    pmax=(int *)malloc(sizeof(int)*(bt.ElemNum));
    for(k=0;k<bt.ElemNum;k++)          //辅助数组初始化
```

配套课程公众号[考机研]
永久联系微信:4550060

```
            pmin[k]=pmax[k]=bt.SqBiTNode[k];
        for(k=bt.ElemNum-1;k>0;k--){      //从最后一个叶结点向根遍历
            if(bt.SqBiTNode[k]!=-1){
                m=(k-1)/2;                //双亲
                if(k%2==1&&bt.SqBiTNode[m]>pmax[k])          //其为左孩子
                    pmin[m]=pmin[k];
                else if(k%2==0&&bt.SqBiTNode[m]<pmin[k])     //其为右孩子
                    pmax[m]=pmax[k];
                else  return false;
            }
        }
        return true;
    }
```

5.4　树、森林

5.4.1　树的存储结构

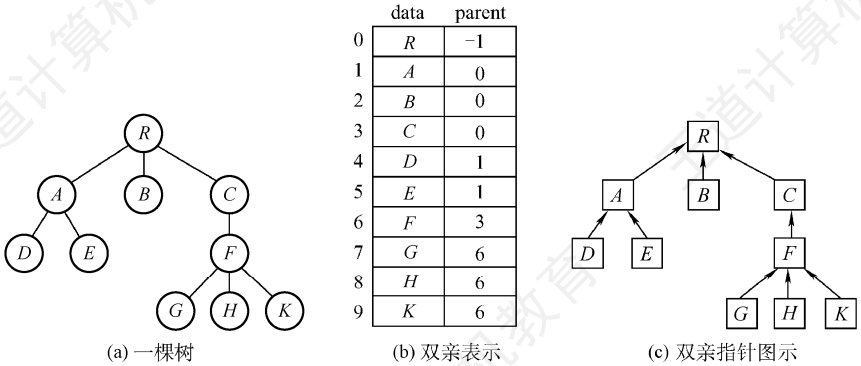
树的存储方式有多种，既可采用顺序存储结构，又可采用链式存储结构，但无论采用何种存储方式，都要求能唯一地反映树中各结点之间的逻辑关系，这里介绍 3 种常用的存储结构。

1．双亲表示法

这种存储结构采用一组连续空间来存储每个结点，同时在每个结点中增设一个伪指针，指示其双亲结点在数组中的位置。如图 5.20 所示，根结点下标为 0，其伪指针域为−1。

双亲表示法的存储结构描述如下：

```
#define MAX_TREE_SIZE 100          //树中最多结点数
typedef struct{                    //树的结点定义
    ElemType data;                 //数据元素
    int parent;                    //双亲位置域
}PTNode;
typedef struct{                    //树的类型定义
    PTNode nodes[MAX_TREE_SIZE];   //双亲表示
    int n;                         //结点数
}PTree;
```



(a) 一棵树　(b) 双亲表示　(c) 双亲指针图示

图 5.20　树的双亲表示法

双亲表示法利用了每个结点（根结点除外）只有唯一双亲的性质，可以很快地得到每个结点的双亲结点，但求结点的孩子时则需要遍历整个结构。

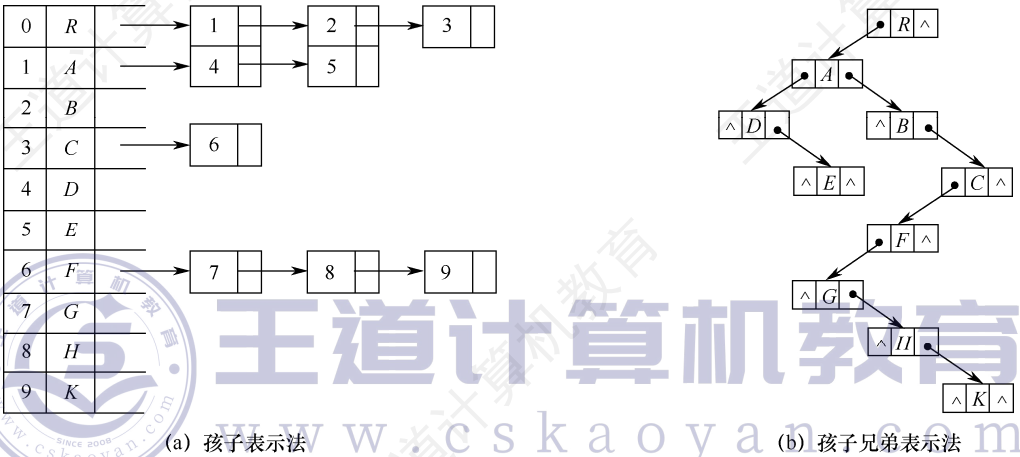
配套课程公众号[考机研]
永久联系微信:4550060

注　意

区别树的顺序存储结构与二叉树的顺序存储结构。在树的顺序存储结构中，数组下标代表结点的编号，下标中所存的内容指示了结点之间的关系。而在二叉树的顺序存储结构中，数组下标既代表了结点的编号，又指示了二叉树中各结点之间的关系。当然，二叉树属于树，因此二叉树也可用树的存储结构来存储，但树却不都能用二叉树的存储结构来存储。

2．孩子表示法

孩子表示法是将每个结点的孩子结点视为一个线性表，且以单链表作为存储结构，则 n 个结点就有 n 个孩子链表（叶结点的孩子链表为空表）。而 n 个头指针又组成一个线性表，为便于查找，可采用顺序存储结构。图 5.21(a)是图 5.20(a)中的树的孩子表示法。



(a) 孩子表示法　　(b) 孩子兄弟表示法

图 5.21　树的孩子表示法和孩子兄弟表示法

与双亲表示法相反，孩子表示法寻找孩子的操作非常方便，而寻找双亲的操作则需要遍历 n 个结点中孩子链表指针域所指向的 n 个孩子链表。

3．孩子兄弟表示法

孩子兄弟表示法也称二叉树表示法，即以二叉链表作为树的存储结构。孩子兄弟表示法使每个结点包括三部分内容：结点值、指向结点第一个孩子结点的指针，以及指向结点下一个兄弟结点的指针（沿此域可以找到结点的所有兄弟结点），如图 5.21(b)所示。

孩子兄弟表示法的存储结构描述如下：

```
typedef struct CSNode{
    ElemType data;                          //数据域
    struct CSNode *firstchild,*nextsibling;     //第一个孩子和右兄弟指针
}CSNode,*CSTree;
```

孩子兄弟表示法比较灵活，其最大的优点是可以方便地实现树转换为二叉树的操作，易于查找结点的孩子等，但缺点是从当前结点查找其双亲结点比较麻烦。若为每个结点增设一个 parent 域指向其父结点，则查找结点的父结点也很方便。

5.4.2　树、森林与二叉树的转换

二叉树和树都可以用二叉链表作为存储结构。从物理结构上看，树的孩子兄弟表示法与二叉树的二叉链表表示法是相同的，因此可以用同一存储结构的不同解释将一棵树转换为二叉树。

配套课程公众号[考机研]
永久联系微信:4550060

1．树转换为二叉树

命题追踪 ▶▶ 树和二叉树的转换及相关性质的推理（2009、2011）

树转换为二叉树的规则：每个结点的左指针指向它的第一个孩子，右指针指向它在树中的相邻右兄弟，这个规则也称*左孩子右兄弟*。根结点没有兄弟，因此树转换得到的二叉树没有右子树，如图 5.22 所示。

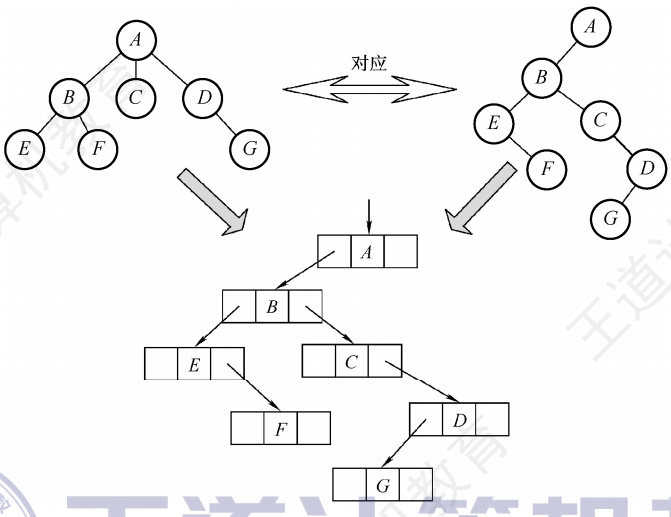


图 5.22　树与二叉树的对应关系

树转换为二叉树的画法：

1）在兄弟结点之间加一连线；

2）对每个结点，只保留它与第一个孩子的连线，而与其他孩子的连线全部抹掉；

3）以树根为轴心，顺时针旋转 45°。

2．森林转换为二叉树

命题追踪 ▶▶ 森林和二叉树的转换及相关性质的推理（2014）

将森林转换为二叉树的规则与树类似。先将森林中的每棵树转换为二叉树，由于任意一棵树对应的二叉树的右子树必空，森林中各棵树的根也可视为兄弟关系，将第二棵树对应的二叉树当作第一棵二叉树根的右子树……以此类推，就可以将森林转换为二叉树。

森林转换为二叉树的画法：

1）将森林中的每棵树转换成相应的二叉树；

2）每棵树的根也可视为兄弟关系，在每棵树的根之间加一根连线；

3）以第一棵树的根为轴心顺时针旋转 45°。

或者先在森林中每棵树的根之间加一根连线，然后再采用树转换为二叉树的方法。

3．二叉树转换为森林

命题追踪 ▶▶ 由遍历序列构造一棵二叉树并转换为对应的森林（2020、2021）

二叉树转换为森林的规则：若二叉树非空，则二叉树的根及其左子树为第一棵树的二叉树形式，所以将根的右链断开。二叉树根的右子树又可视为一个由除第一棵树外的森林转换后的二叉树，应用同样的方法，直到最后只剩一棵没有右子树的二叉树为止，最后将每棵二叉树依次转换

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

成树，就得到了原森林，如图 5.23 所示。二叉树转换为树或森林是唯一的。

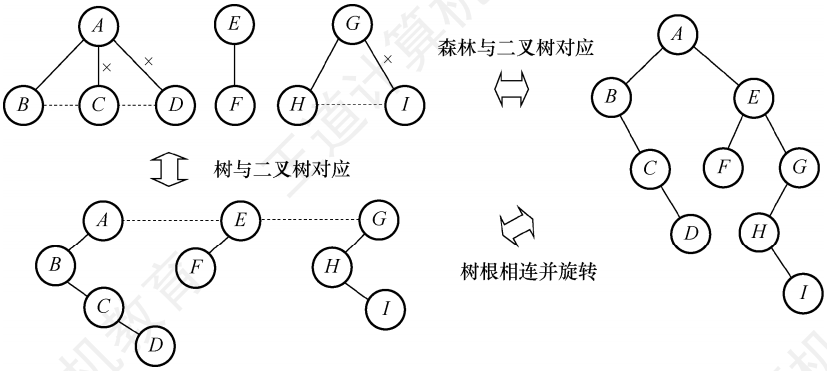


图 5.23　森林与二叉树的对应关系

5.4.3　树和森林的遍历

1．树的遍历

命题追踪 ▶▶ 树与二叉树遍历方法的对应关系（2019）

树的遍历是指用某种方式访问树中的每个结点，且仅访问一次。主要有两种方式：

1）先根遍历。若树非空，则按如下规则遍历：

- 先访问根结点。
- 再依次遍历根结点的每棵子树，遍历子树时仍遵循先根后子树的规则。

其遍历序列与这棵树相应二叉树的先序序列相同。

2）后根遍历。若树非空，则按如下规则遍历：

- 先依次遍历根结点的每棵子树，遍历子树时仍遵循先子树后根的规则。
- 再访问根结点。

其遍历序列与这棵树相应二叉树的中序序列相同。

图 5.22 的树的先根遍历序列为 *ABEFCDG*，后根遍历序列为 *EFBCGDA*。

另外，树也有层次遍历，与二叉树的层次遍历思想基本相同，即按层序依次访问各结点。

2．森林的遍历

按照森林和树相互递归的定义，可得到森林的两种遍历方法。

1）先序遍历森林。若森林为非空，则按如下规则遍历：

- 访问森林中第一棵树的根结点。
- 先序遍历第一棵树中根结点的子树森林。
- 先序遍历除去第一棵树之后剩余的树构成的森林。

2）中序遍历森林。森林为非空时，按如下规则遍历：

- 中序遍历森林中第一棵树的根结点的子树森林。
- 访问第一棵树的根结点。
- 中序遍历除去第一棵树之后剩余的树构成的森林。

图 5.23 的森林的先序遍历序列为 *ABCDEFGHI*，中序遍历序列为 *BCDAFEHIG*。

命题追踪 ▶▶ 森林与二叉树遍历方法的对应关系（2020）

当森林转换成二叉树时，其第一棵树的子树森林转换成左子树，剩余树的森林转换成右子树，

王道计算机教育 www.wangdaokaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

可知森林的先序和中序遍历即其对应二叉树的先序和中序遍历。

树和森林的遍历与二叉树的遍历关系见表 5.1。

表 5.1　树和森林的遍历与二叉树遍历的对应关系

树	森　林	二　叉　树
先根遍历	先序遍历	先序遍历
后根遍历	中序遍历	中序遍历

注　意

森林的遍历方法的命名按照严蔚敏老师的经典教材。有些教材也将森林的中序遍历称为后序遍历，称中序遍历是相对其二叉树而言的，称后序遍历是因为根确实是最后才访问。

5.4.4　本节试题精选

一、单项选择题

01. 下列关于树的说法中，正确的是（　）。

I. 对于有 n 个结点的二叉树，其高度为 $\log_2 n$

II. 完全二叉树中，若一个结点没有左孩子，则它必是叶结点

III. 高度为 h（$h>0$）的完全二叉树对应的森林所含的树的个数一定是 h

IV. 一棵树中的叶子数一定等于与其对应的二叉树的叶子数

A. I 和 III　　B. IV　　C. I 和 II　　D. II

02. 利用二叉链表存储森林时，根结点的右指针是（　）。

A. 指向最左兄弟　　B. 指向最右兄弟　　C. 一定为空　　D. 不一定为空

03. 设森林 F 中有 3 棵树，第 1、2、3 棵树的结点个数分别为 M_1, M_2 和 M_3，与森林 F 对应的二叉树根结点的右子树上的结点个数是（　）。

A. M_1　　B. M_1+M_2　　C. M_3　　D. M_2+M_3

04. 设森林 F 中有 4 棵树，第 1、2、3、4 棵树的结点数分别为 a、b、c 和 d，与森林 F 对应的二叉树的根结点的左子树上的结点数是（　）。

A. a　　B. $b+c+d$　　C. $a-1$　　D. $a+b+c$

05. 设森林 F 对应的二叉树为 B，它有 m 个结点，B 的根为 p，p 的右子树结点数为 n，森林 F 中第一棵树的结点数是（　）。

A. $m-n$　　B. $m-n-1$　　C. $n+1$　　D. 条件不足，无法确定

06. 设森林 F 对应的二叉树是一棵具有 16 个结点的完全二叉树，则森林 F 中树的数目和结点最多的树的结点数分别是（　）。

A. 2 和 8　　B. 2 和 9　　C. 4 和 8　　D. 4 和 9

07. 森林 $T=(T_1, T_2, \cdots, T_m)$ 转化为二叉树 BT 的过程为：若 $m=0$，则 BT 为空，若 $m\neq 0$，则（　）。

A. 将中间子树 T_{mid}（$\text{mid}=(1+m)/2$）的根作为 BT 的根；将$(T_1, T_2, \cdots, T_{\text{mid}-1})$转换为 BT 的左子树；将$(T_{\text{mid}+1}, \cdots, T_m)$转换为 BT 的右子树

B. 将子树 T_1 的根作为 BT 的根；将 T_1 的子树森林转换成 BT 的左子树；将$(T_2, T_3, \cdots, T_m)$转换成 BT 的右子树

C. 将子树 T_1 的根作为 BT 的根；将 T_1 的左子树森林转换成 BT 的左子树；将 T_1 的右子树森林转换为 BT 的右子树；其他以此类推

配套课程公众号[考机研]
永久联系微信:4550060

D. 将森林 T 的根作为 BT 的根；将$(T_1, T_2, \cdots, T_m)$转化为该根下的结点，得到一棵树，然后将这棵树再转化为二叉树 BT

08. 设 F 是一个森林，B 是由 F 变换来的二叉树。若 F 中有 n 个非终端结点，则 B 中右指针域为空的结点有（ ）个。

A. $n-1$　　B. n　　C. $n+1$　　D. $n+2$

09. 设某树的孩子兄弟链表示中共有 6 个空的左指针域、7 个空的右指针域，包括 5 个结点的左、右指针域都为空，则该树中叶结点的个数是（ ）。

A. 7　　B. 6　　C. 5　　D. 不能确定

10. 若 T_1 是由有序树 T 转换而来的二叉树，则 T 中结点的后根序列就是 T_1 中结点的（ ）序列。

A. 先序　　B. 中序　　C. 后序　　D. 层序

11. 某二叉树结点的中序序列为 $BDAECF$，后序序列为 $DBEFCA$，则该二叉树对应的森林包括（ ）棵树。

A. 1　　B. 2　　C. 3　　D. 4

12. 设 X 是树 T 中的一个非根结点，B 是 T 所对应的二叉树。在 B 中，X 是其双亲结点的右孩子，下列结论中正确的是（ ）。

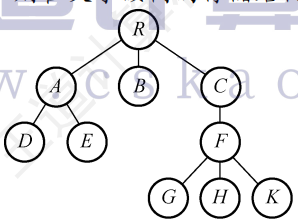
A. 在树 T 中，X 是其双亲结点的第一个孩子

B. 在树 T 中，X 一定无右边兄弟

C. 在树 T 中，X 一定是叶结点

D. 在树 T 中，X 一定有左边兄弟

13. 下图是一棵逻辑上的树 T，则在关于该树的存储结构的叙述中，错误的是（ ）。



A. 若 T 采用双亲表示法，则有 9 个指向双亲的指针

B. 若 T 采用孩子表示法，则在 T 中查找某个结点的孩子比双亲表示法更方便

C. 若 T 采用孩子兄弟表示法，则在 T 中查找某个结点的双亲的时间复杂度为 $O(1)$

D. 双亲表示法是顺序存储结构，孩子表示法和孩子兄弟表示法是链式存储结构

14. 在森林的二叉树表示中，结点 M 和结点 N 是同一父结点的左儿子和右儿子，则在该森林中（ ）。

A. M 和 N 有同一双亲　　B. M 和 N 可能无公共祖先

C. M 是 N 的儿子　　D. M 是 N 的左兄弟

15. 【2009 统考真题】将森林转换为对应的二叉树，若在二叉树中，结点 u 是结点 v 的父结点的父结点，则在原来的森林中，u 和 v 可能具有的关系是（ ）。

I. 父子关系　II. 兄弟关系　III. u 的父结点与 v 的父结点是兄弟关系

A. 只有 II　　B. I 和 II　　C. I 和 III　　D. I、II 和 III

16. 【2011 统考真题】已知一棵有 2011 个结点的树，其叶结点个数为 116，该树对应的二叉树中无右孩子的结点个数是（ ）。

A. 115　　B. 116　　C. 1895　　D. 1896

17. 【2014 统考真题】将森林 F 转换为对应的二叉树 T，F 中叶结点的个数等于（ ）。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

A. T 中叶结点的个数　　B. T 中度为 1 的结点个数

C. T 中左孩子指针为空的结点个数　　D. T 中右孩子指针为空的结点个数

18. 【2019 统考真题】若将一棵树 T 转化为对应的二叉树 BT，则下列对 BT 的遍历中，其遍历序列与 T 的后根遍历序列相同的是（　）。

A. 先序遍历　　B. 中序遍历

C. 后序遍历　　D. 按层遍历

19. 【2020 统考真题】已知森林 F 及与之对应的二叉树 T，若 F 的先根遍历序列是 a, b, c, d, e, f，后根遍历序列是 b, a, d, f, e, c，则 T 的后序遍历序列是（　）。

A. b, a, d, f, e, c　　B. b, d, f, e, c, a

C. b, f, e, d, c, a　　D. f, e, d, c, b, a

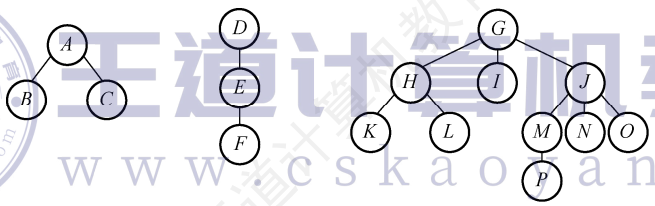
20. 【2021 统考真题】某森林 F 对应的二叉树为 T，若 T 的先序遍历序列是 a, b, d, c, e, g, f，中序遍历序列是 b, d, a, e, g, c, f，则 F 中树的棵数是（　）。

A. 1　　B. 2　　C. 3　　D. 4

二、综合应用题

01. 给定一棵树的先根遍历序列和后根遍历序列，能否唯一确定一棵树？若能，请举例说明；若不能，请给出反例。

02. 将下面一个由 3 棵树组成的森林转换为二叉树。



03. 已知某二叉树的先序序列和中序序列分别为 *ABDEHCFIMGJKL* 和 *DBHEAIM FCGKLJ*，请画出这棵二叉树，并画出二叉树对应的森林。

04. 编程求以孩子兄弟表示法存储的森林的叶结点数。

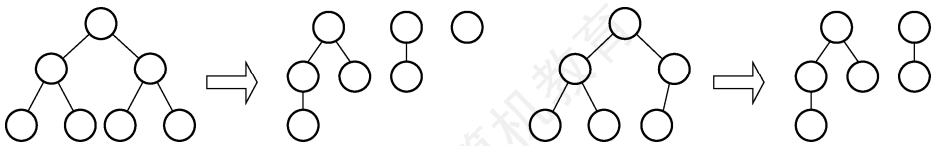
05. 以孩子兄弟链表为存储结构，请设计递归算法求树的深度。

5.4.5　答案与解析

一、单项选择题

01. D

若 n 个结点的二叉树是一棵单支树，则其高度为 n。完全二叉树中最多存在一个度为 1 的结点且只有左孩子，若不存在左孩子，则一定也不存在右孩子，因此必是叶结点，选项 II 正确。只有满二叉树才具有性质 III，如下图所示。



(a) 满二叉树转化为对应的森林　　(b) 非满二叉树转化为对应的森林

在树转换为二叉树时，若有几个叶结点具有共同的双亲，则转换成二叉树后只有一个叶结点（最右边的叶结点），如下图所示，选项 IV 错误。注意，若树中的任意两个叶结点都不存在相同

配套课程公众号[考机研]
永久联系微信:4550060

的双亲，则树中的叶子数才有可能与其对应的二叉树中的叶子数相等。

02．D

森林与二叉树具有对应关系，因此，我们存储森林时应先将森林转换成二叉树，转换的方法就是“左孩子右兄弟”，与树不同的是，若存在第二棵树，则二叉链表的根结点的右指针指向的是森林中的第二棵树的根结点。若此森林只有一棵树，则根结点的右指针为空。因此，右指针可能为空也可能不为空。

03．D

与树转换为二叉树不同，森林中的每棵树是独立的，因此先要将每棵树的根结点全部视为兄弟结点的关系。森林转换为二叉树后，树 2 作为树 1 的根结点的右子树，树 3 作为树 2 的根结点的右子树，因此森林 F 对应的二叉树根结点的右子树上的结点个数是 M_2+M_3。

04．C

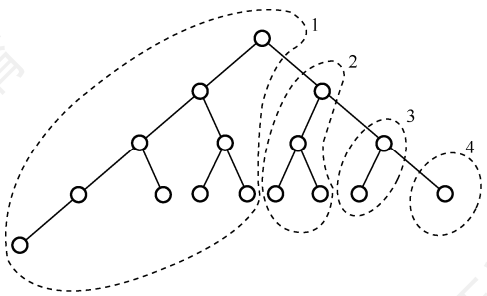
森林转换为二叉树后，二叉树的根结点为第 1 棵树的根结点，二叉树的根结点的左子树包含第 1 棵树的所有孩子，因此森林 F 对应的二叉树的根结点的左子树上的结点数是 $a-1$。

05．A

森林转换成二叉树时采用孩子兄弟表示法，根结点及其左子树为森林中的第一棵树。右子树为其他剩余的树。所以，第一棵树的结点个数为 $m-n$。

06．D

森林转换为二叉树后，二叉树的根结点及其左子树由第 1 棵树转换得到，二叉树的根结点的右子树由剩余的森林转换得到，以此类推，可以划分出第 2, 3,… 棵树的结点。具有 16 个结点的完全二叉树的形态如下图所示，沿二叉树的根结点往右下遍历，共有 4 个结点，可知森林中有 4 棵树，其中第 1 棵树的结点数最多，有 9 个。



07．B

将森林中每棵树的根结点视为兄弟结点的关系，再按照“左孩子右兄弟”的规则来进行转化。

08．C

根据森林与二叉树转换规则“左孩子右兄弟”。二叉树 B 中右指针域为空代表该结点没有兄弟结点。森林中每棵树的根结点从第二个开始依次连接到前一棵树的根的右孩子，因此最后一棵树的根结点的右指针为空。另外，每个非终端结点，其所有孩子结点在转换之后，最后一个孩子的右指针也为空，所以树 B 中右指针域为空的结点有 $n+1$ 个。

09．B

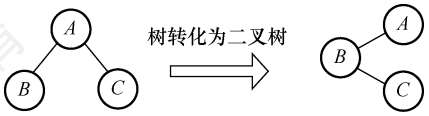
在树的孩子兄弟表示法中，若一个结点没有孩子（叶结点），则表现为该结点的左指针域为

配套课程公众号[考机研]
永久联系微信:4550060

空，因此本题答案为“6”。至于“5 个结点的左、右指针域都为空”，表示树中有 5 个结点既没有孩子又没有右兄弟，约束条件比题中的“求叶结点的个数”要求更严格。

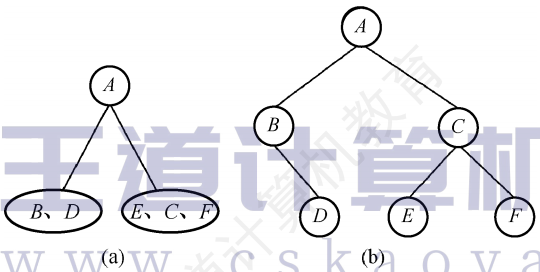
10．B

有序树 T 转换成二叉树 T_1 时，T 的后根序列是对应 T_1 的中序序列还是后序序列呢（显然树的后根序列不可能对应二叉树的先序序列和层序序列）？看下图所示的例子，在树 T 中，叶结点 B 应最先访问，在 T_1 中，B 的右兄弟 C 转换为它的右孩子，若对应 T_1 的后序序列，则 C 应在 B 的前面访问，所以 T 的后根序列不可能对应 T_1 的后序序列。



11．C

根据二叉树的前序序列和中序序列可以确定一棵二叉树。根据后序序列，A 是二叉树的根结点。根据中序序列，二叉树的形态如下图(a)所示。对于 A 的左子树，根据后序序列，B 比 D 后被访问，因此 B 必为 D 的父结点，又根据中序序列，D 是 B 的右儿子。对于 A 的右子树，同理可确定结点 E、C、F 的关系。此二叉树的形态如下图(b)所示。



再根据二叉树与森林的对应关系，森林中树的棵数即其对应二叉树（向右上旋转 45°后）的根结点 A 及其右兄弟数，或解释为：对应二叉树从根结点 A 开始不断往右孩子访问，所访问到的结点数。可知此森林中有 3 棵树，根结点分别为 A, C 和 F。

12．D

在二叉树 B 中，X 是其双亲的右孩子，因此在树 T 中，X 必是其双亲结点的右兄弟，换句话说，X 在树中必有左兄弟。

13．C

若 T 采用双亲表示法存储，则除根结点外，其余每个结点都有指向其双亲的指针，T 共有 10 个结点，于是有 9 个指向双亲的指针，选项 A 正确。若 T 采用孩子表示法存储，则每个结点的孩子被视为一个线性表，且以单链表作为存储结构，只要遍历该单链表，就能找到某个结点的所有孩子，而双亲表示法要寻找某个结点的孩子，就必须遍历整棵树，选项 B 正确。若 T 采用孩子兄弟表示法，则在 T 中查找某个结点的双亲也必须遍历整棵树，时间复杂度为 $O(n)$，选项 C 错误。选项 D 显然正确。

14．B

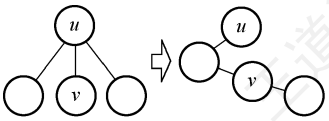
在森林的二叉树表示中，当 M 和 N 的父结点是二叉树根结点时，M 和 N 在不同的树上。因此 M 和 N 可能无公共祖先。

15．B

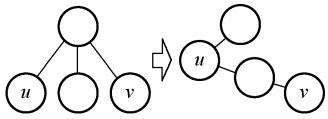
森林与二叉树的转换规则为“左孩子右兄弟”。在最后生成的二叉树中，父子关系在对应森林关系中可能是兄弟关系或者原本就是父子关系。

配套课程公众号[考机研]
永久联系微信:4550060

情形I：若结点 v 是结点 u 的第二个孩子结点，转换时，结点 v 就变成结点 u 第一个孩子的右孩子，符合要求。情形II：结点 u 和 v 是兄弟结点的关系，但二者之中还有一个兄弟结点 k，则转换后结点 v 就变为结点 k 的右孩子，而结点 k 则是结点 u 的右孩子，符合要求。

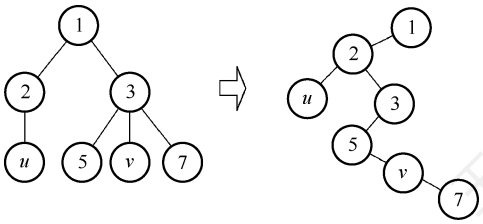


图I



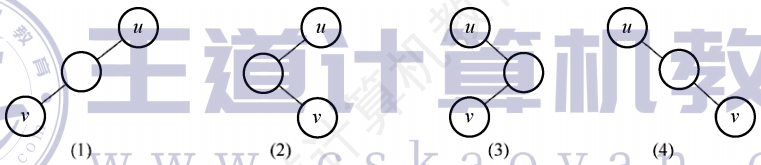
图II

情形III：若结点 u 的父结点与 v 的父结点是兄弟关系，则转换后，结点 u 和 v 分别在两者最左父结点的两棵子树中，不可能出现在同一条路径中。



图III

【另解】由题意可知 u 是 v 的父结点的父结点，如下图所示，有四种情况：

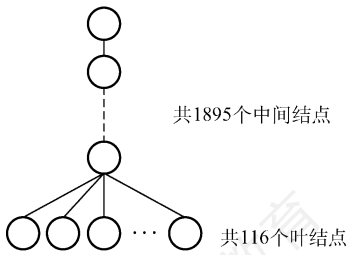


根据树与二叉树的转换规则，将这四种情况转换成树中结点的关系。（1）在原来的树中 u 是 v 的父结点的父结点；（2）在树中 u 是 v 的父结点；（3）在树中 u 是 v 的父结点的兄弟；（4）在树中 u 与 v 是兄弟关系。由此可知选项I和II正确。

16．D

树转换为二叉树时，树的每个分支结点的所有子结点中的最右子结点无右孩子，根结点转换后也没有右孩子，因此，对应二叉树中无右孩子的结点个数＝分支结点数＋1＝2011－116＋1＝1896。

通常本题应采用特殊法求解，设题意中的树是如下图所示的结构，则对应的二叉树中仅有前115个叶结点有右孩子，所以无右孩子的结点个数＝2011－115＝1896。



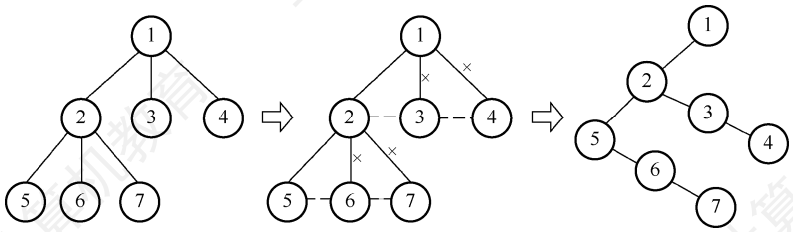
17．C

将森林转化为二叉树相当于用孩子兄弟表示法来表示森林。在变化过程中，原森林某结点的第一个孩子结点作为它的左子树，它的兄弟作为它的右子树。森林中的叶结点没有孩子结点，转化为二叉树时，该结点就没有左结点，因此 F 中叶结点的个数等于 T 中左孩子指针为空的结点个数。此题还可通过一些特例来排除选项A、B和D。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

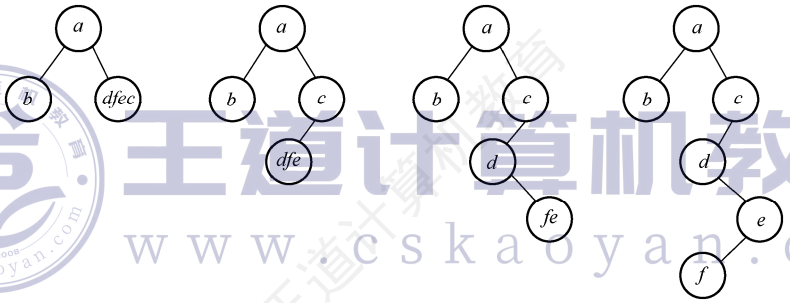
18．B

后根遍历树可分为两步：①从左到右访问双亲结点的每个孩子（转化为二叉树后，先访问根结点，再访问右子树）；②访问完所有孩子后再访问它们的双亲结点（转化为二叉树后，先访问左子树，再访问根结点），因此树的后根遍历序列与其相应二叉树的中序遍历序列相同。对于此类题，采用特殊值法求解通常会更便捷，左下图树 T 转换为二叉树 BT 的过程如下图所示，树的后序遍历序列显然和其相应二叉树的中序遍历序列相同，均为 5,6,7,2,3,4,1。



19．C

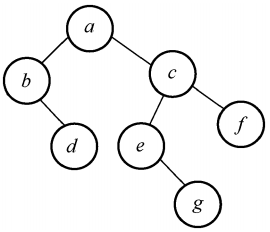
森林 F 的先根遍历序列对应于其二叉树 T 的先序遍历序列，森林 F 的后根遍历序列对应于其二叉树 T 的中序遍历序列。即 T 的先序遍历序列为 a, b, c, d, e, f，中序遍历序列为 b, a, d, f, e, c。根据二叉树 T 的先序序列和中序序列可以唯一确定它的结构，构造过程如下：



可以得到二叉树 T 的后序序列为 b, f, e, d, c, a。

20．C

由二叉树 T 的先序序列和中序序列可以构造出 T，如下图所示。由森林转化成二叉树的规则可知，森林中每棵树的根结点以右子树的方式相连，所以 T 中的结点 a、c、f 为 F 中树的根结点，森林 F 中有 3 棵树。

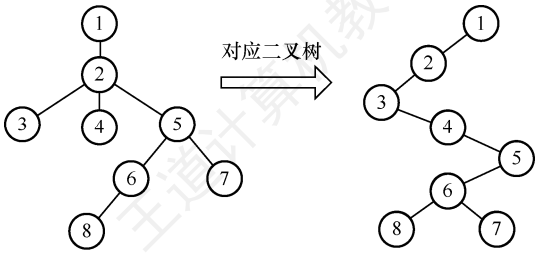


二、综合应用题

01．【解答】

一棵树的先根遍历结果与其对应二叉树的先序遍历结果相同，树的后根遍历结果与其对应二叉树表示的中序遍历结果相同。二叉树的先序序列和中序序列能够唯一地确定这棵二叉树，因此，根据题目给出的条件，利用树的先根遍历序列和后根遍历序列能够唯一地确定这棵树。例如，对于下图所示的树，对应二叉树的先序序列为 1, 2, 3, 4, 5, 6, 8, 7，中序序列为 3, 4, 8, 6, 7, 5, 2, 1。原树的先根遍历序列为 1, 2, 3, 4, 5, 6, 8, 7，后根遍历序列为 3, 4, 8, 6, 7, 5, 2, 1。

配套课程公众号[考机研]
永久联系微信:4550060

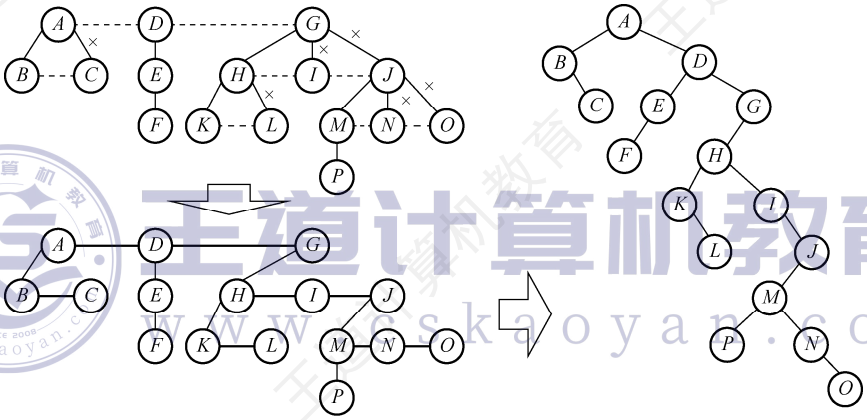


注 意

树的先根遍历、后根遍历与对应二叉树的前序遍历、中序遍历对应。

02.【解答】

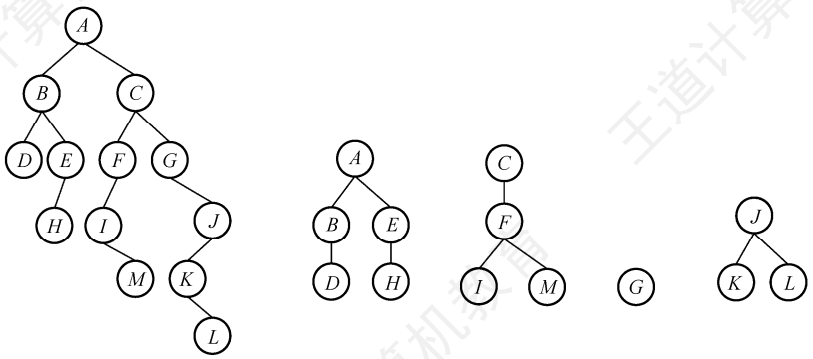
根据树与二叉树“左孩子右兄弟”的转换规则，将森林转换为二叉树的过程如下：①将每棵树的根结点也视为兄弟关系，在兄弟结点之间加一连线。②对每个结点，只保留它与第一个子结点的连线，与其他子结点的连线全部抹掉。③以树根为轴心，顺时针旋转45°。结果如下图所示。



03.【解答】

知道二叉树的先序和中序遍历后，可以唯一确定这棵树的结构。然后把二叉树转换到树和森林的方式是，若结点 x 是双亲 y 的左孩子，则把 x 的右孩子、右孩子的右孩子……都与 y 用连线连起来，最后去掉所有双亲到右孩子的连线。

最后得到的二叉树及对应的森林如下图所示。



04.【解答】

当森林（树）以孩子兄弟表示法存储时，若结点没有孩子（fch=NULL），则它必是叶子，总的叶结点个数是孩子子树（fch）上的叶子数和兄弟子树（nsib）上的叶结点个数之和。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

算法代码如下：

```
typedef struct node
{
    ElemType    data;                   //数据域
    struct node *fch,*nsib;             //孩子与兄弟域
}*Tree;
int  Leaves(Tree t){                    //计算以孩子兄弟表示法存储的森林的叶子数
    if(t==NULL)
        return  0;                      //树空返回 0
    if(t->fch==NULL)                    //若结点无孩子，则该结点必是叶子
        return  1+Leaves(t->nsib);      //返回叶结点和其兄弟子树中的叶结点数
    else                                //孩子子树和兄弟子树中叶子数之和
        return  Leaves(t->fch)+Leaves(t->nsib);
}
```

05.【解答】

由孩子兄弟链表表示的树，求高度的算法思想：采用递归算法，若树为空，高度为零；否则，高度为第一子女树高度加 1 和兄弟子树高度的大者。其非递归算法使用队列，逐层遍历树，取得树的高度。算法代码如下：

```
int  Height(CSTree bt){
//递归求以孩子兄弟链表表示的树的深度
    int hc,hs;
    if(bt==NULL)
        return  0;
    else{//否则，高度取子女高度+1 和兄弟子树高度的大者
        hc=Height(bt->firstchild);      //第一子女树高
        hs=Height(bt->nextsibling);     //兄弟树高
        if(hc+1>hs)
            return  hc+1;
        else
            return  hs;
    }
}
```

5.5　树与二叉树的应用

5.5.1　哈夫曼树和哈夫曼编码

1．哈夫曼树的定义

在介绍哈夫曼树之前，先介绍几个相关的概念：

在许多应用中，树中结点常常被赋予一个表示某种意义的数值，称为该结点的权。

从树的根到一个结点的路径长度与该结点上权值的乘积，称为该结点的带权路径长度。

树中所有叶结点的带权路径长度之和称为该树的带权路径长度，记为

$$\mathrm{WPL}=\sum_{i=1}^{n} w_i l_i$$

式中，w_i 是第 i 个叶结点所带的权值，l_i 是该叶结点到根结点的路径长度。

在含有 n 个带权叶结点的二叉树中，其中带权路径长度（WPL）最小的二叉树称为哈夫曼树，也称最优二叉树。例如，图 5.24 中的 3 棵二叉树都有 4 个叶结点 a, b, c, d，分别带权 7, 5, 2, 4，它们的带权路径长度分别为

配套课程公众号[考机研]
永久联系微信:4550060

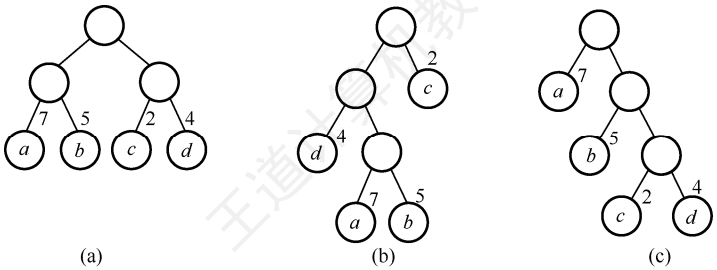


图 5.24　具有不同带权长度的二叉树

(a) WPL = 7×2 + 5×2 + 2×2 + 4×2 = 36。

(b) WPL = 4×2 + 7×3 + 5×3 + 2×1 = 46。

(c) WPL = 7×1 + 5×2 + 2×3 + 4×3 = 35。

其中，图 5.24(c)树的 WPL 最小。可以验证，它恰好为哈夫曼树。

2．哈夫曼树的构造

给定 n 个权值分别为 $w_1, w_2, \cdots, w_n$ 的结点，构造哈夫曼树的算法描述如下：

1）将这 n 个结点分别作为 n 棵仅含一个结点的二叉树，构成森林 F。

命题追踪 ▶ 分析哈夫曼树的路径上权值序列的合法性（2010）

2）构造一个新结点，从 F 中选取两棵根结点权值最小的树作为新结点的左、右子树，并且将新结点的权值置为左、右子树上根结点的权值之和。

3）从 F 中删除刚才选出的两棵树，同时将新得到的树加入 F 中。

4）重复步骤 2）和 3），直至 F 中只剩下一棵树为止。

命题追踪 ▶ 哈夫曼树的性质（2010、2019）

从上述构造过程中可以看出哈夫曼树具有如下特点：

1）每个初始结点最终都成为叶结点，且权值越小的结点到根结点的路径长度越大。

2）构造过程中共新建了 $n-1$ 个结点（双分支结点），因此哈夫曼树的结点总数为 $2n-1$。

3）每次构造都选择 2 棵树作为新结点的孩子，因此哈夫曼树中不存在度为 1 的结点。

例如，权值{7, 5, 2, 4}的哈夫曼树的构造过程如图 5.25 所示。

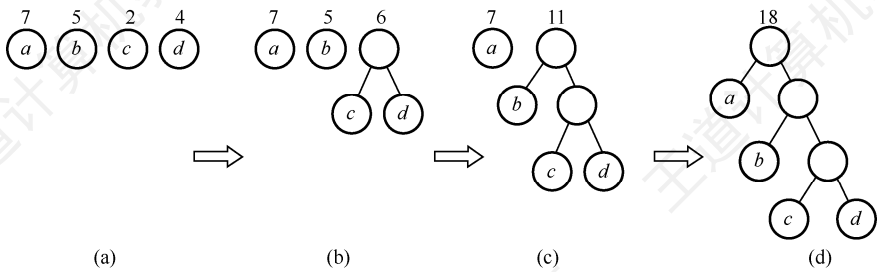


图 5.25　哈夫曼树的构造过程

3．哈夫曼编码

在数据通信中，若对每个字符用相等长度的二进制位表示，称这种编码方式为固定长度编码。若允许对不同字符用不等长的二进制位表示，则这种编码方式称为可变长度编码。可变长度编码比固定长度编码要好得多，其特点是对频率高的字符赋以短编码，而对频率较低的字符则赋以较长一些的编码，从而可以使字符的平均编码长度减短，起到压缩数据的效果。

配套课程公众号[考机研]
永久联系微信:4550060

命题追踪 ▶▶ 根据哈夫曼编码对编码序列进行译码（2017）

若没有一个编码是另一个编码的前缀，则称这样的编码为前缀编码。举例：设计字符 A, B 和 C 对应的编码 0, 10 和 110 是前缀编码。对前缀编码的解码很简单，因为没有一个编码是其他编码的前缀。所以识别出第一个编码，将它翻译为原字符，再对剩余的码串执行同样的解码操作。例如，码串 0010110 可被唯一地翻译为 A, A, B 和 C。另举反例：若再将字符 D 的编码设计为 11，此时 11 是 110 的前缀，则上述码串的后三位就无法唯一翻译。

命题追踪 ▶▶ 哈夫曼树的构造及相关的分析（2012、2018、2021、2023）

命题追踪 ▶▶ 前缀编码的分析及应用（2014、2020）

可以利用二叉树来设计二进制前缀编码。假设为 A, B, C, D 四个字符设计前缀编码，可以用图 5.26 所示的二叉树来表示，4 个叶结点分别表示 4 个字符，且约定左分支表示 0，右分支表示 1，从根到叶结点的路径上用分支标记组成的序列作为该叶结点字符的编码，可以证明如此得到的必为前缀编码。由图 5.26 得到字符 A, B, C, D 的前缀编码分别为 0, 10, 110, 111。

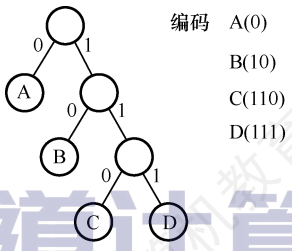


图 5.26　前缀编码示例

命题追踪 ▶▶ 哈夫曼编码和定长编码的差异（2022）

哈夫曼编码是一种非常有效的数据压缩编码。由哈夫曼树得到哈夫曼编码是很自然的过程。首先，将每个字符当作一个独立的结点，其权值为它出现的频度（或次数），构造出对应的哈夫曼树。然后，将从根到叶结点的路径上分支标记的字符串作为该字符的编码。图 5.27 所示为一个由哈夫曼树构造哈夫曼编码的示例，矩形方块表示字符及其出现的次数。

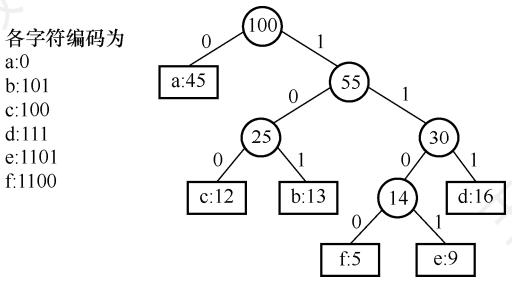


图 5.27　由哈夫曼树构造哈夫曼编码

这棵哈夫曼树的 WPL 为

$$\mathrm{WPL} = 1\times45 + 3\times(13 + 12 + 16) + 4\times(5 + 9) = 224$$

此处的 WPL 可视为最终编码得到二进制编码的长度，共 224 位。若采用 3 位固定长度编码，则得到的二进制编码长度为 300 位，因此哈夫曼编码共压缩了 25%的数据。利用哈夫曼树可以设计出总长度最短的二进制前缀编码。

王道计算机教育 www.wangdaokaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

注　意

左分支和右分支究竟是表示 0 还是表示 1 没有明确规定，因此构造出的哈夫曼树并不唯一，但各哈夫曼树的带权路径长度 WPL 相同且为最优。此外，如有若干权值相同的结点，则构造出的哈夫曼树更可能不同，但 WPL 必然相同且为最优。

5.5.2　并查集

1．并查集的概念

并查集是一种简单的集合表示，它支持以下 3 种操作：

1）`Initial(S)`：将集合 `S` 中的每个元素都初始化为只有一个单元素的子集合。

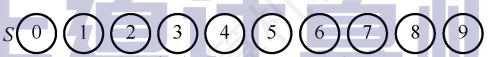
2）`Union(S,Root1,Root2)`：把集合 `S` 中的子集合 `Root2` 并入子集合 `Root1`。要求 `Root1` 和 `Root2` 互不相交，否则不执行合并。

3）`Find(S,x)`：查找集合 `S` 中单元素 `x` 所在的子集合，并返回该子集合的根结点。

2．并查集的存储结构

通常用树的双亲表示作为并查集的存储结构，每个子集合以一棵树表示。所有表示子集合的树，构成表示全集合的森林，存放在双亲表示数组内。通常用数组元素的下标代表元素名，用根结点的下标代表子集合名，根结点的双亲域为负数（可设置为该子集合元素数量的相反数）。

例如，若设有一个全集合为 $S=\{0,1,2,3,4,5,6,7,8,9\}$，初始化时每个元素自成一个单元素子集合，每个子集合的数组值为−1，如图 5.28 所示。



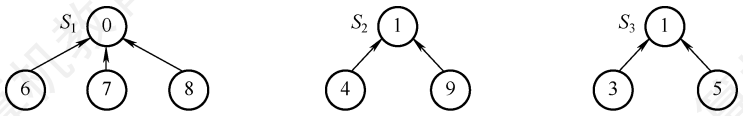
(a) 全集合S初始化时形成一个森林

0	1	2	3	4	5	6	7	8	9
−1	−1	−1	−1	−1	−1	−1	−1	−1	−1

(b) 初始化时形成的(森林)双亲表示

图 5.28　并查集的初始化

经过一段时间的计算后，这些子集合合并为 3 个更大的子集合，即 $S_1=\{0,6,7,8\}$，$S_2=\{1,4,9\}$，$S_3=\{2,3,5\}$，此时并查集的树形和存储结构如图 5.29 所示。



(a) 集合的树形表示

0	1	2	3	4	5	6	7	8	9
−4	−3	−3	2	1	2	0	0	0	1

(b) 集合S_1、S_2和S_3的(森林)双亲表示

图 5.29　用树表示并查集

为了得到两个子集合的并，只需将其中一个子集合根结点的双亲指针指向另一个集合的根结点。因此，$S_1 \cup S_2$ 可以具有如图 5.30 所示的表示。

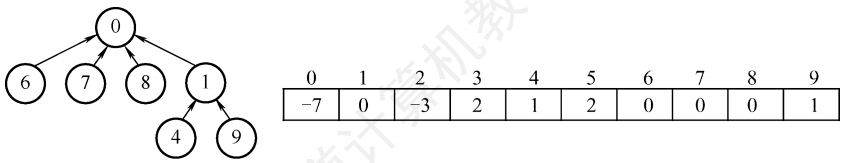


图 5.30　$S_1 \cup S_2$ 可能的表示方法

配套课程公众号[考机研]
永久联系微信:4550060

在采用树的双亲指针数组表示作为并查集的存储表示时，集合元素的编号从 0 到 SIZE-1。其中 SIZE 是最大元素的个数。

3．并查集的基本实现

并查集的结构定义如下：

```
#define SIZE 100
int     UFSets[SIZE];                    //集合元素数组（双亲指针数组）
```

下面是并查集主要运算的实现。

（1）并查集的初始化操作

```
void Initial(int S[]){                   //S 即并查集
    for(int i=0;i<SIZE;i++)              //每个自成单元素集合
        S[i]=-1;
}
```

（2）并查集的 Find 操作

在并查集 S 中查找并返回包含元素 x 的树的根。

```
int Find(int S[],int x){
    while(S[x]>=0)                       //循环寻找 x 的根
        x=S[x];
    return  x;                           //根的 S[]小于 0
}
```

判断两个元素是否属于同一集合，只需分别找到它们的根，再比较根是否相同即可。

（3）并查集的 Union 操作

求两个不相交子集合的并集。若将两个元素所在的集合合并为一个集合，则需要先找到两个元素的根，再令一棵子集树的根指向另一棵子集树的根。

```
void Union(int S[],int Root1,int Root2){
    if(Root1==Root2) return;             //要求 Root1 与 Root2 是不同的集合
    S[Root2]=Root1;                      //将根 Root2 连接到另一根 Root1 下面
}
```

Find 操作和 Union 操作的时间复杂度分别为 $O(d)$和 $O(1)$，其中 d 为树的深度。

4．并查集实现的优化

在极端情况下，n 个元素构成的集合树的深度为 n，则 Find 操作的最坏时间复杂度为 $O(n)$。改进的办法是：在做 Union 操作之前，首先判别子集中的成员数量，然后令成员少的根指向成员多的根，即把小树合并到大树，为此可令根结点的绝对值保存集合树中的成员数量。

（1）改进的 Union 操作

```
void Union(int S[],int Root1,int Root2){
    if(Root1==Root2) return;
    if(S[Root2]>S[Root1]){               //Root2 结点数更少
        S[Root1]+=S[Root2];              //累加集合树的结点总数
        S[Root2]=Root1;                  //小树合并到大树
    }
    else{                                //Root1 结点数更少
        S[Root2]+=S[Root1];              //累加结点总数
        S[Root1]=Root2;                  //小树合并到大树
    }
}
```

采用这种方法构造得到的集合树，其深度不超过$\lfloor \log_2 n \rfloor + 1$。

随着子集逐对合并，集合树的深度越来越大，为了进一步减少确定元素所在集合的时间，还可进一步对上述 Find 操作进行优化，当所查元素 x 不在树的第二层时，在算法中增加一个压缩

配套课程公众号[考机研]
永久联系微信:4550060

路径的功能，即将从根到元素 x 路径上的所有元素都变成根的孩子。

（2）改进的 Find 操作

```
int Find(int S[],int x){
    int root=x;
    while(s[root]>=0)                    //循环找到根
        root=s[root];
    while(x!=root){                      //压缩路径
        int t=S[x];                      //t 指向 x 的父结点
        S[x]=root;                       //x 直接挂到根结点下面
        x=t;
    }
    return root;                         //返回根结点编号
}
```

通过 Find 操作的压缩路径优化后，可使集合树的深度不超过 $O(\alpha(n))$，其中 $\alpha(n)$是一个增长极其缓慢的函数，对于常见的正整数 n，通常 $\alpha(n)\leqslant 4$。

并查集应用的相关举例见本书配套课程。

5.5.3 本节试题精选

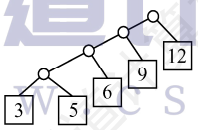
一、单项选择题

01. 在有 n 个叶结点的哈夫曼树中，非叶结点的总数是（ ）。

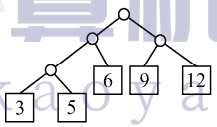
A. $n-1$　　B. n　　C. $2n-1$　　D. $2n$

02. 给定整数集合{3, 5, 6, 9, 12}，与之对应的哈夫曼树是（ ）。

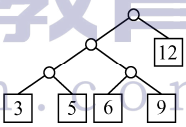
A.



B.



C.



D.

03. 下列编码中，（ ）不是前缀码。

A. {00, 01, 10, 11}　　B. {0, 1, 00, 11}

C. {0, 10, 110, 111}　　D. {10, 110, 1110, 1111}

04. 设哈夫曼编码的长度不超过 4，若已对两个字符编码为 1 和 01，则还最多可对（ ）个字符编码。

A. 2　　B. 3　　C. 4　　D. 5

05. 一棵哈夫曼树共有 215 个结点，对其进行哈夫曼编码，共能得到（ ）个不同的码字。

A. 107　　B. 108　　C. 214　　D. 215

06. 设某哈夫曼树有 5 个叶结点，则该哈夫曼树的高度最高可以是（ ）。

A. 3　　B. 4　　C. 5　　D. 6

07. 以下对于哈夫曼树的说法中，错误的是（ ）

A. 用一组权值构造出的哈夫曼树可能不唯一，但带权路径长度唯一

B. 哈夫曼树具有最小的带权路径长度

C. 哈夫曼树中没有度为 1 的结点

D. 哈夫曼树中除了度为 1 的结点，还有度为 2 的结点和叶结点

08. 下列关于哈夫曼树的说法中，错误的是（ ）。

I. 哈夫曼树的总结点数不能是偶数

配套课程公众号[考机研]
永久联系微信:4550060

II. 哈夫曼树中度为 1 的结点数等于度为 2 和 0 的结点数之差

III. 哈夫曼树的带权路径长度等于其所有分支结点的权值之和

A. 仅 III　　B. I 和 II　　C. 仅 II　　D. I、II 和 III

09. 若度为 m 的哈夫曼树中，叶结点个数为 n，则非叶结点的个数为（　）。

A. $n-1$　　B. $\lfloor n/m \rfloor - 1$

C. $\lceil (n-1)/(m-1) \rceil$　　D. $\lceil n/(m-1) \rceil - 1$

10. 并查集的结构是一种（　）。

A. 二叉链表存储的二叉树　　B. 双亲表示法存储的树

C. 顺序存储的二叉树　　D. 孩子表示法存储的树

11. 并查集中最核心的两个操作是：①查找，查找两个元素是否属于同一个集合；②合并，若两个元素不属于同一个集合，且所在的两个集合互不相交，则合并这两个集合。假设初始长度为 10（0～9）的并查集，按 1-2、3-4、5-6、7-8、8-9、1-8、0-5、1-9 的顺序进行查找和合并操作，最终并查集共有（　）个集合。

A. 1　　B. 2　　C. 3　　D. 4

12. 下列关于并查集的说法中，正确的是（　）（注，本题涉及图的考点）。

A. 并查集不能检测图中是否存在环路的问题

B. 通过路径优化后的并查集在最坏情况下的高度仍是 $O(n)$

C. Find 操作返回集合中元素个数的相反数，它用来作为某个集合的标志

D. Union 操作时可根据当前集合的规模，将小集合合并到大集合中

13. 下列关于并查集的叙述中，（　）是错误的（注，本题涉及图的考点）。

A. 并查集是用双亲表示法存储的树

B. 并查集可用于实现克鲁斯卡尔算法

C. 并查集可用于判断无向图的连通性

D. 在长度为 n 的并查集中进行查找操作的时间复杂度为 $O(\log_2 n)$

14. 【2010 统考真题】n（$n \geq 2$）个权值均不相同的字符构成哈夫曼树，关于该树的叙述中，错误的是（　）。

A. 该树一定是一棵完全二叉树

B. 树中一定没有度为 1 的结点

C. 树中两个权值最小的结点一定是兄弟结点

D. 树中任意一个非叶结点的权值一定不小于下一层任意一个结点的权值

15. 【2014 统考真题】5 个字符有如下 4 种编码方案，不是前缀编码的是（　）。

A. 01,0000,0001,001,1　　B. 011,000,001,010,1

C. 000,001,010,011,100　　D. 0,100,110,1110,1100

16. 【2015 统考真题】下列选项给出的是从根分别到达两个叶结点路径上的权值序列，能属于同一棵哈夫曼树的是（　）。

A. 24, 10, 5 和 24, 10, 7　　B. 24,10, 5 和 24, 12, 7

C. 24, 10, 10 和 24, 14, 11　　D. 24, 10, 5 和 24, 14, 6

17. 【2017 统考真题】已知字符集{a, b, c, d, e, f, g, h}，若各字符的哈夫曼编码依次是 0100, 10, 0000, 0101, 001, 011, 11, 0001，则编码序列 0100011001001011110101 的译码结果是（　）。

A. a c g a b f h　　B. a d b a g b b　　C. a f b e a g d　　D. a f e e f g d

18. 【2018 统考真题】已知字符集{a, b, c, d, e, f}，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，

王道计算机教育 www.kaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

则对应字符集中各字符的哈夫曼编码可能是（ ）。

A. 00, 1011, 01, 1010, 11, 100　　B. 00, 100, 110, 000, 0010, 01

C. 10, 1011, 11, 0011, 00, 010　　D. 0011, 10, 11, 0010, 01, 000

19.【2019统考真题】对 n 个互不相同的符号进行哈夫曼编码。若生成的哈夫曼树共有115个结点，则 n 的值是（ ）。

A. 56　　B. 57　　C. 58　　D. 60

20.【2021统考真题】若某二叉树有5个叶结点，其权值分别为10, 12, 16, 21, 30，则其最小的带权路径长度（WPL）是（ ）。

A. 89　　B. 200　　C. 208　　D. 289

21.【2022统考真题】对任意给定的含 n（$n>2$）个字符的有限集 S，用二叉树表示 S 的哈夫曼编码集和定长编码集，分别得到二叉树 T_1 和 T_2。下列叙述中，正确的是（ ）。

A. T_1 与 T_2 的结点数相同　　B. T_1 的高度大于 T_2 的高度

C. 出现频次不同的字符在 T_1 中处于不同的层

D. 出现频次不同的字符在 T_2 中处于相同的层

22.【2023统考真题】在由6个字符组成的字符集 S 中，各字符出现的频次分别为3, 4, 5, 6, 8, 10，为 S 构造的哈夫曼编码的加权平均长度为（ ）。

A. 2.4　　B. 2.5　　C. 2.67　　D. 2.75

二、综合应用题

01. 设给定权集 $w=\{5, 7, 2, 3, 6, 8, 9\}$，试构造关于 w 的一棵哈夫曼树，并求其加权路径长度WPL。

02.【2012统考真题】设有6个有序表A, B, C, D, E, F，分别含有10, 35, 40, 50, 60和200个数据元素，各表中的元素按升序排列。要求通过5次两两合并，将6个表最终合并为1个升序表，并使最坏情况下比较的总次数达到最小。请回答下列问题：

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 n（$n\geq 2$）个不等长升序表的合并策略，并说明理由。

03.【2020统考真题】若任意一个字符的编码都不是其他字符编码的前缀，则称这种编码具有前缀特性。现有某字符集（字符个数≥2）的不等长编码，每个字符的编码均为二进制的0、1序列，最长为 L 位，且具有前缀特性。请回答下列问题：

1）哪种数据结构适宜保存上述具有前缀特性的不等长编码？

2）基于你所设计的数据结构，简述从0/1串到字符串的译码过程。

3）简述判定某字符集的不等长编码是否具有前缀特性的过程。

5.5.4 答案与解析

一、单项选择题

01. A

由哈夫曼树的构造过程可知，哈夫曼树中只有度为0和2的结点。在非空二叉树中，有 $n_0=n_2+1$，所以 $n_2=n-1$。

【另解】n 个结点构造哈夫曼树需要 $n-1$ 次合并过程，每次合并新建一个分支结点，所以选择选项A。

02. C

配套课程公众号[考机研]
永久联系微信:4550060

首先，3 和 5 构造为一棵子树，其根权值为 8，然后该子树与 6 构造为一棵新子树，根权值为 14，再后 9 与 12 构造为一棵子树，最后两棵子树共同构造为一棵哈夫曼树。

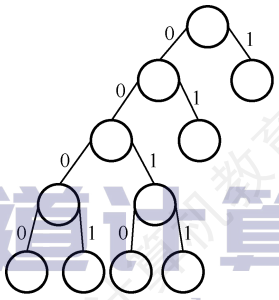
03．B

若没有一个编码是另一个编码的前缀，则称这样的编码为前缀编码。在选项 B 中，0 是 00 的前缀，1 是 11 的前缀。

04．C

在哈夫曼编码中，一个编码不能是任何其他编码的前缀。3 位编码可能是 001，对应的 4 位编码只能是 0000 和 0001。3 位编码也可能是 000，对应的 4 位编码只能是 0010 和 0011。若全采用 4 位编码，则可以为 0000, 0001, 0010 和 0011。题中问的是最多，所以选择选项 C。

【另解】若哈夫曼编码的长度只允许小于或等于 4，则哈夫曼树的高度最高是 5，已知一个字符编码为 1，另一个字符编码是 01，这说明第二层和第三层各有一个叶结点，为使得该树从第 3 层起能够对尽可能多的字符编码，余下的二叉树应该是满二叉树，如下图所示，底层可以有 4 个叶结点，最多可以再对 4 个字符编码。



王道计算机教育
www.cskaoyan.com

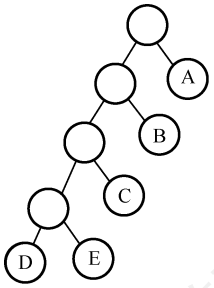
05．B

根据上题的结论，叶结点数为(215 + 1)/2 = 108，所以共有 108 个不同的码字。

【另解】在哈夫曼树中只有度为 0 和 2 的结点，结点总数 $n = n_0 + n_2$，且 $n_0 = n_2 + 1$，由题知 $n = 215$，$n_0 = 108$。

06．C

在哈夫曼树的构造中，每个初始结点最终都成为叶结点，5 个初始结点构造的哈夫曼树共新建 4 个双分支结点，4 个双分支结点所构成的高度最高的哈夫曼树如下图所示，其高度是 5。



07．D

在哈夫曼树的构造过程中，每次选根的权值最小的两棵树，一棵作为左子树，一棵作为右子树，生成新的二叉树，新的二叉树根的权值应为其左右两棵子树根结点权值的和。至于谁做左子树，谁做右子树，没有限制，所以构造的哈夫曼树是不唯一的，但其带权路径长度是最小的和唯一的。哈夫曼树只有度为 0 和 2 的结点，度为 0 的结点是外结点，带有权值，没有度为 1 的结点。

配套课程公众号[考机研]
永久联系微信:4550060

08. C

n 个初始结点构造的哈夫曼树共新建 $n-1$ 个双分支结点，因此哈夫曼树的结点总数是 $2n-1$，是个奇数，选项 I 正确。哈夫曼树中没有度为 1 的结点，选项 II 错误。哈夫曼的带权路径长度有两种计算方法：①所有叶结点的带权路径长度之和；②所有分支结点的权值之和，选项 III 正确。

09. C

一棵度为 m 的哈夫曼树应只有度为 0 和 m 的结点，设度为 m 的结点有 n_m 个，度为 0 的结点有 n_0 个，又设结点总数为 N，$N=n_0+n_m$。因有 N 个结点的哈夫曼树有 $N-1$ 条分支，则 $mn_m=N-1=n_m+n_0-1$，整理得 $(m-1)n_m=n_0-1$，$n_m=(n_0-1)/(m-1)$。

10. B

并查集的存储结构是用双亲表示法存储的树，主要是为了方便两个重要的操作。

11. C

初始时，0～9 各自成一个集合。查找 1-2 时，合并{1}和{2}；查找 3-4 时，合并{3}和{4}；查找 5-6 时，合并{5}和{6}；查找 7-8 时，合并{7}和{8}；查找 8-9 时，合并{7, 8}和{9}；查找 1-8 时，合并{1, 2}和{7, 8, 9}；查找 0-5 时，合并{0}和{5, 6}；查找 1-9 时，它们属于同一个集合。最终的集合为{0, 5, 6}、{1, 2, 7, 8, 9}和{3, 4}，因此答案选择选项 C。

12. D

依次探测图的各条边，用并查集检查该边依附的两个顶点是否已属于同一集合（两个顶点的根结点是否相同）。若是，则说明图中存在环路，选项 A 错误。经过路径优化后，并查集在最坏情况下的高度远小于 $O(n)$，选项 B 错误。`Find` 操作总返回当前根结点作为集合的标志，选项 C 错误。

13. D

在用并查集实现 Kruskal 算法求图的最小生成树时：判断是否加入一条边之前，先查找这条边关联的两个顶点是否属于同一个集合（判断加入这条边之后是否形成回路），若形成回路，则继续判断下一条边；若不形成回路，则将该边和边对应的顶点加入最小生成树 T，并继续判断下一条边，直到所有顶点都已加入最小生成树 T。选项 B 正确。用并查集判断无向图连通性的方法：遍历无向图的边，每遍历到一条边，就把这条边连接的两个顶点合并到同一个集合中，处理完所有边后，只要是相互连通的顶点都会被合并到同一个子集合中，相互不连通的顶点一定在不同的子集合中。选项 C 正确。未做路径优化的并查集在最坏情况下的高度为 n，此时查找操作的时间复杂度为 $O(n)$，时间复杂度通常指最坏情况下的时间复杂度。选项 D 错误。

14. A

哈夫曼树为带权路径长度最小的二叉树，不一定是完全二叉树。哈夫曼树中没有度为 1 的结点，选项 B 正确。构造哈夫曼树时，最先选取两个权值最小的结点作为左、右子树构造一棵新的二叉树，选项 C 正确。哈夫曼树中任意一个非叶结点的权值为其左、右子树根结点的权值之和，可知，哈夫曼树中任意一个非叶结点的权值一定不小于下一层任意一个结点的权值。

15. D

前缀编码的定义是在一个字符集中，任何一个字符的编码都不是另一个字符编码的前缀。选项 D 中的编码 110 是编码 1100 的前缀，违反了前缀编码的规则，所以选项 D 不是前缀编码。

16. D

在哈夫曼树中，左右孩子权值之和为父结点权值。仅以分析选项 A 为例：若两个 10 分别属于两棵不同的子树，则根的权值不等于其孩子的权值和，不符；若两个 10 属同棵子树，则其权值不等于其两个孩子（叶结点）的权值和，不符。选项 B、C 选项的排除方法相同。

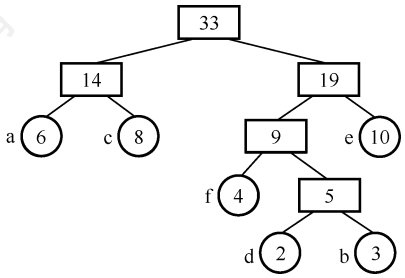
王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

17．D

哈夫曼编码是前缀编码，各个编码的前缀不同，因此直接拿编码序列与哈夫曼编码一一比对即可。序列可分割为 0100 011 001 001 011 11 0101，译码结果是 afeefgd。选项 D 正确。

18．A

根据各字符出现的次数构造的哈夫曼树如下图所示。由图可知，a、c 和 e 的编码长度应该相同；a 和 c 的第 1 个编码应该相同，且与 e 的第 1 个编码不同；b 和 d 的前 3 个编码应该相同。

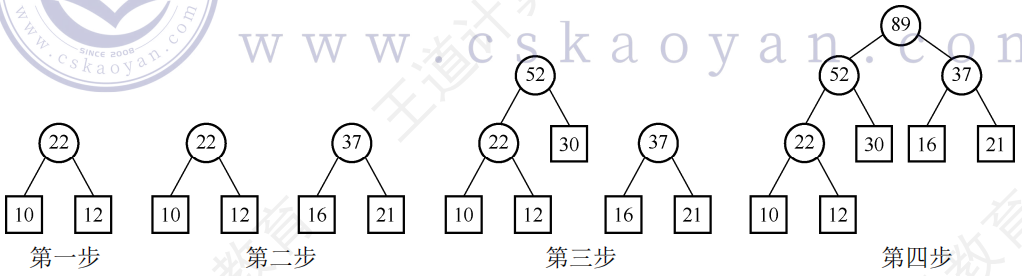


19．C

n 个符号构造成哈夫曼树的过程中，共新建了 $n-1$ 个结点（双分支结点），因此哈夫曼树的结点总数为 $2n-1=115$，n 的值为 58。

20．B

对于带权值的结点，构造出哈夫曼树的带权路径长度（WPL）最小，哈夫曼树的构造过程如下图所示。求得其 WPL = (10 + 12)×3 + (30 + 16 + 21)×2 = 200。



21．D

可以画一个简单的特例来证明。图 1 是满足条件的二叉树 T_1，图 2 是满足条件的二叉树 T_2，结点中有值表示这个结点是编码字符。T_1 和 T_2 的结点数不同，选项 A 错误。T_1 的高度等于 T_2 的高度，选项 B 错误。出现频次不同的字符在 T_1 中也可能处于相同的层，选项 C 错误。对于定长编码集，所有字符一定都在 T_2 中处于相同的层，而且都是叶结点。

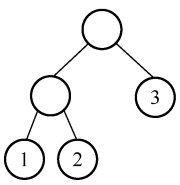


图 1

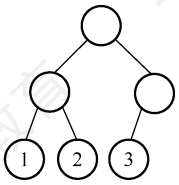
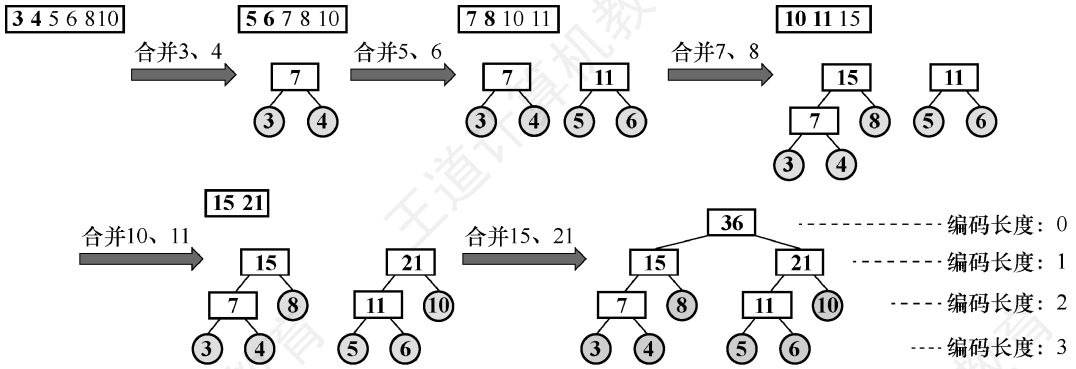


图 2

22．B

构建哈夫曼树的过程如下图所示。

配套课程公众号[考机研]
永久联系微信:4550060

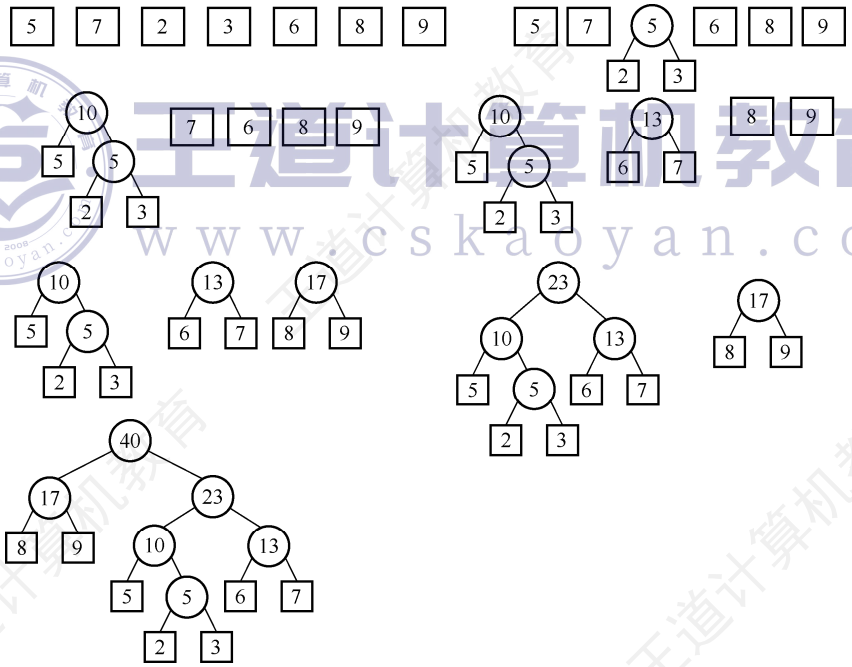


对叶结点的哈夫曼编码，共有 4 个长度为 3 的叶结点、2 个长度为 2 的叶结点，编码的加权平均长度为[(3+4+5+6)×3+(8+10)×2]/(3+4+5+6+8+10)=2.5。

二、综合应用题

01.【解答】

根据哈夫曼树的构造方法，每次从森林中选取两个根结点值最小的树合并成一棵树，将原先的两棵树作为左、右子树，且新根结点的值为左、右孩子关键字之和。构造过程如下图所示。



由构造出的哈夫曼树可得 WPL=(2+3)×4+(5+6+7)×3+(8+9)×2=108。

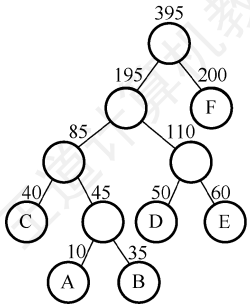
注 意

哈夫曼树并不唯一，但带权路径长度一定是相同的。

02.【解答】

1）最先合并的表中的元素在后续的每次合并中都会再次参与比较，因此求最小合并次数类似于求最小带权路径长度，此时可立即想到哈夫曼树。根据哈夫曼树的构造过程，每次选择表集合中长度最小的两个表进行合并。6 个表的合并顺序如下图所示。

配套课程公众号[考机研]
永久联系微信:4550060



根据图中的哈夫曼树，6 个序列的合并过程如下：

① 在表集合{10, 35, 40, 50, 60, 200}中，选择表 A 与表 B 合并，生成含 45 个元素的表 AB。

② 在表集合{40, 45, 50, 60, 200}中，将表 AB 与表 C 合并，生成含 85 个元素的表 ABC。

③ 在表集合{50, 60, 85, 200}中，表 D 与表 E 合并，生成含 110 个元素的表 DE。

④ 在表集合{85, 110, 200}中，表 ABC 与表 DE 合并，生成含 195 个元素的表 ABCDE。

⑤ 当前表集合为{195, 200}，表 ABCDE 与表 F 合并，生成含 395 个元素的表 ABCDEF。

因为合并两个长度分别为 m 和 n 的有序表，最坏情况下需要比较 $m+n-1$ 次，所以最坏情况下比较的总次数计算如下：

第 1 次合并：最多比较次数 = 10 + 35 – 1 = 44。

第 2 次合并：最多比较次数 = 40 + 45 – 1 = 84。

第 3 次合并：最多比较次数 = 50 + 60 – 1 = 109。

第 4 次合并：最多比较次数 = 85 + 110 – 1 = 194。

第 5 次合并：最多比较次数 = 195 + 200 – 1 = 394。

比较的总次数最多为 44 + 84 + 109 + 194 + 394 = 825。

2）各表的合并策略是：对多个有序表进行两两合并时，若表长不同，则最坏情况下总的比较次数依赖于表的合并次序。可以借助于哈夫曼树的构造思想，依次选择最短的两个表进行合并，此时可以获得最坏情况下的最佳合并效率。

03.【解答】

1）使用一棵二叉树保存字符集中各字符的编码，每个编码对应于从根开始到达某叶结点的一条路径，路径长度等于编码位数，路径到达的叶结点中保存该编码对应的字符。

2）从左至右依次扫描 0/1 串中的各位。从根开始，根据串中当前位沿当前结点的左子指针或右子指针下移，直到移动到叶结点时为止。输出叶结点中保存的字符。然后从根开始重复这个过程，直到扫描到 0/1 串结束，译码完成。

3）二叉树既可用于保存各字符的编码，又可用于检测编码是否具有前缀特性。判定编码是否具有前缀特性的过程，也是构建二叉树的过程。初始时，二叉树中仅含有根结点，其左子指针和右子指针均为空。

依次读入每个编码 C，建立/寻找从根开始对应于该编码的一条路径，过程如下：

对每个编码，从左至右扫描 C 的各位，根据 C 的当前位（0 或 1）沿结点的指针（左子指针或右子指针）向下移动。当遇到空指针时，创建新结点，让空指针指向该新结点并继续移动。沿指针移动的过程中，可能遇到三种情况：

① 若遇到了叶结点（非根），则表明不具有前缀特性，返回。

② 若在处理 C 的所有位的过程中，均没有创建新结点，则表明不具有前缀特性，返回。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

③ 若在处理 C 的最后一个编码位时创建了新结点，则继续验证下一个编码。

若所有编码均通过验证，则编码具有前缀特性。

归纳总结

本章的内容较多，其中二叉树是极其重要的考查点。关于二叉树的有关操作，在 2014 年的统考中首次出现了线性表以外的算法设计题，需要引起读者的注意。

遍历是二叉树的各种操作的基础，统考时会考查遍历过程中对结点的各种其他操作，而且容易结合递归算法和利用栈或队列的非递归算法。读者需重点掌握各种遍历方法的代码书写，并学会在遍历的基础上，进行一些其他的相关操作。其中递归算法短小精悍，出现的概率较大，请读者不要掉以轻心，要做到对几种遍历方式的程序模板烂熟于心，并结合一定数量的习题，才可以在考试中快速地写出漂亮的代码。

二叉树遍历算法的递归程序：

```
void Track(BiTree *p){
    if(p!=NULL){
        //(1)
        Track(p->lchild);
        //(2)
        Track(p->rchild);
        //(3)
    }
}
```

访问函数 visit() 位于 (1)、(2)、(3) 的位置，分别对应于先序、中序、后序遍历。但对于具体题目来说，设计算法时要灵活应用。请读者认真练习下面的例题。

例题：设二叉树的存储结构为二叉链表，编写有关二叉树的递归算法。

1）统计二叉树中度为 1 的结点个数。

2）统计二叉树中度为 2 的结点个数。

3）统计二叉树中度为 0 的结点个数。

4）统计二叉树的高度。

5）统计二叉树的宽度。

6）从二叉树中删去所有叶结点。

7）计算指定结点*p 所在的层次。

8）计算二叉树中各结点中的最大元素的值。

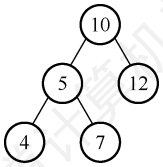
9）交换二叉树中每个结点的两个子女。

10）以先序次序输出一颗二叉树中所有结点的数据值及结点所在的层次。

思维拓展

输入一个整数 data 和一棵二元树。从树的根结点开始往下访问一直到叶结点，所经过的所有结点形成一条路径。打印出路径及与 data 相等的所有路径。例如，输入整数 22 和下图所示的二元树，则打印出两条路径 10, 12 和 10, 5, 7。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060



注　意

使用数组或栈保存访问的路径，并记录当前路径上所有元素的和 sum。若当前结点为叶结点，且当前结点值与 sum 的和等于 data，则满足条件，打印当前路径。然后递归返回到父结点，注意在递归返回之前要先减去当前结点元素的值。使用前序遍历操作的递归算法模板可以简化程序。

王道计算机教育
www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060